

AI 500

Foundations of

Artificial Intelligence

L9: Multi-layer Neural Networks

By Murtaza Taj



Derivatives

Function

- $f(x) = x^n$
- $\frac{d(f(x))}{dx} = nx^{n-1} \frac{d(x)}{dx} = nx^{n-1}$
- $f(x) = x^3$
- $\frac{d(f(x))}{dx} = 3x^{2-1} \frac{d(x)}{dx} = 3x^2$

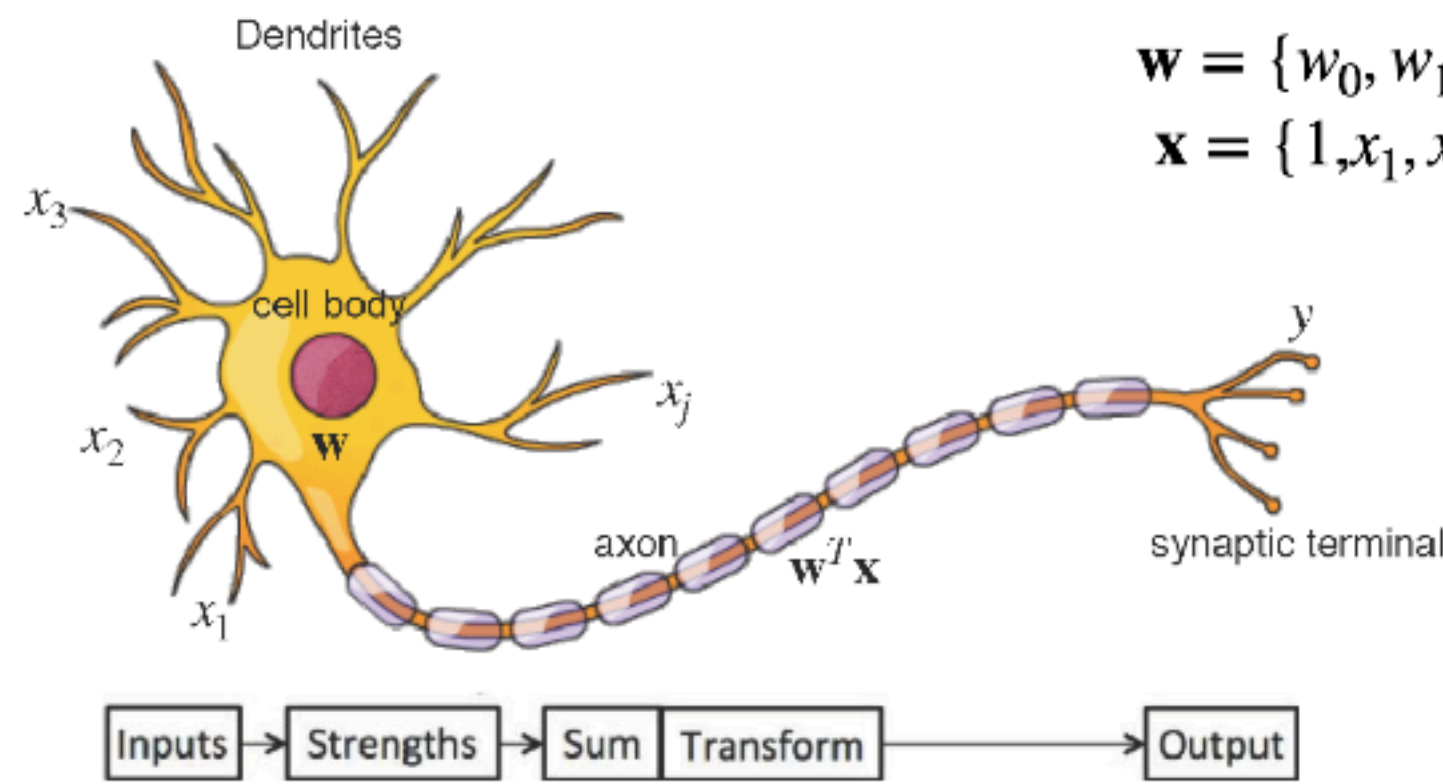
Product Rule

- $f(x) = x^m y^n$
- $f'(x, y) = x^m \frac{\partial(y^n)}{\partial y} + y^n \frac{\partial(x^m)}{\partial x}$
- $f(x, y) = \frac{x^m}{y^n} = x^m y^{-n}$

Chain Rule

- $z = x^m$
- $y = 1 + z^n$
- $\frac{\partial y}{\partial x} = \frac{\partial z}{\partial x} \frac{\partial y}{\partial z}$
- $\frac{\partial y}{\partial x} = (mx^{m-1})(nz^{n-1})$

Biological Neuron



$$\mathbf{w} = \{w_0, w_1, w_2, \dots\}^T$$

$$\mathbf{x} = \{1, x_1, x_2, \dots\}^T$$

Analytical vs. Iterative Solution

$$\mathbf{w} = \{w_0, w_1, w_2, \dots\}^T$$

$$\mathbf{x}^{(i)} = \{1, x_1^{(i)}, x_2^{(i)}, \dots\}^T$$

$$y^{(i)} = \mathbf{w}^T \mathbf{x}^{(i)} = \sum_j w_j x_j^{(i)}$$

$$E = \sum_i (t^{(i)} - y^{(i)})^2 = \sum_i (t^{(i)} - \mathbf{w}^T \mathbf{x}^{(i)})^2$$

Direct/Analytical Approach

$$\frac{\partial E}{\partial w_j} = - \sum_i (t^{(i)} - y^{(i)}) x_j^{(i)}$$

$$A\mathbf{x} = \mathbf{b}$$

Iterative Approach

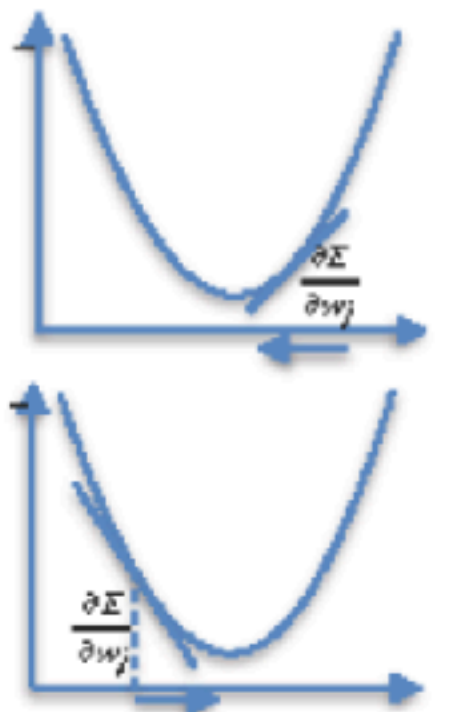
$$w_j \leftarrow w_j - \eta \frac{\partial E}{\partial w_j}$$

$$w_j \leftarrow w_j + \Delta w_j$$

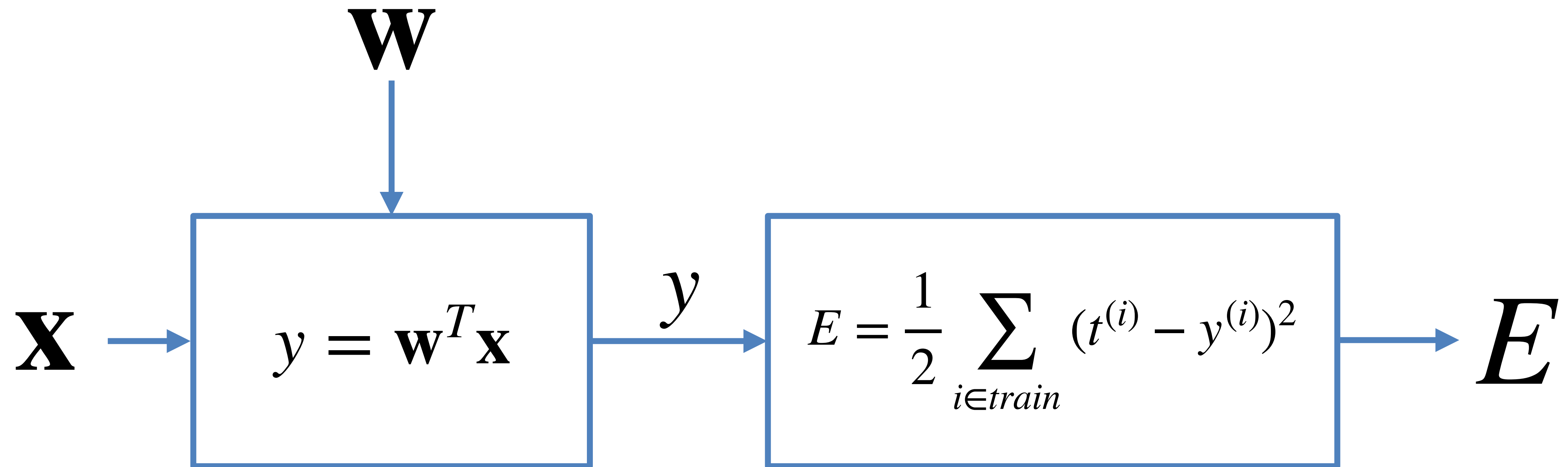
$$\Delta w_j = - \eta \frac{\partial E}{\partial w_j}$$

$$\Delta w_j = \eta (t^{(i)} - y^{(i)}) x_j^{(i)}$$

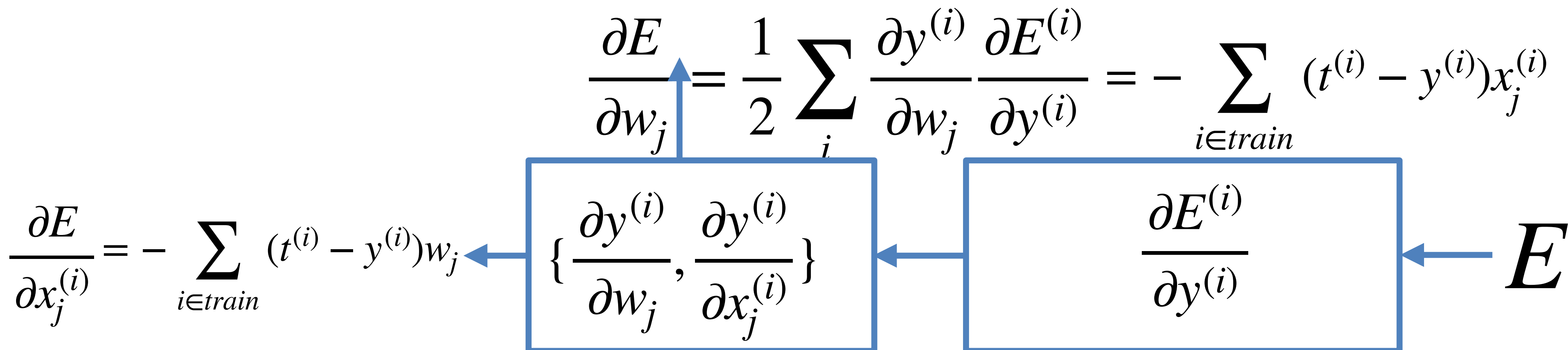
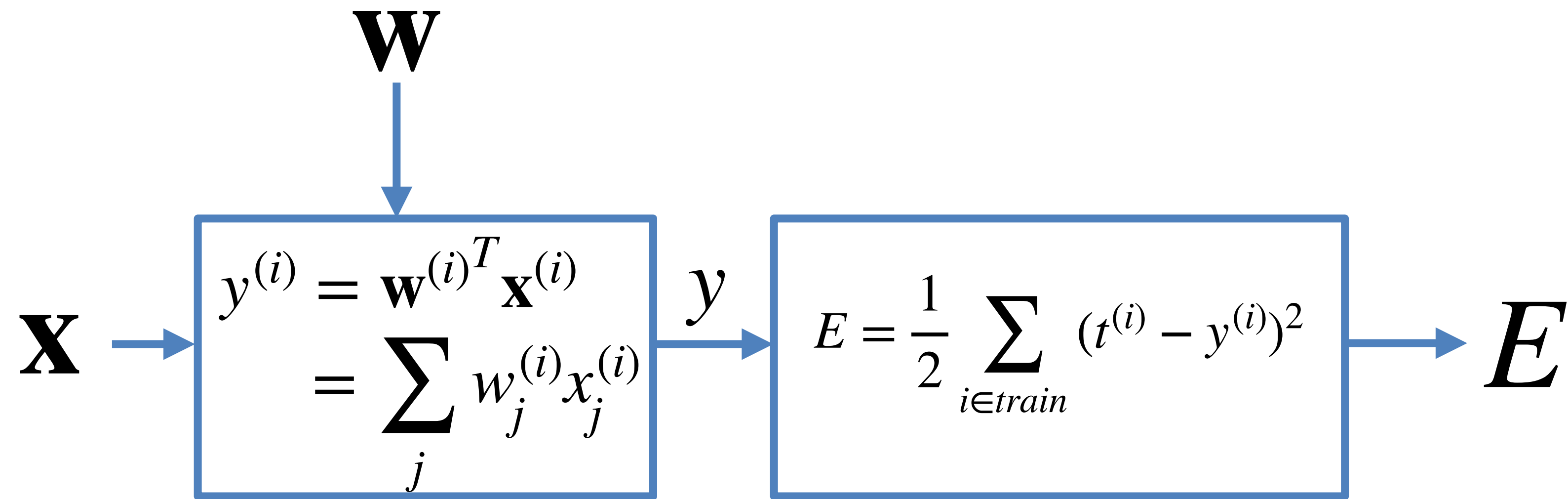
where η is the learning rate



Key Computation: Forward Pass



Key Computation: Forward/Backward Pass



Gradient Descent

- $\Delta w_j \leftarrow \text{Initialize to ZEROS}$
- For each iteration
 - For each training example $(\mathbf{x}^{(i)}, t^{(i)})$
 - For each weight w_j

$$\Delta w_j \leftarrow \Delta w_j + \eta x_j^{(i)} (t^{(i)} - y^{(i)})$$

- For each weight w_j

$$w_j \leftarrow w_j + \Delta w_j$$

$$\Delta w_j = \eta \sum_i x_j^{(i)} (t^{(i)} - y^{(i)})$$

Example: Computational Graph

Computation Graph

- For example here is a simple mathematical equation

$$y = (a + b) \times (b - c)$$

$$y = ab - ac + b^2 - bc$$

Computation Graph

- For example here is a simple mathematical equation

$$y = (a + b) \times (b - c)$$

$$y = ab - ac + b^2 - bc$$

- We want to calculate gradient w.r.t a , b , c

$$\frac{\partial y}{\partial a}, \frac{\partial y}{\partial b}, \frac{\partial y}{\partial c}$$

Computation Graph

- For example here is a simple mathematical equation

$$y = (a + b) \times (b - c)$$

$$y = ab - ac + b^2 - bc$$

- We want to calculate gradient w.r.t a , b , c

$$\frac{\partial y}{\partial a}, \frac{\partial y}{\partial b}, \frac{\partial y}{\partial c}$$

$$\frac{\partial y}{\partial a} = b - c$$

Computation Graph

- For example here is a simple mathematical equation

$$y = (a + b) \times (b - c)$$

$$y = ab - ac + b^2 - bc$$

- We want to calculate gradient w.r.t a , b , c

$$\frac{\partial y}{\partial a}, \frac{\partial y}{\partial b}, \frac{\partial y}{\partial c}$$

$$\frac{\partial y}{\partial a} = b - c$$

$$\frac{\partial y}{\partial b} = a + b + b - c$$

Computation Graph

- For example here is a simple mathematical equation

$$y = (a + b) \times (b - c)$$

$$y = ab - ac + b^2 - bc$$

- We want to calculate gradient w.r.t a , b , c

$$\frac{\partial y}{\partial a}, \frac{\partial y}{\partial b}, \frac{\partial y}{\partial c}$$

$$\frac{\partial y}{\partial a} = b - c$$

$$\frac{\partial y}{\partial b} = a + b + b - c$$

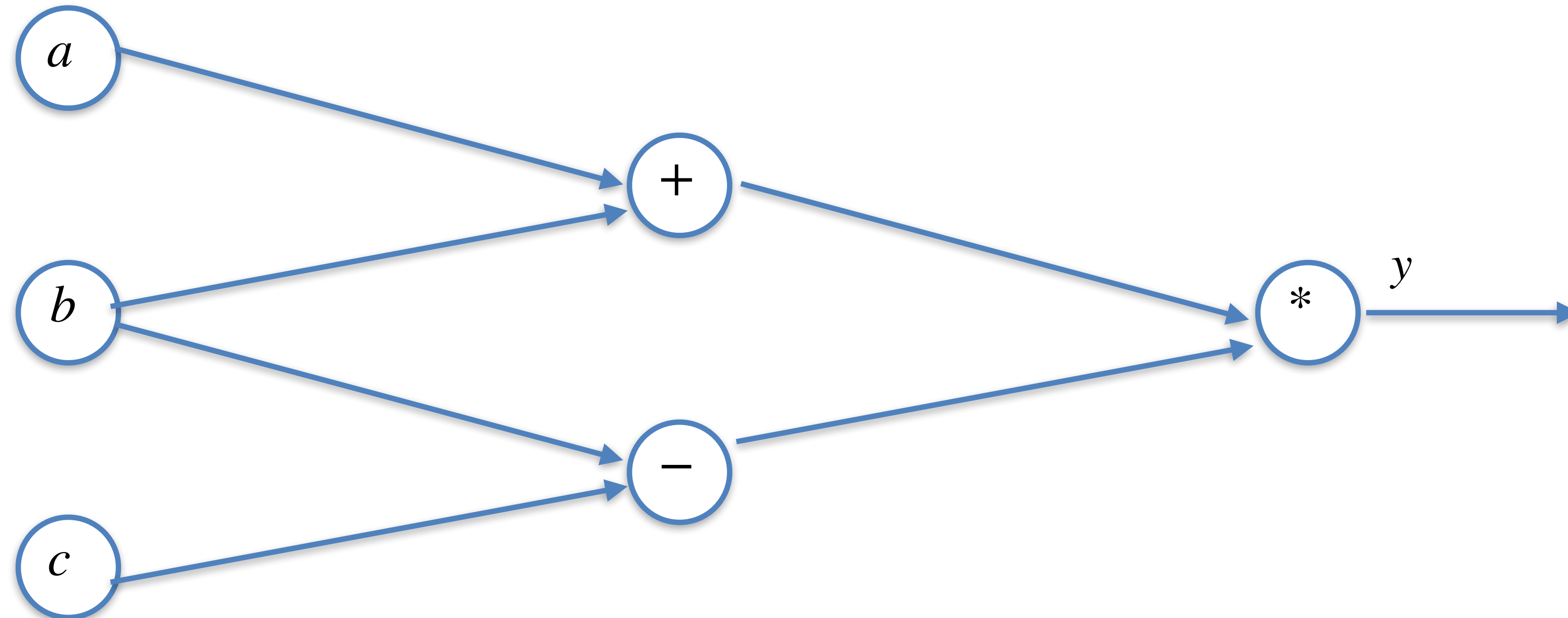
$$\frac{\partial y}{\partial c} = -(a + b)$$

Computation Graph

- For example here is a simple mathematical equation

$$y = (a + b) \times (b - c)$$

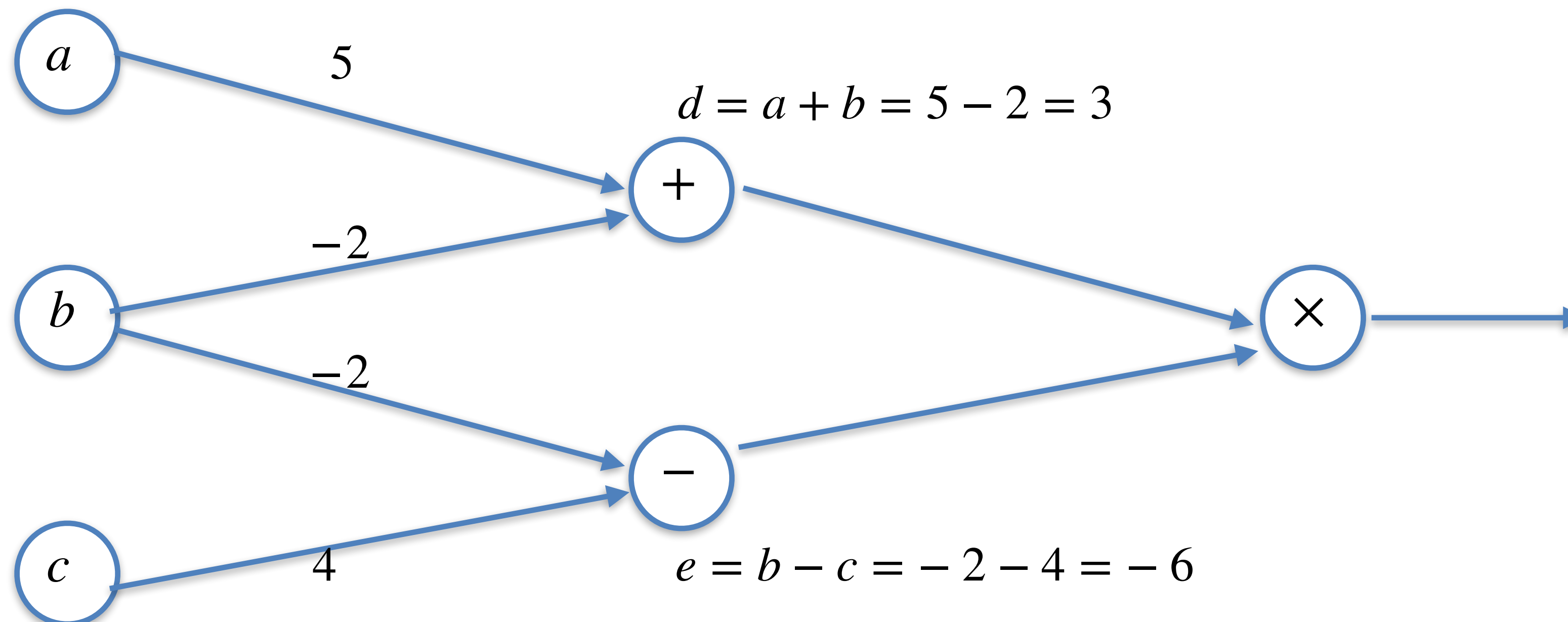
- We can write a computation graph of above equation as follows:



Computation Graph - Forward Pass

$$y = (a + b) \times (b - c)$$

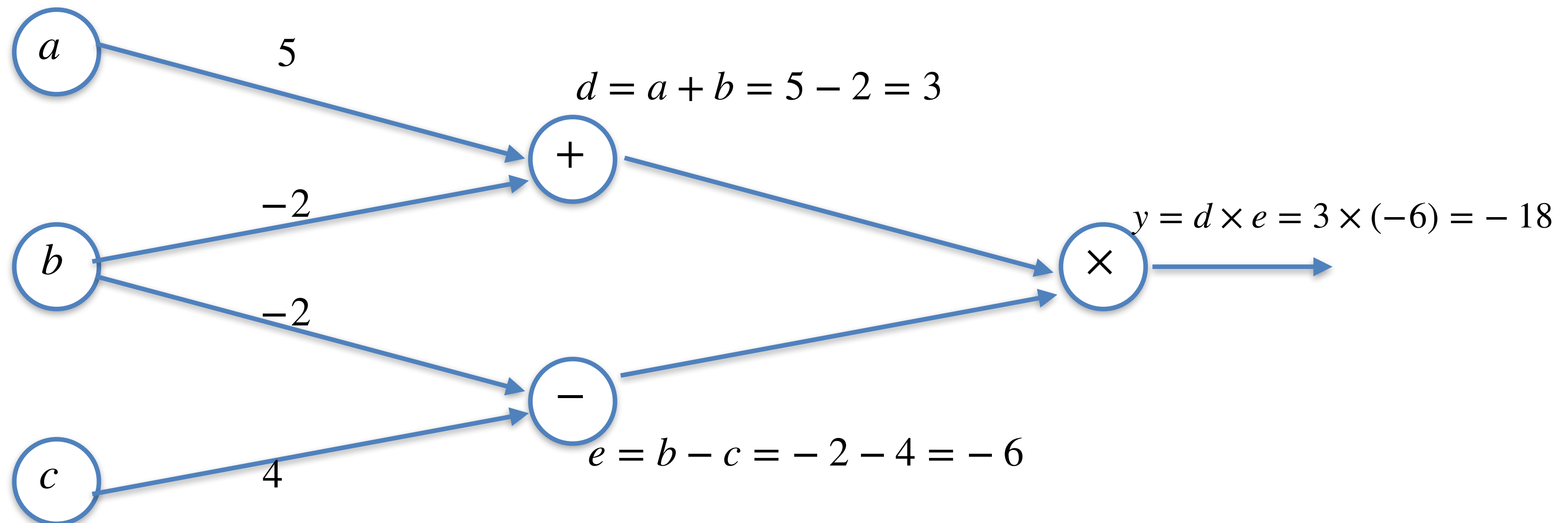
- Let us consider $a = 5$, $b = -2$, $c = 4$
- First we will use these values to compute d and e



Computation Graph - Forward Pass

$$y = (a + b) \times (b - c)$$

- Let us consider $a = 3$, $b = -2$, $c = 4$
- Now we will use d and e to compute y

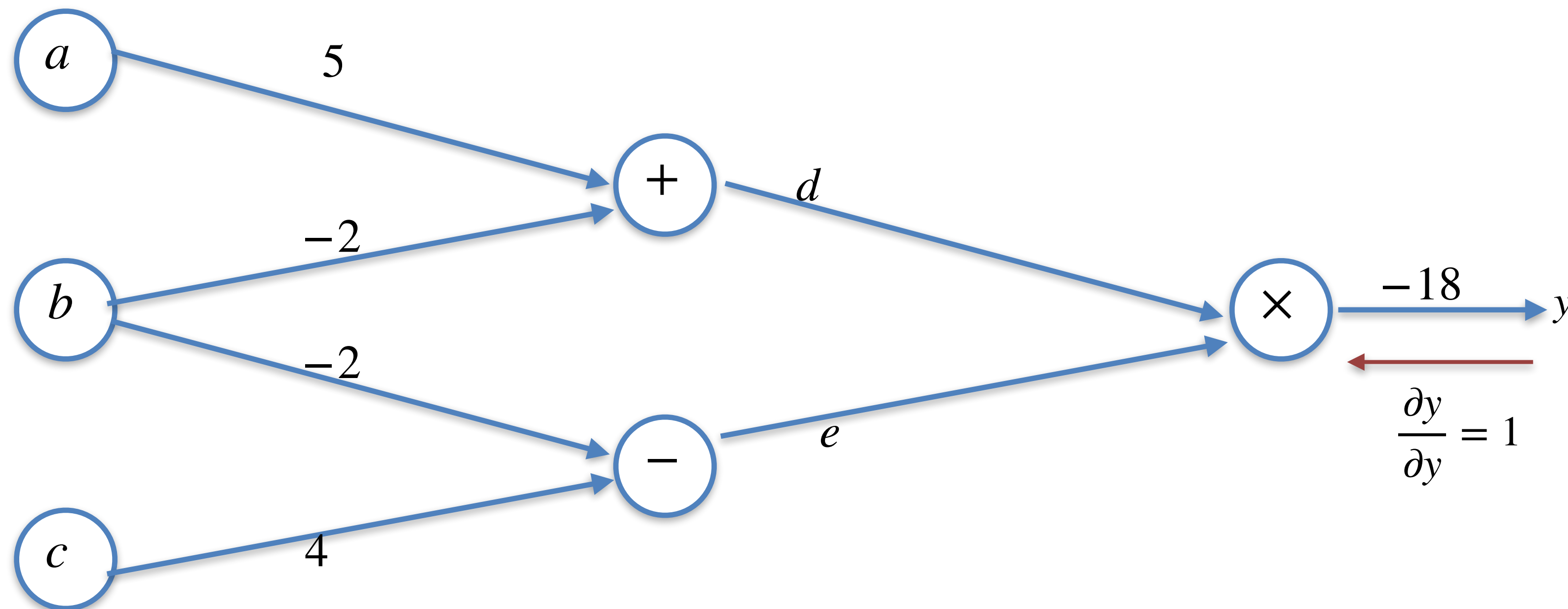


Computation Graph - Backward Pass

- We start the backward pass by finding the derivative of the final output with respect to the final output (itself!).

$$\frac{\partial y}{\partial y} = 1$$

- Our computational graph now looks as shown below

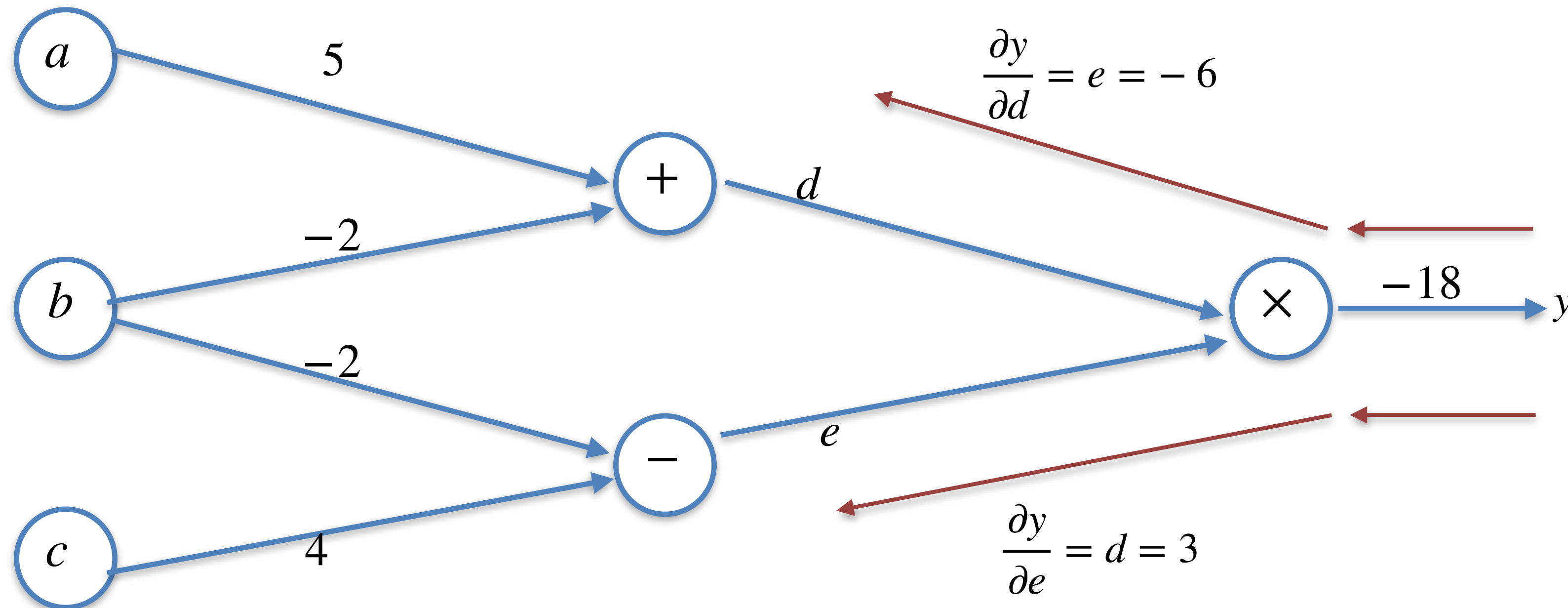


Computation Graph - Backward Pass

- Now given $y = d \times e$

$$\frac{\partial y}{\partial d} = e = b - c$$

$$\frac{\partial y}{\partial e} = d = a + b$$



Computation Graph - Backward Pass

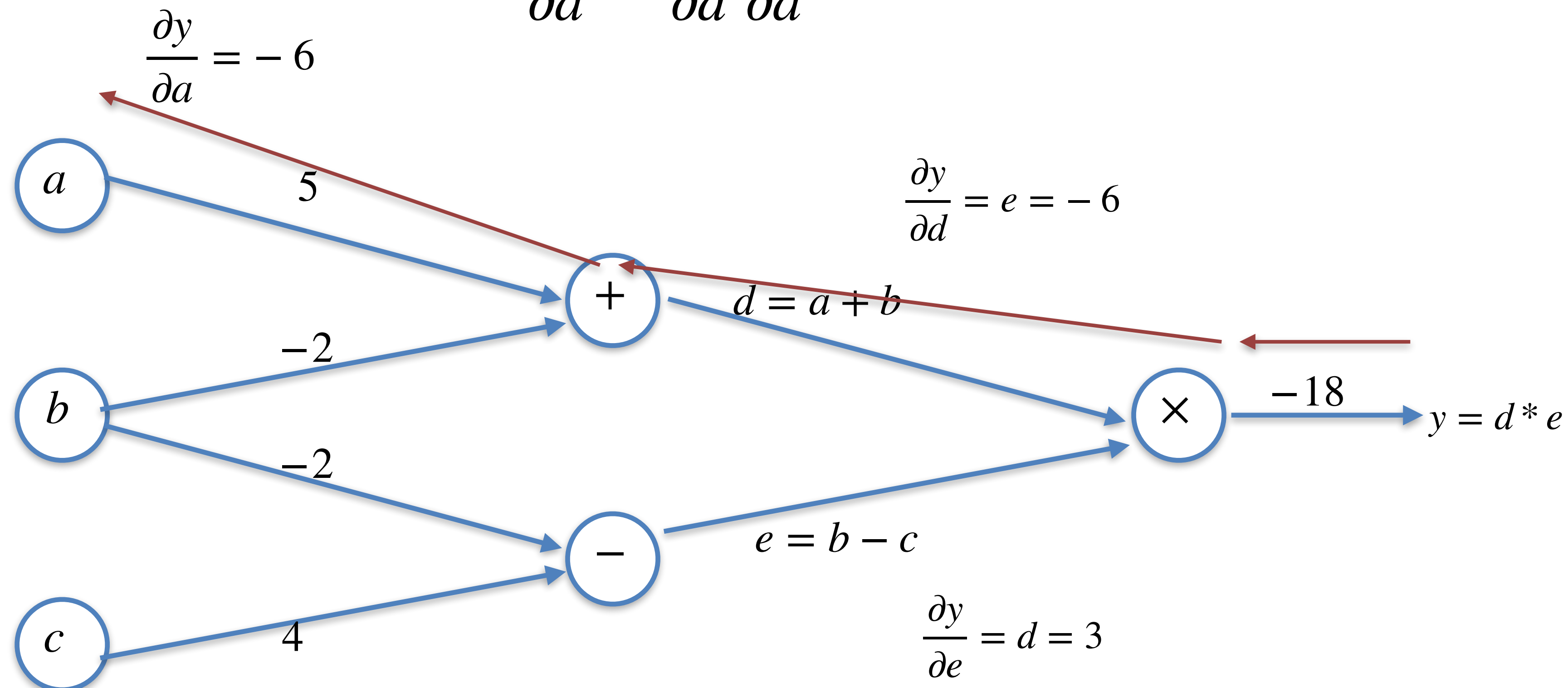
- We want to calculate gradient w.r.t a, b, c

$$y = (a + b) \times (b - c)$$

$$y = d \times e$$

$$\frac{\partial y}{\partial a}, \frac{\partial y}{\partial b}, \frac{\partial y}{\partial c}$$

- From chain rule we have $\frac{\partial y}{\partial a} = \frac{\partial y}{\partial d} \frac{\partial d}{\partial a} = e \times 1 = -6$



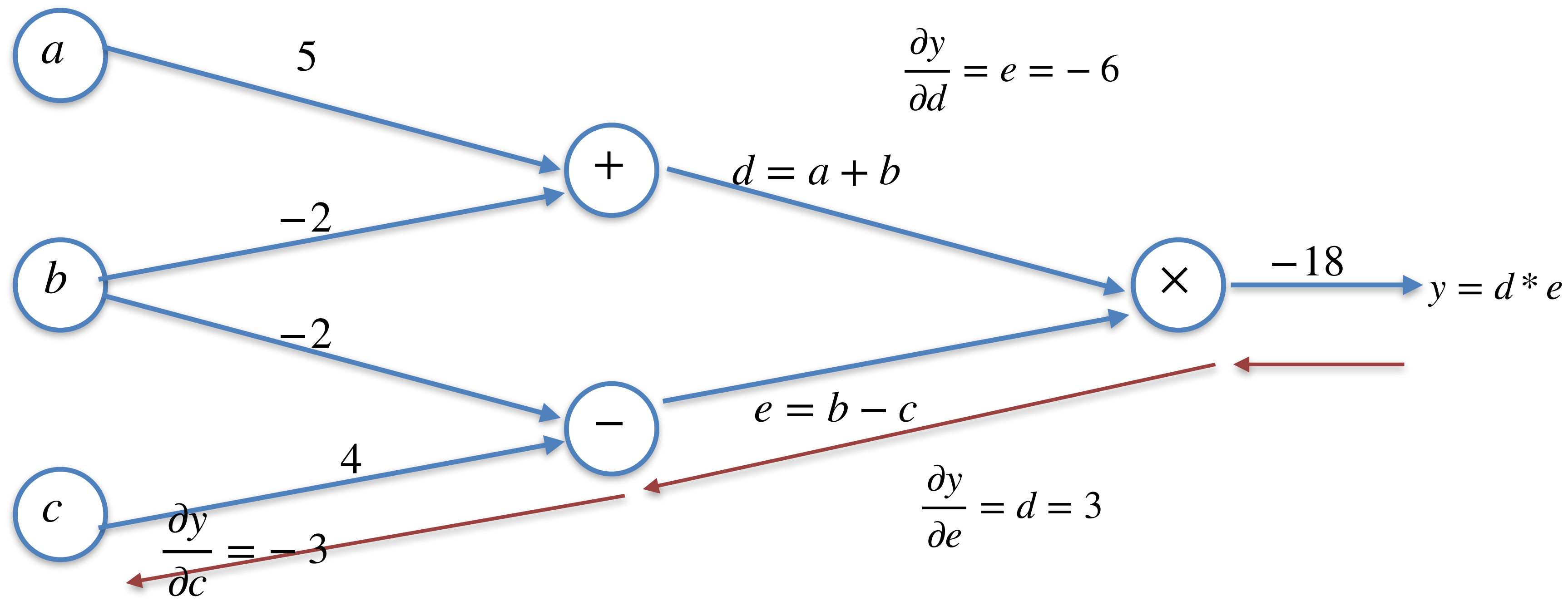
$$y = (a + b) \times (b - c)$$

Computation Graph - Backward Pass

- We want to calculate gradient w.r.t a, b, c

$$\frac{\partial y}{\partial a}, \frac{\partial y}{\partial b}, \frac{\partial y}{\partial c}$$

- From chain rule we have $\frac{\partial y}{\partial c} = \frac{\partial y}{\partial e} \frac{\partial e}{\partial c} = d \times (-1) = -3$



$$y = (a + b) \times (b - c)$$

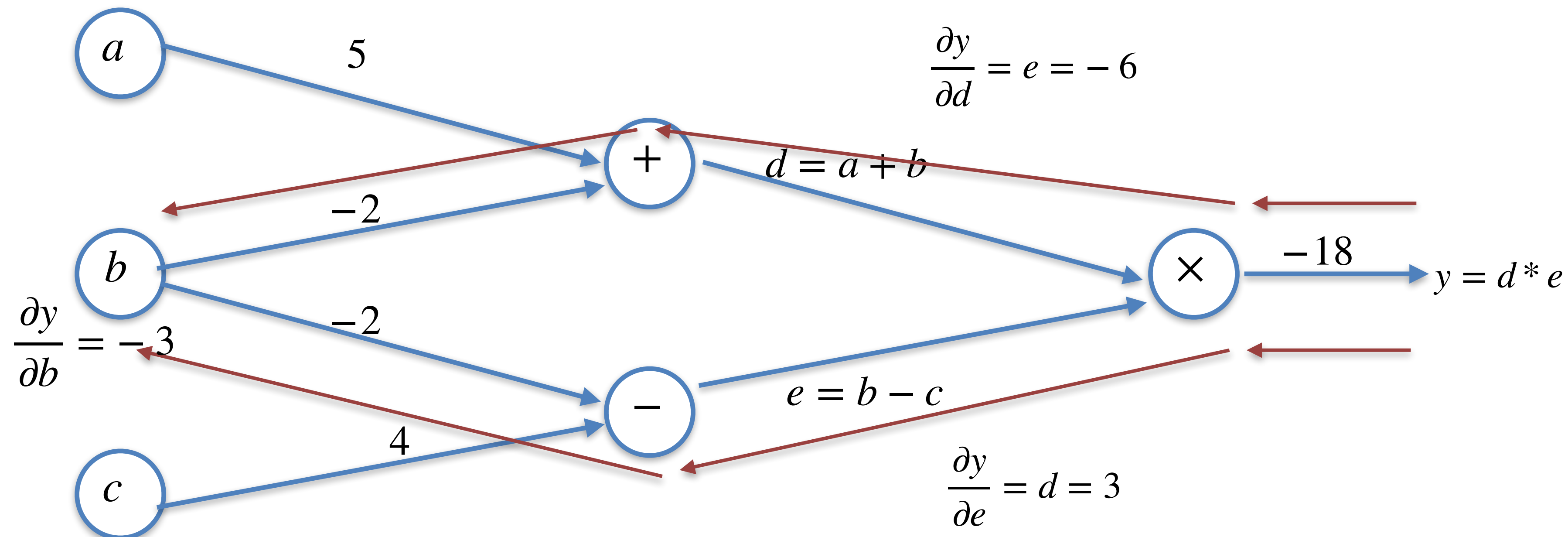
Computation Graph - Backward Pass

- We want to calculate gradient w.r.t a, b, c

$$\frac{\partial y}{\partial a}, \frac{\partial y}{\partial b}, \frac{\partial y}{\partial c}$$

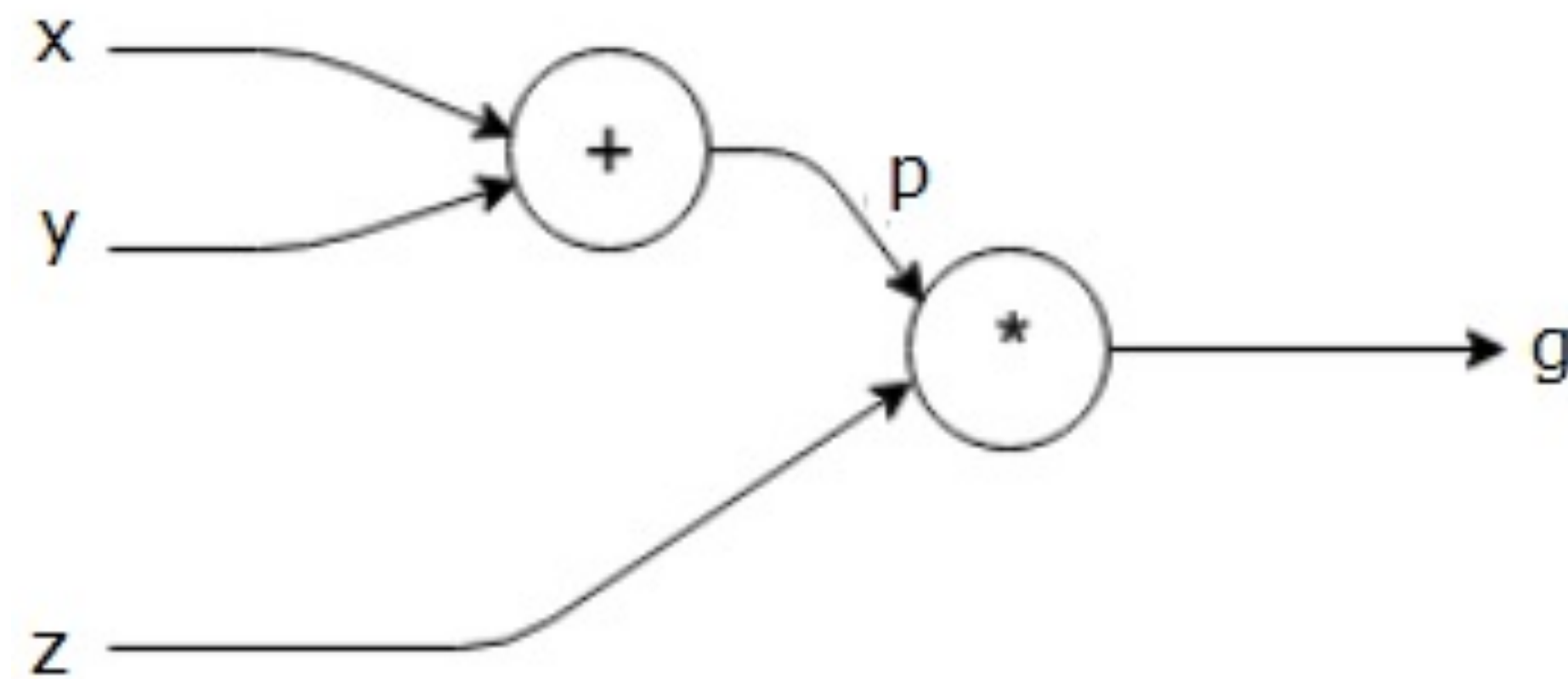
- From chain rule we have
$$\frac{\partial y}{\partial b} = \frac{\partial y}{\partial e} \frac{\partial e}{\partial b} + \frac{\partial y}{\partial d} \frac{\partial d}{\partial b}$$

$$= d \times 1 + e \times 1 = -3$$

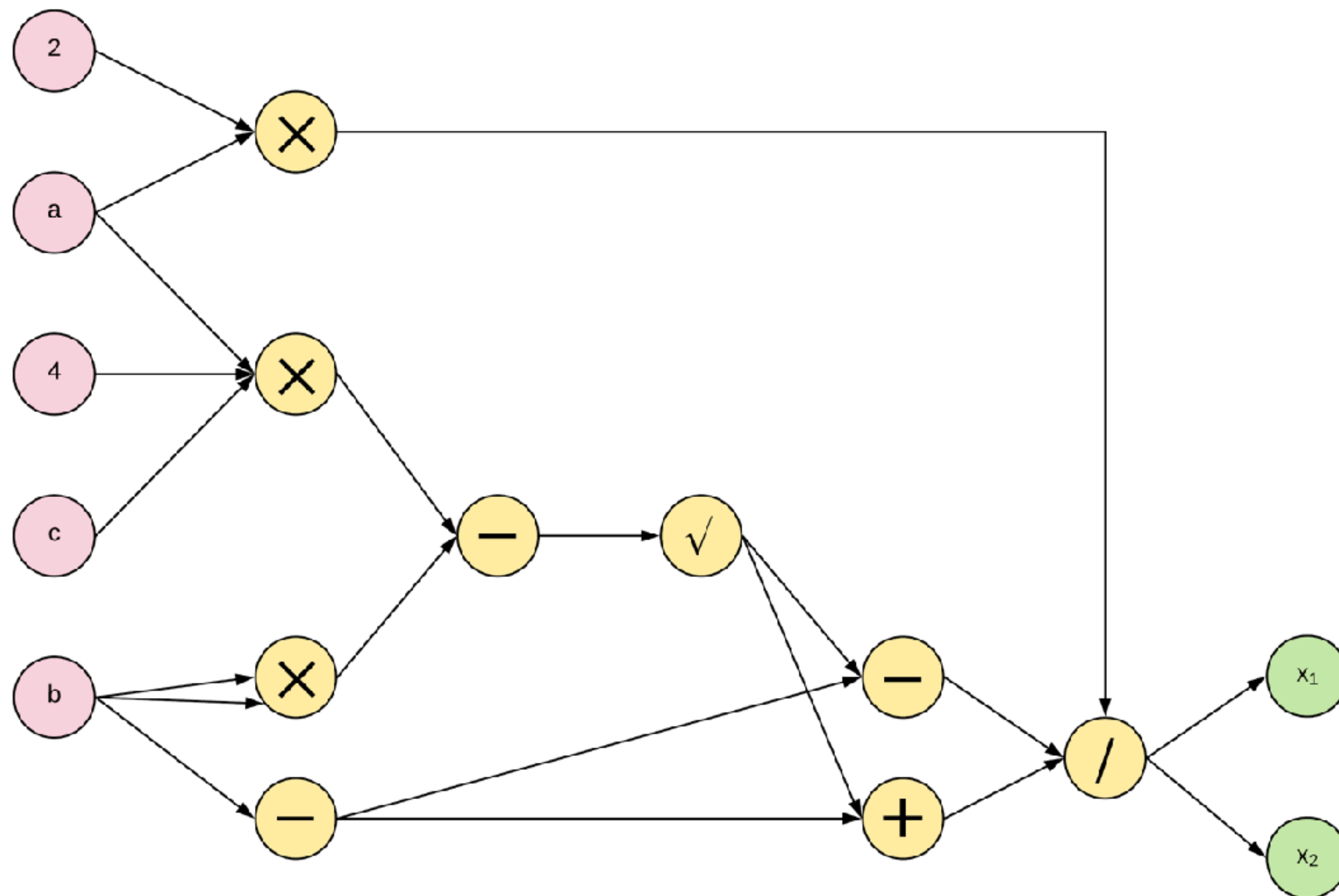


DIY Example

- $g = (x + y)z$



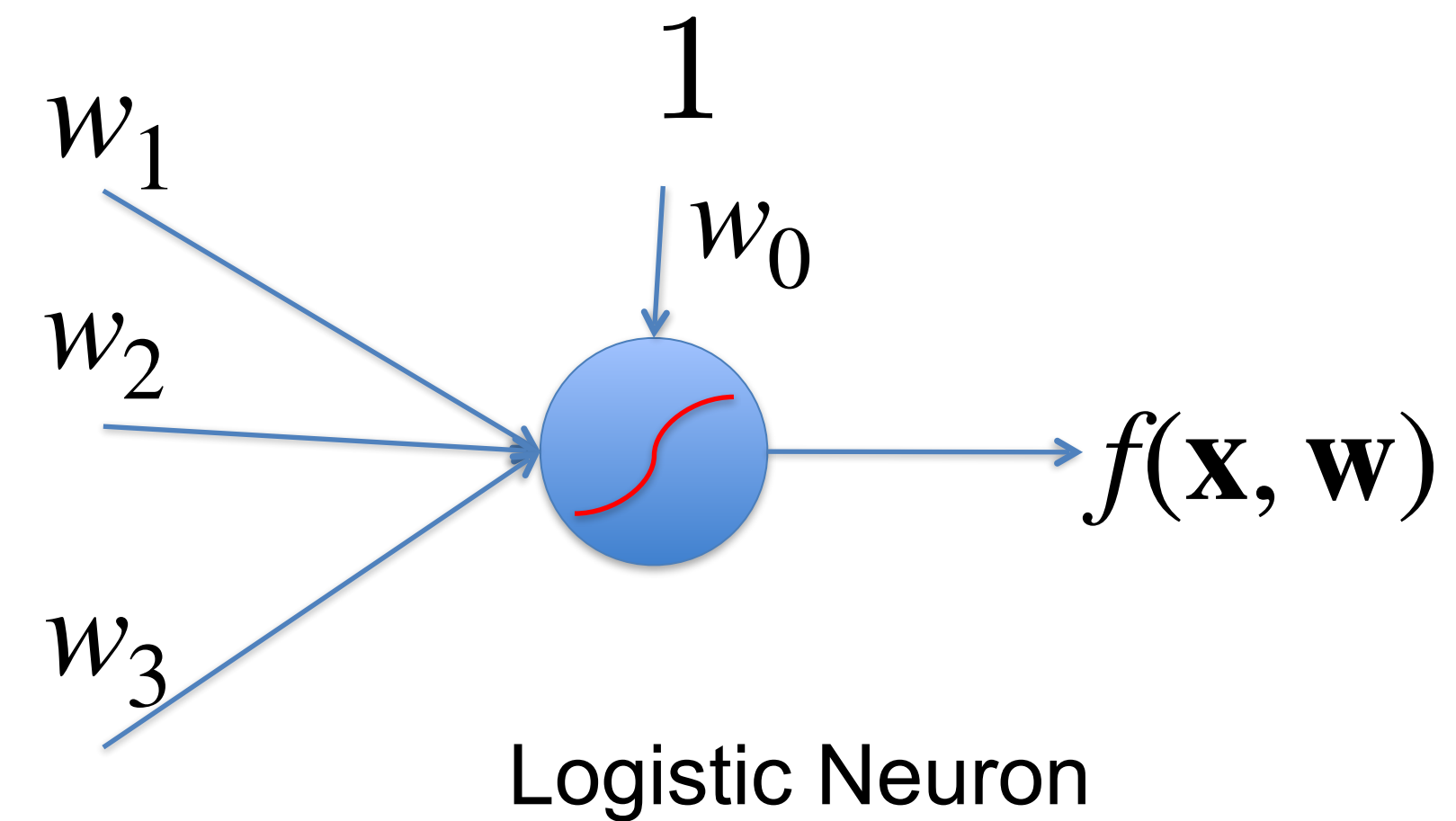
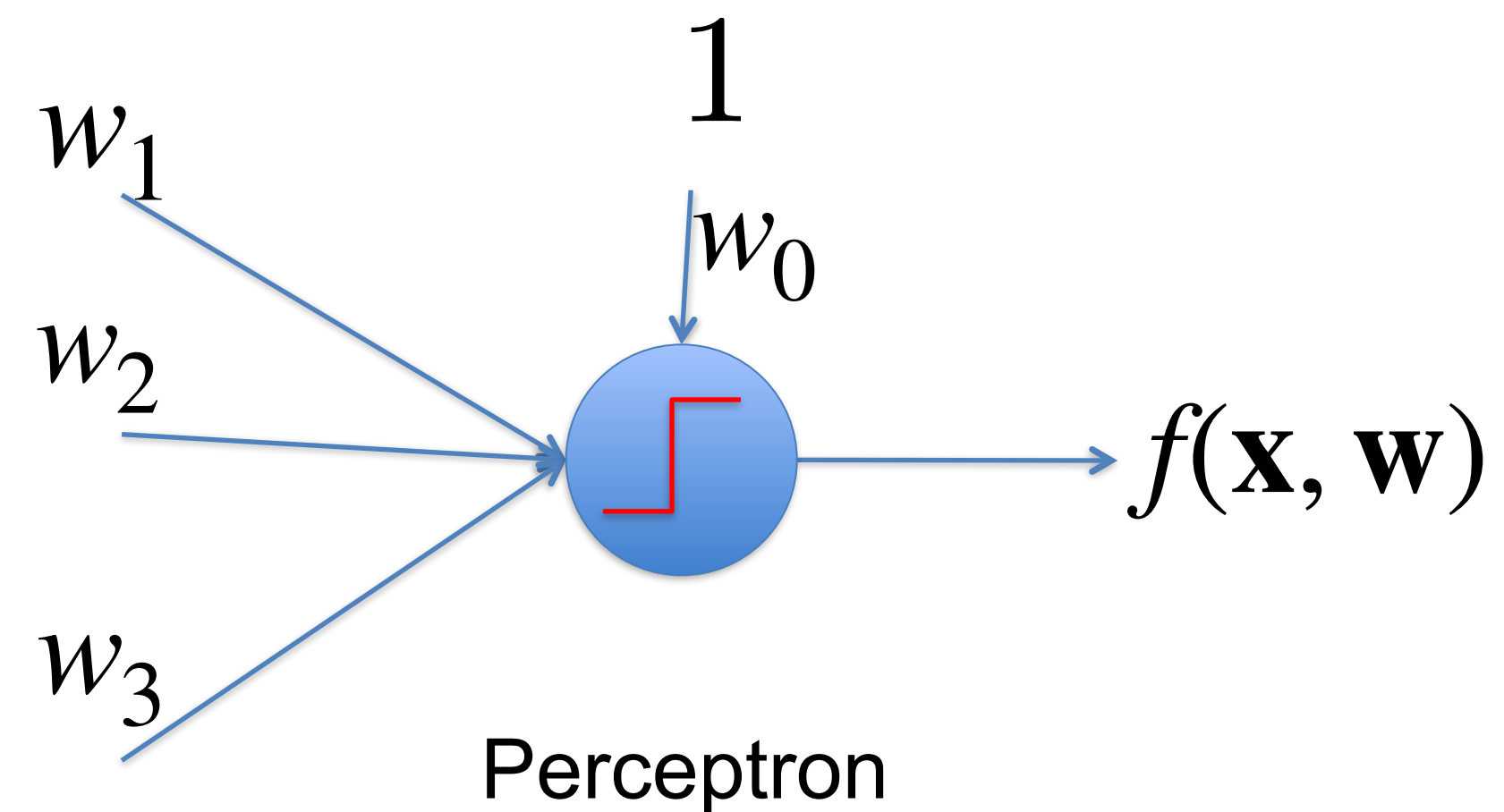
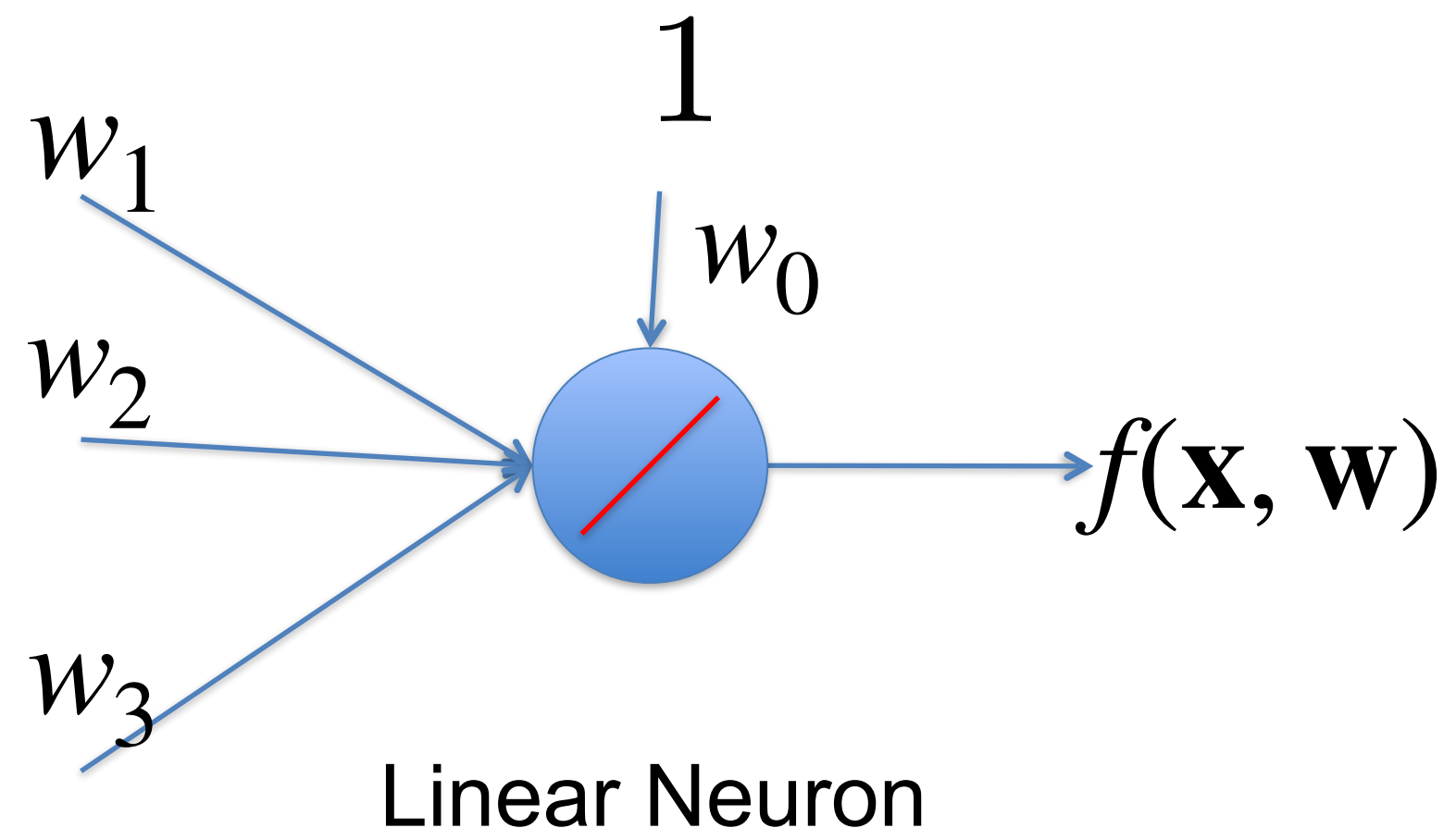
DIY Examples



$$\frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Types of Neuron

Types of Neurons

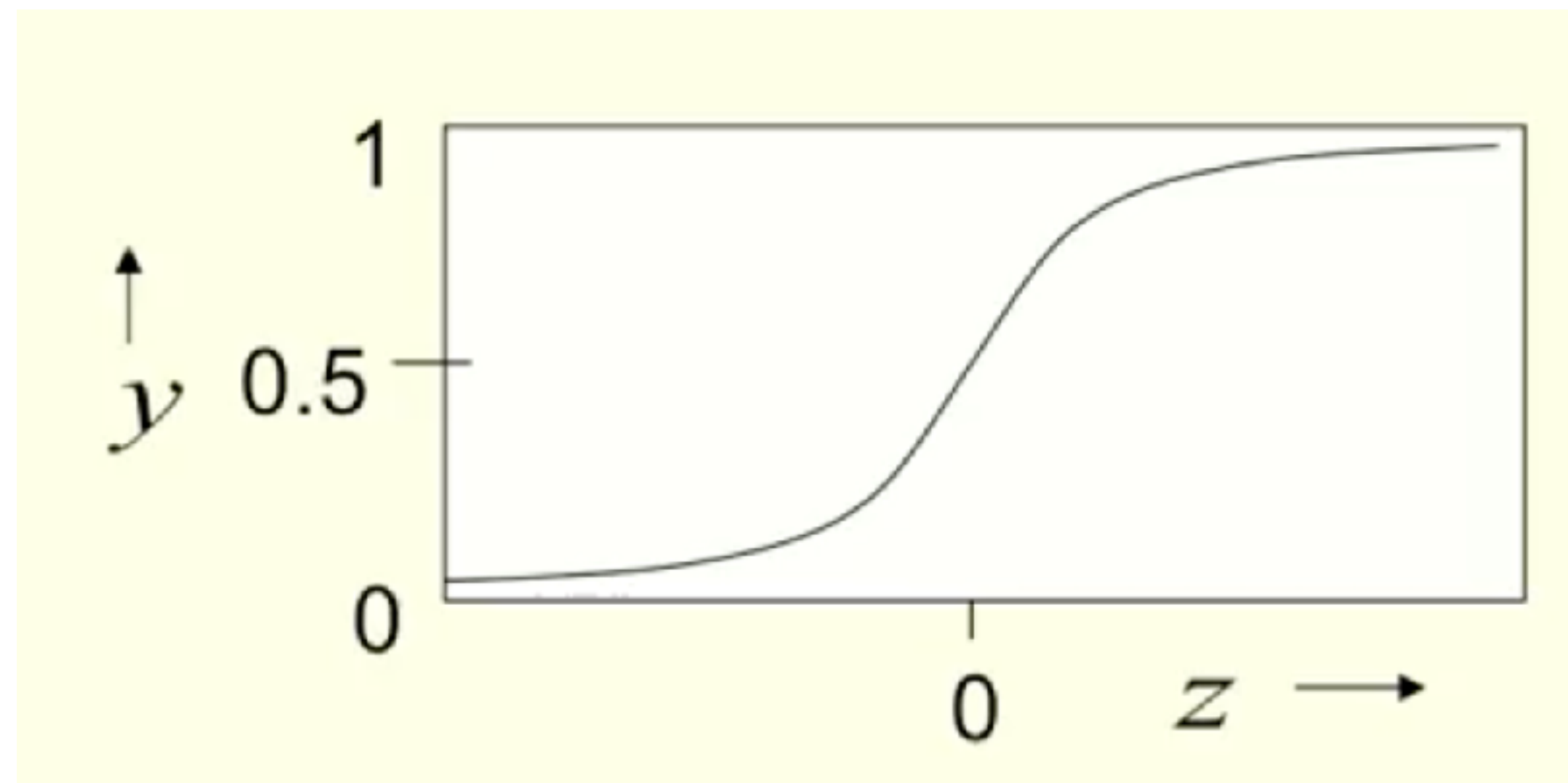


Potentially more. Require a convex loss function for gradient descent

Logistic Neuron

Logistic Neuron

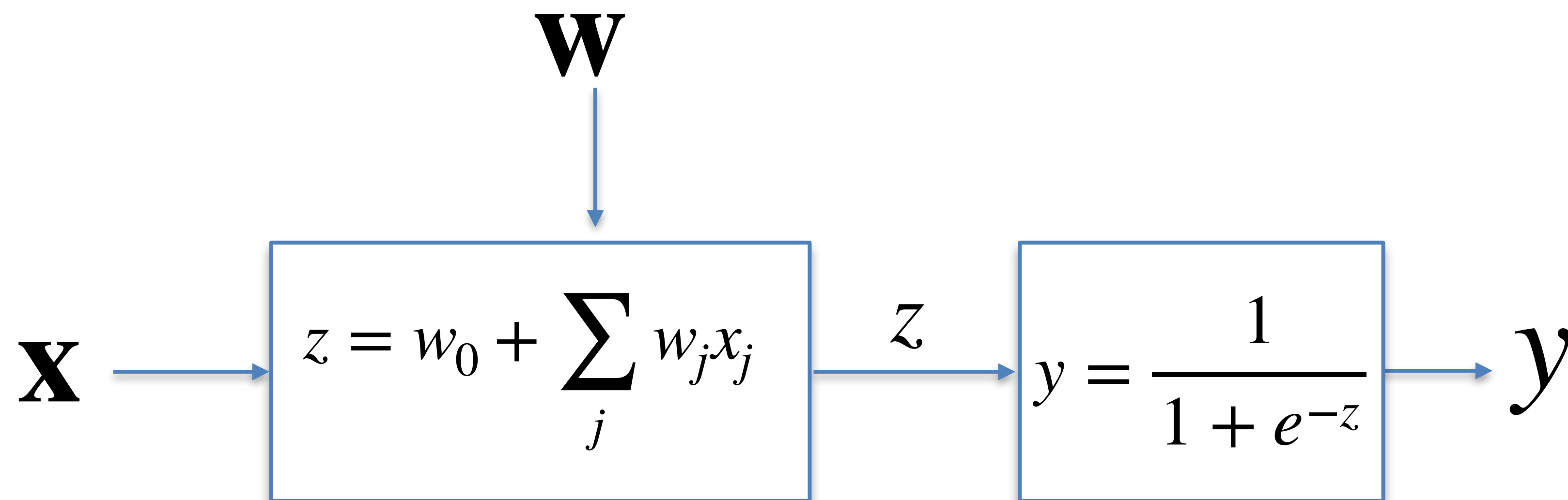
- These give a real-valued output that is a smooth and bounded function of their total input
- They have nice derivatives which make learning easy



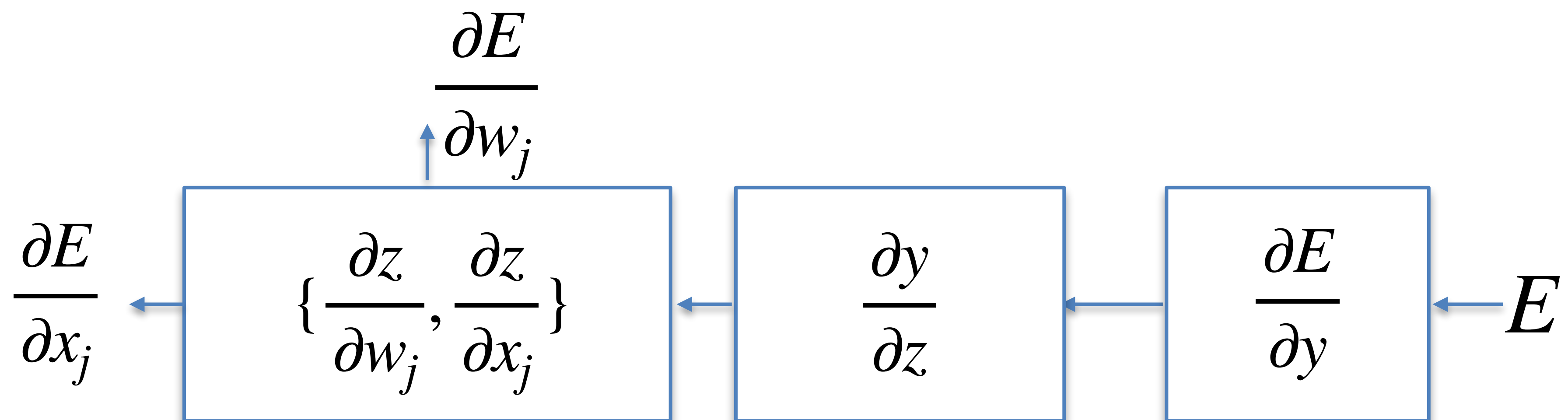
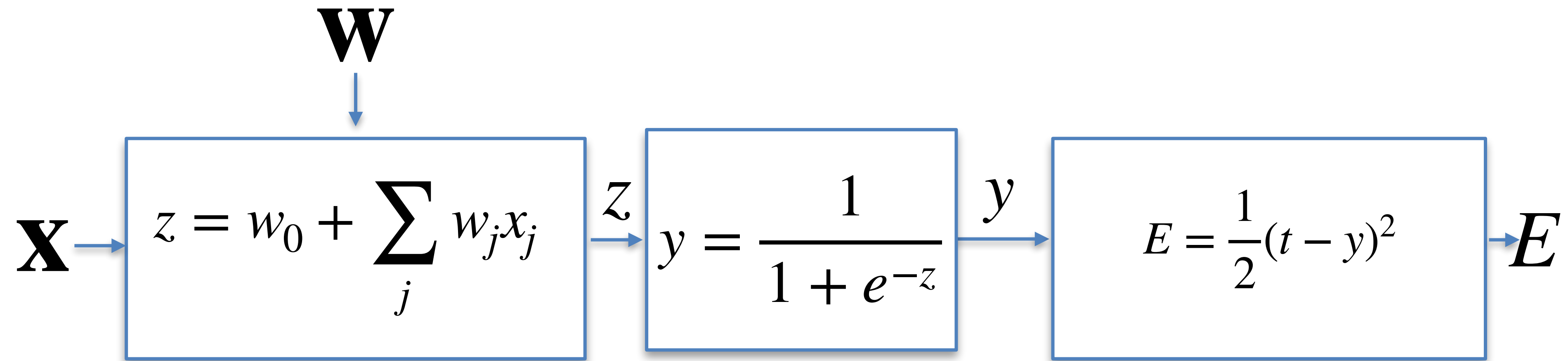
$$z = w_0 + \sum_j x_j w_j$$

$$y = \frac{1}{1 + e^{-z}}$$

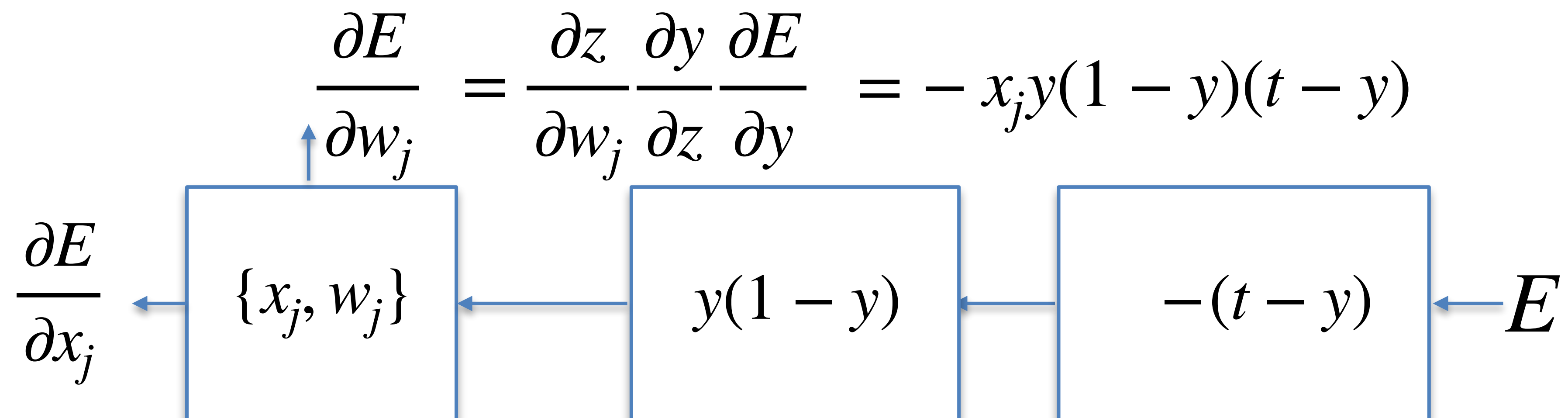
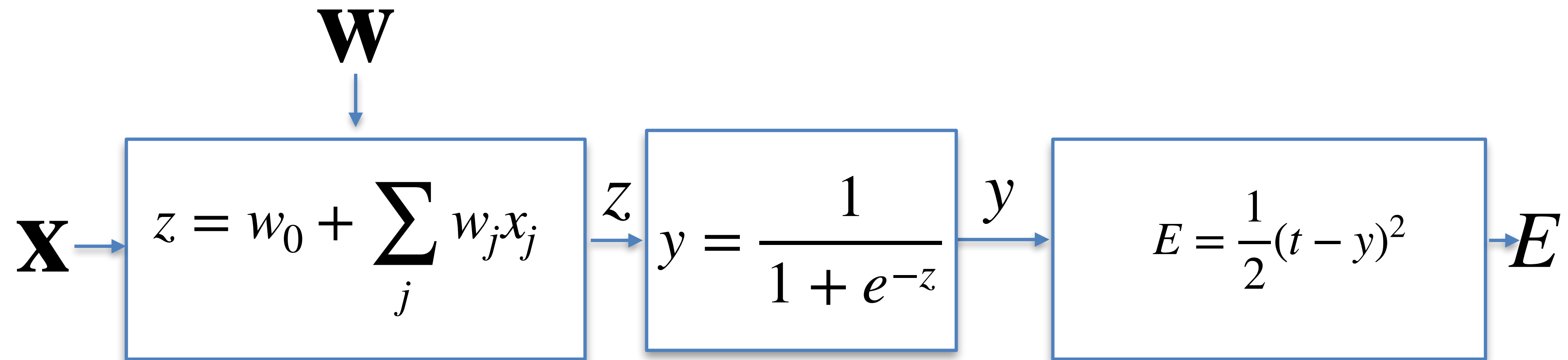
Key Computation: Forward-Prop



Key Computation: Forward-Prop



Key Computation: Back-Prop



Derivation of Logistic Neuron

$$\frac{d}{dx} [x^n] = n x^{n-1}$$

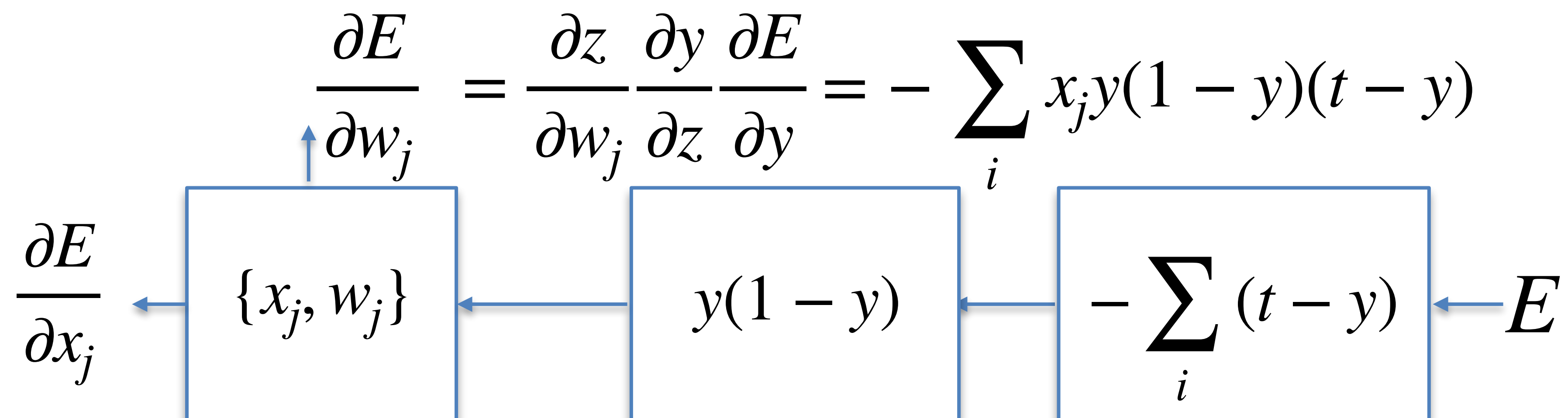
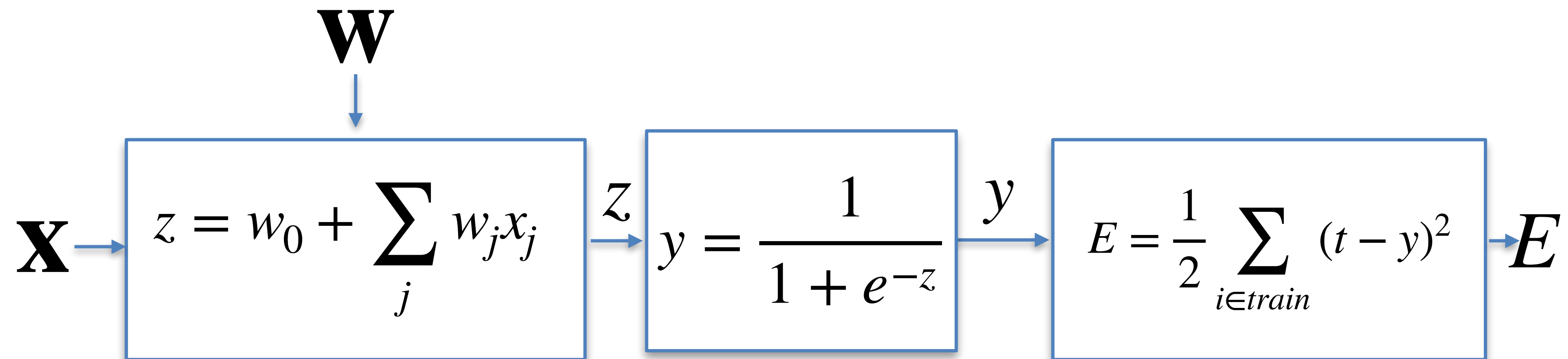
Calculusworkshop.com

$$y = \frac{1}{1 + e^{-z}} = (1 + e^{-z})^{-1}$$

$$\frac{\partial y}{\partial z} = \frac{-1(-e^{-z})}{(1 + e^{-z})^2} = \left(\frac{1}{1 + e^{-z}} \right) \left(\frac{e^{-z}}{1 + e^{-z}} \right) = y(1 - y)$$

$$\text{because } \frac{e^{-z}}{1 + e^{-z}} = \frac{(1 + e^{-z}) - 1}{(1 + e^{-z})} = \frac{(1 + e^{-z})}{(1 + e^{-z})} - \frac{1}{(1 + e^{-z})} = 1 - y$$

Key Computation: Back-Prop



Key Computation: Back-Prop

- Iterative Solution

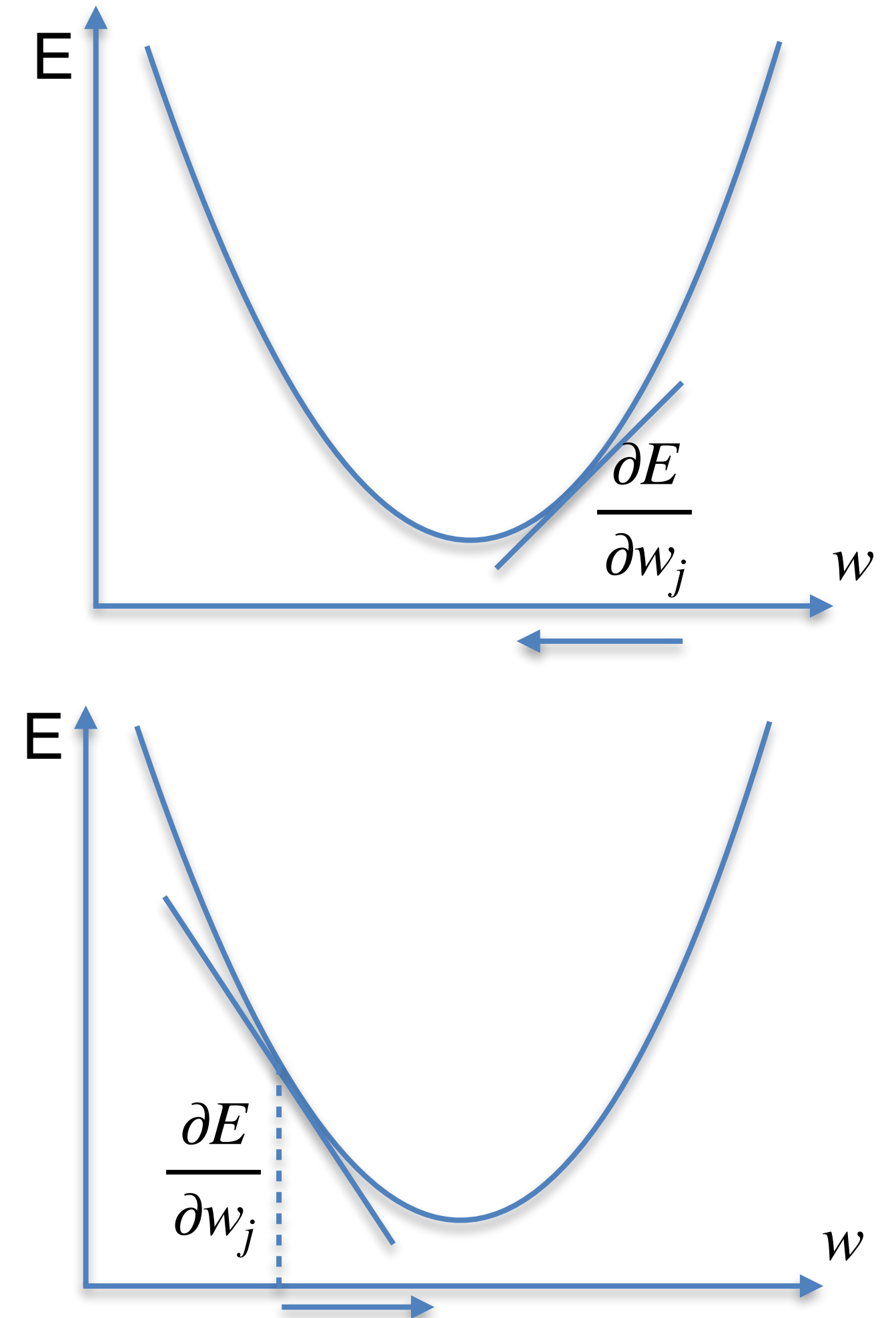
$$w_j \leftarrow w_j - \eta \frac{\partial E}{\partial w_j}$$

$$w_j \leftarrow w_j + \Delta w_j$$

$$\Delta w_j = -\eta \frac{\partial E}{\partial w_j}$$

$$\Delta w_j^{(i)} = \eta \sum_i x_j^{(i)} y^{(i)} (1 - y^{(i)}) (t^{(i)} - y^{(i)})$$

where η is the learning rate



Activtion Functions

sigmoid vs tanh

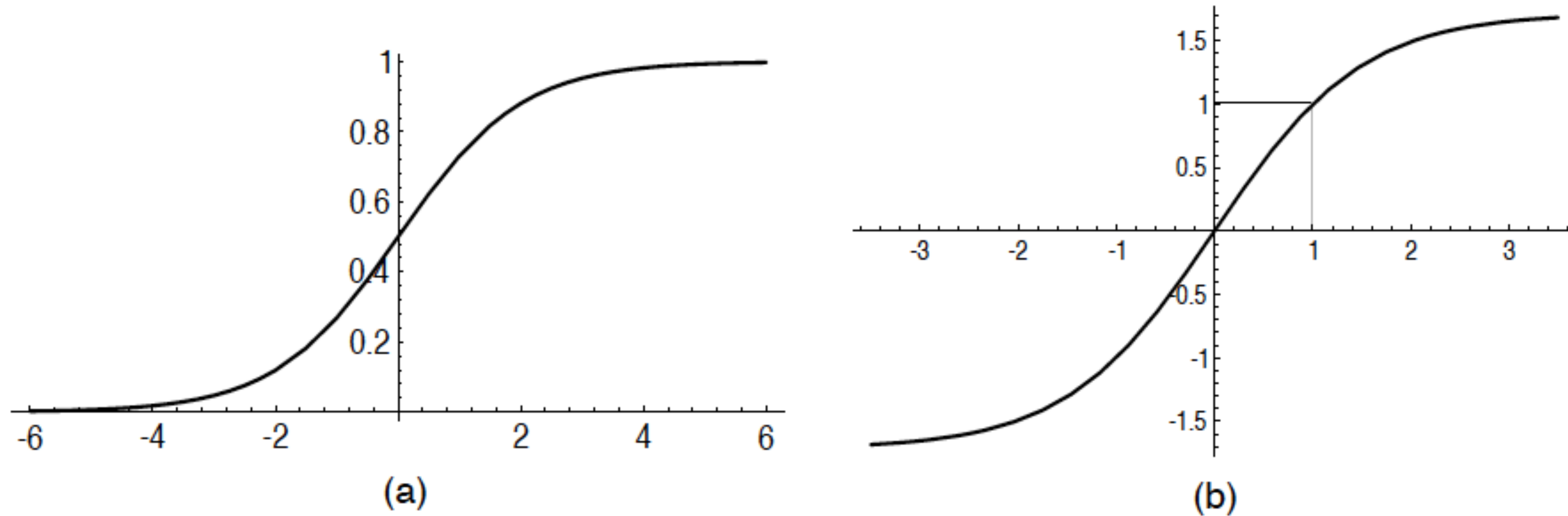
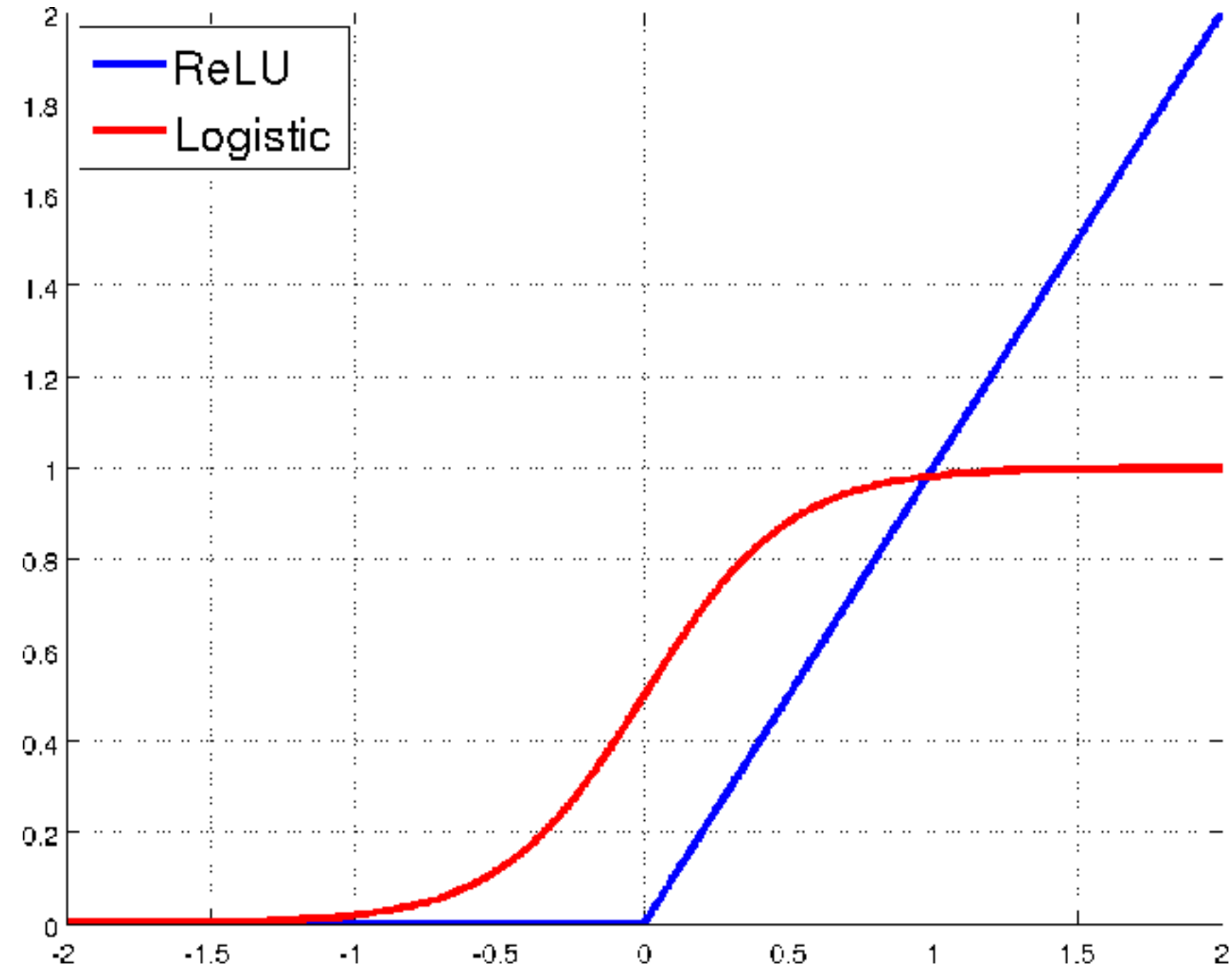


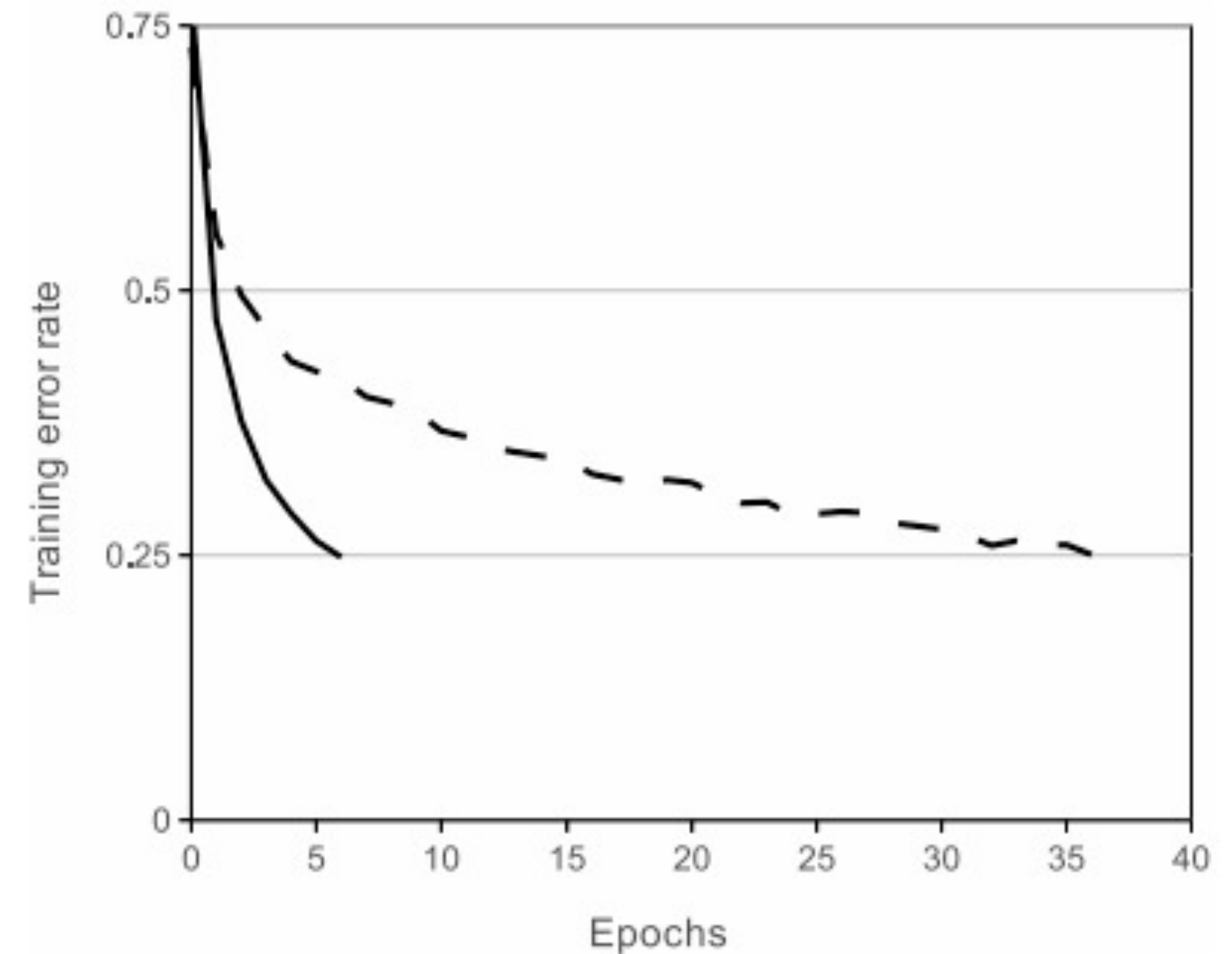
Fig. 4. (a) Not recommended: the standard logistic function, $f(x) = 1/(1 + e^{-x})$. (b) Hyperbolic tangent, $f(x) = 1.7159 \tanh(\frac{2}{3}x)$.

Rectified Linear Units (ReLU)



► ReLU $\max(0, z)$

► Leaky ReLU $\max(0.1z, z)$



[Krizhevsky et al., NIPS12]

ReLU Variants for Transformers

SiLU and GELU

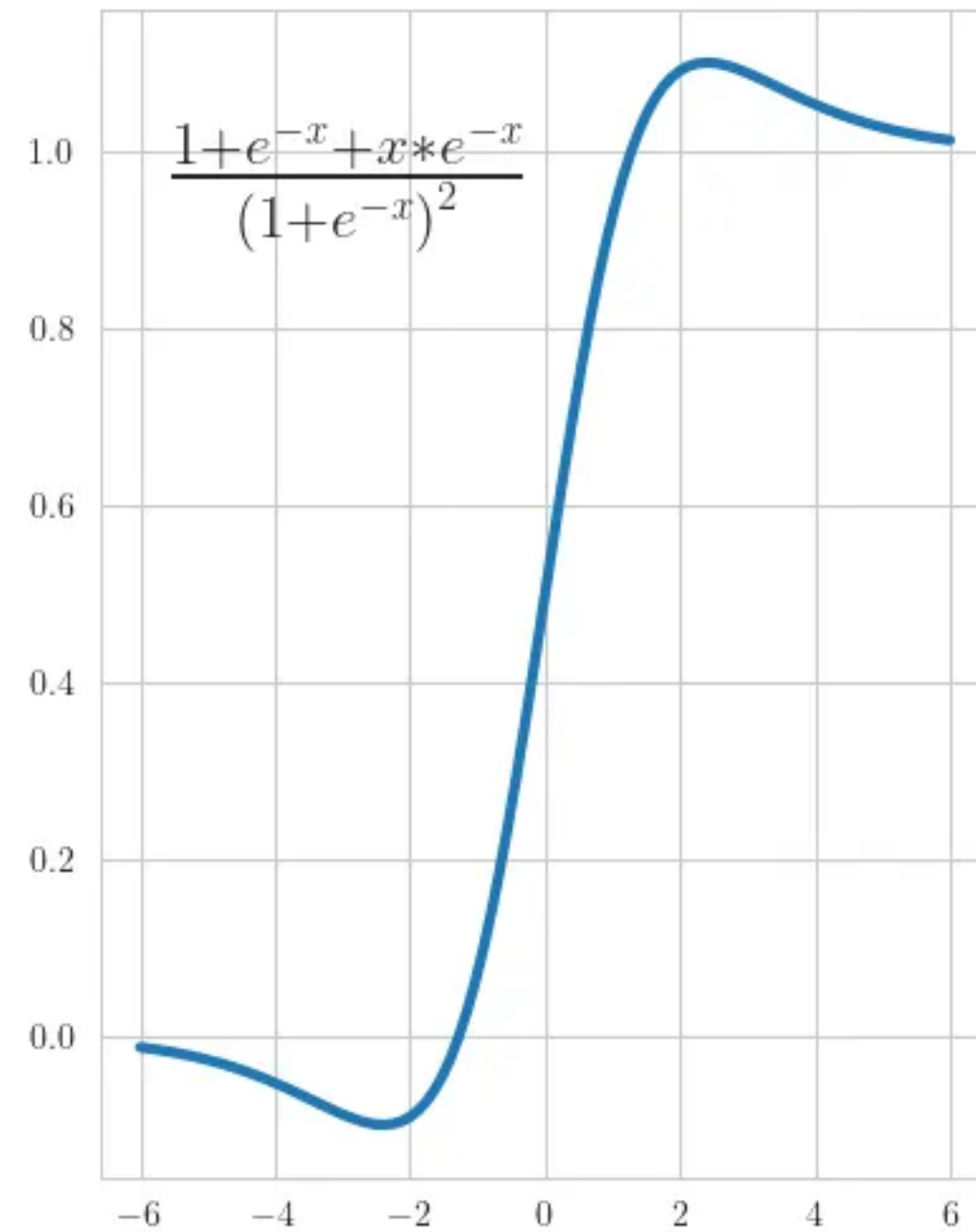
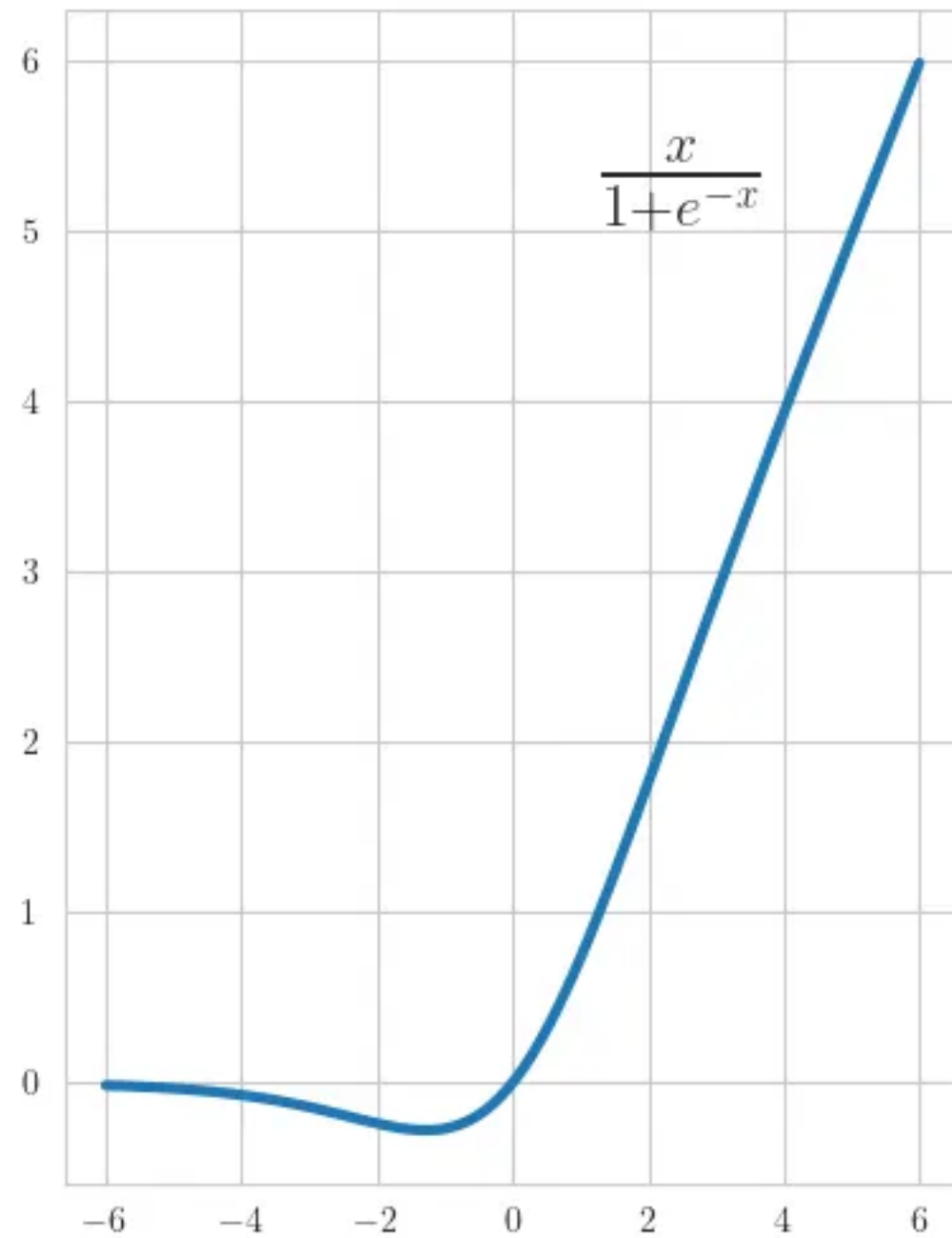
$$\text{SiLU}(x) = x \cdot \frac{1}{1 + e^{-x}}$$

$$\text{GELU}(x) = 0.5 \cdot x \cdot \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + 0.044715x^3) \right) \right)$$

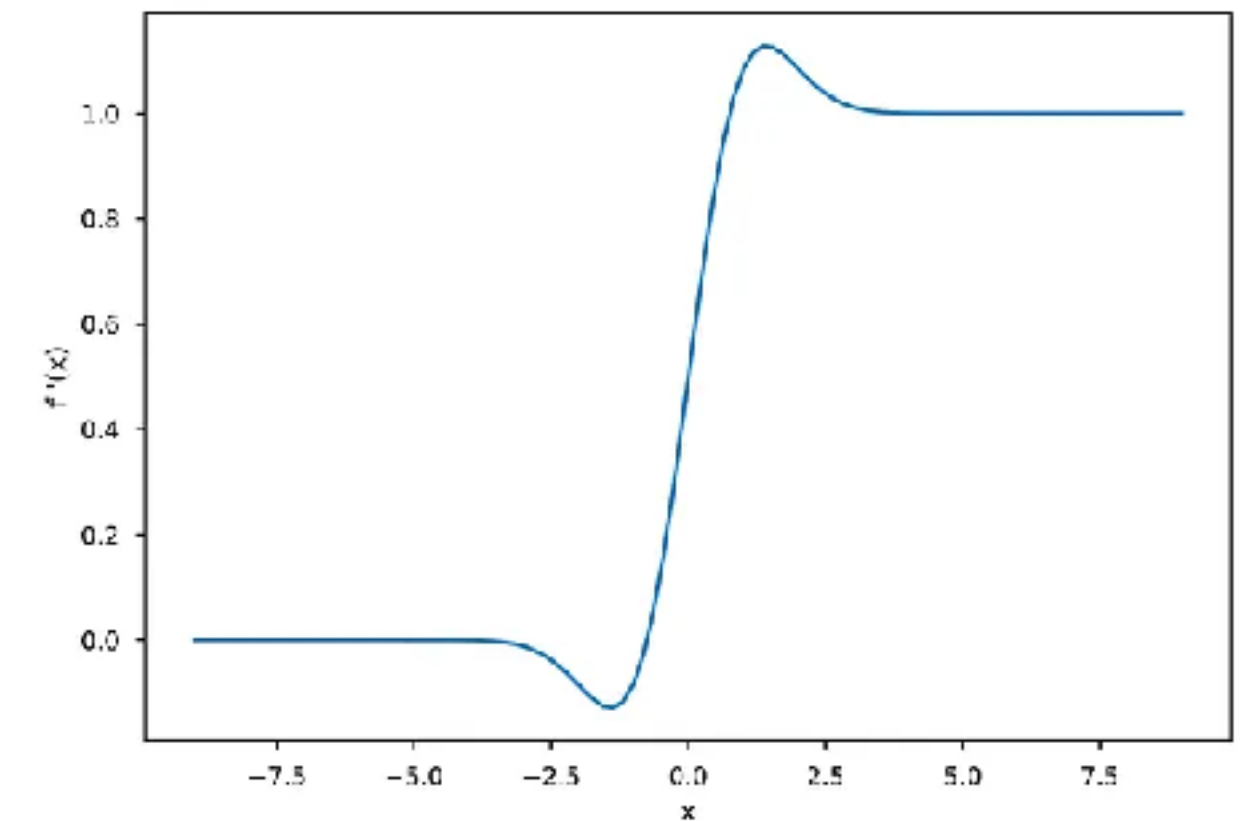
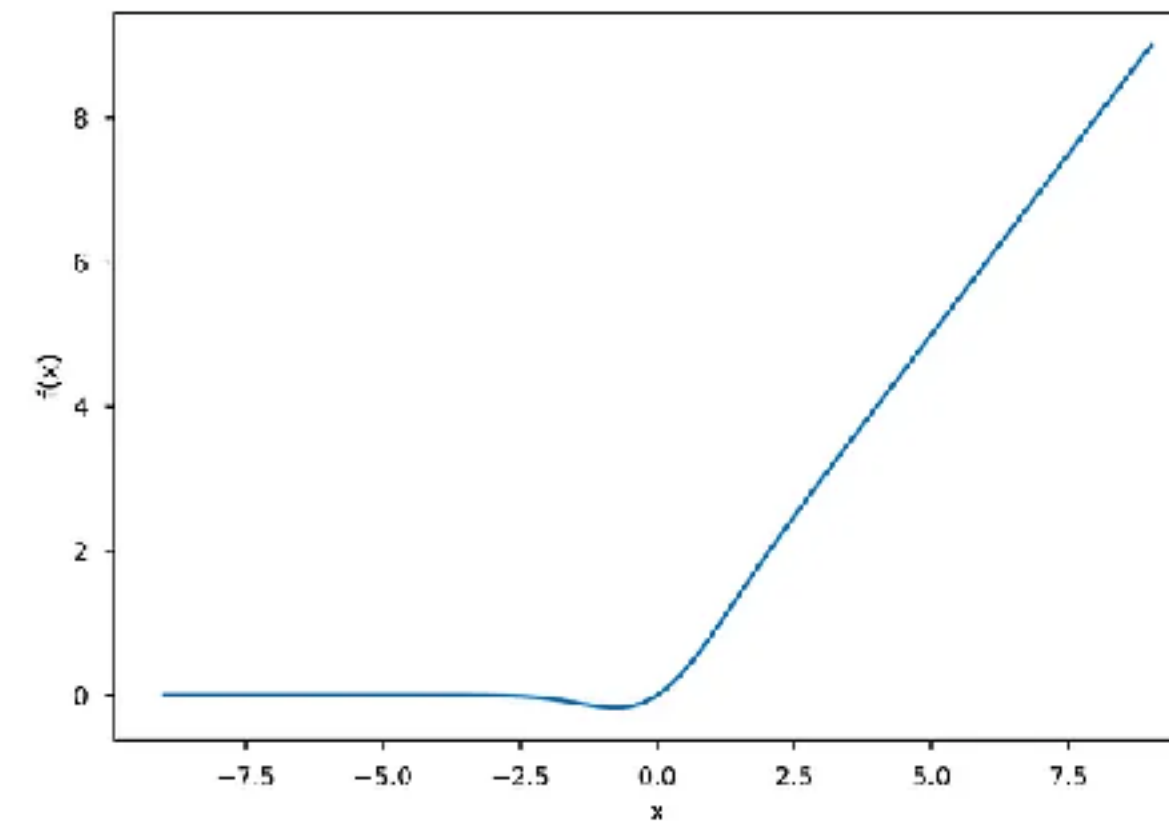
Sigmoid-Weighted Linear Unit (SiLU) / Swish

Function

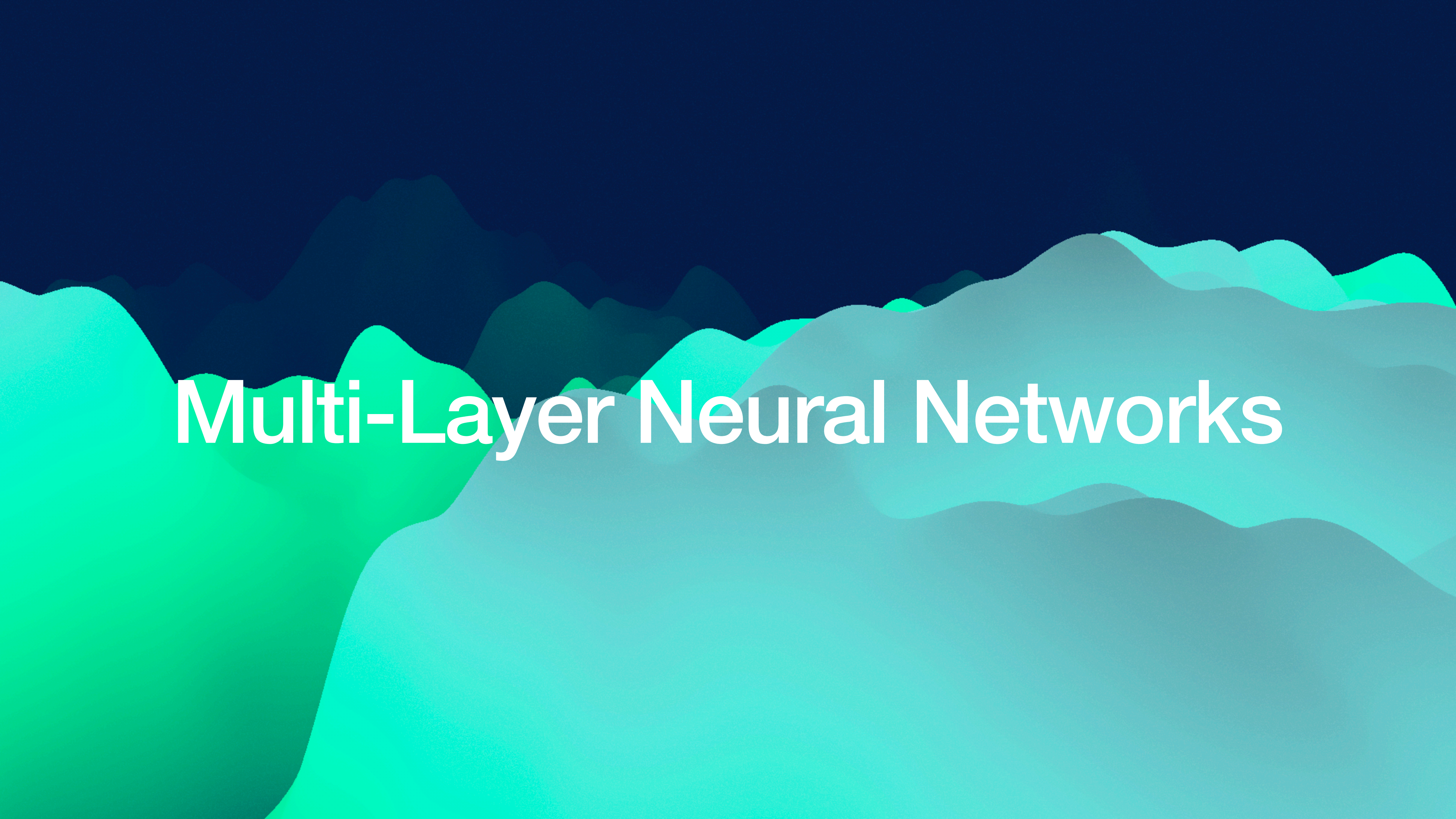
Derivative



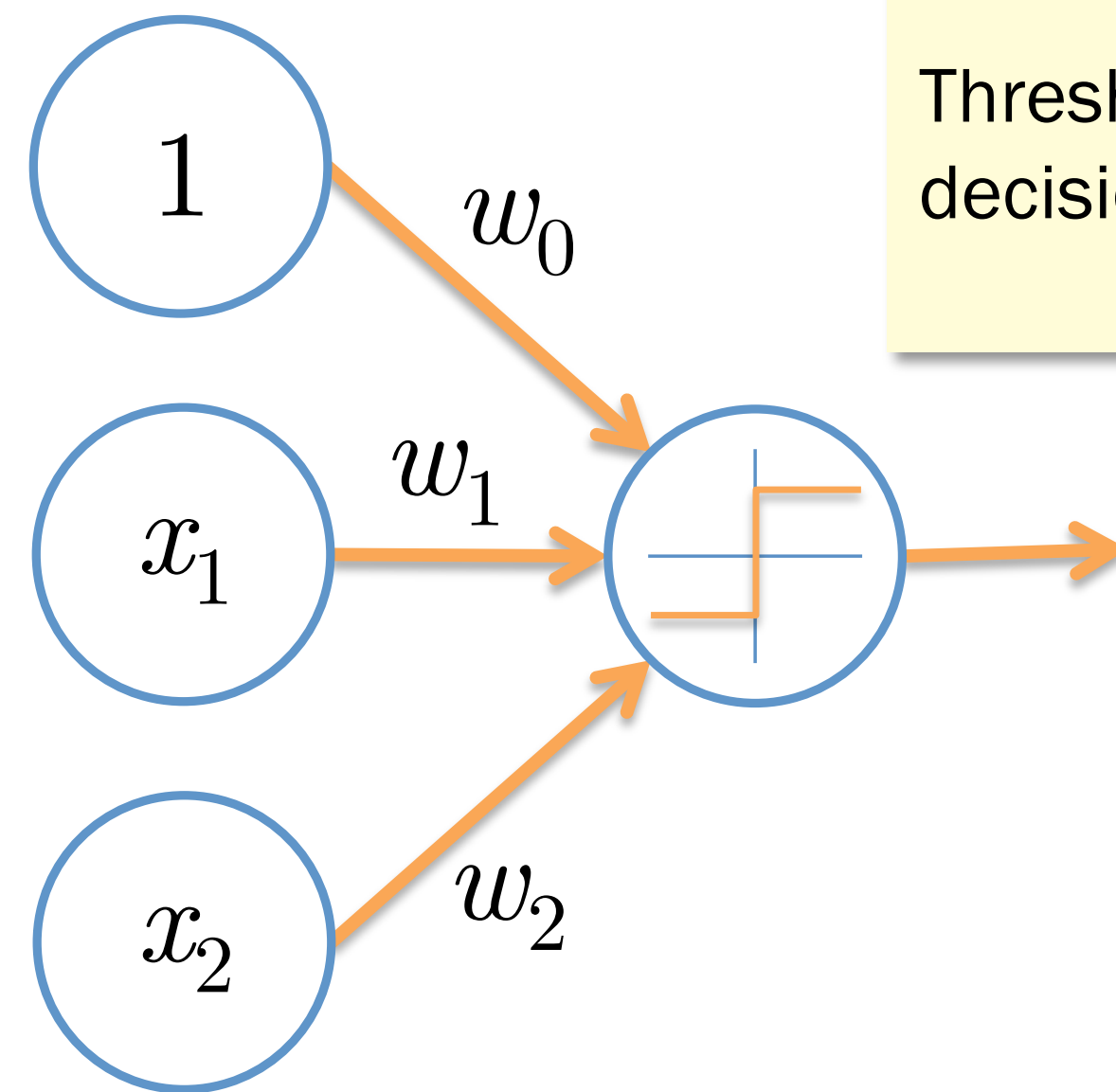
GELU function and it's Derivative



Multi-Layer Neural Networks

The background features a dark navy blue upper section. Below this, there are several overlapping, wavy, organic shapes in various shades of teal and green. These shapes have soft, blurred edges, creating a layered, atmospheric effect. The colors range from a deep, muted teal to a bright, vibrant green, with some areas appearing more saturated than others.

Perceptron

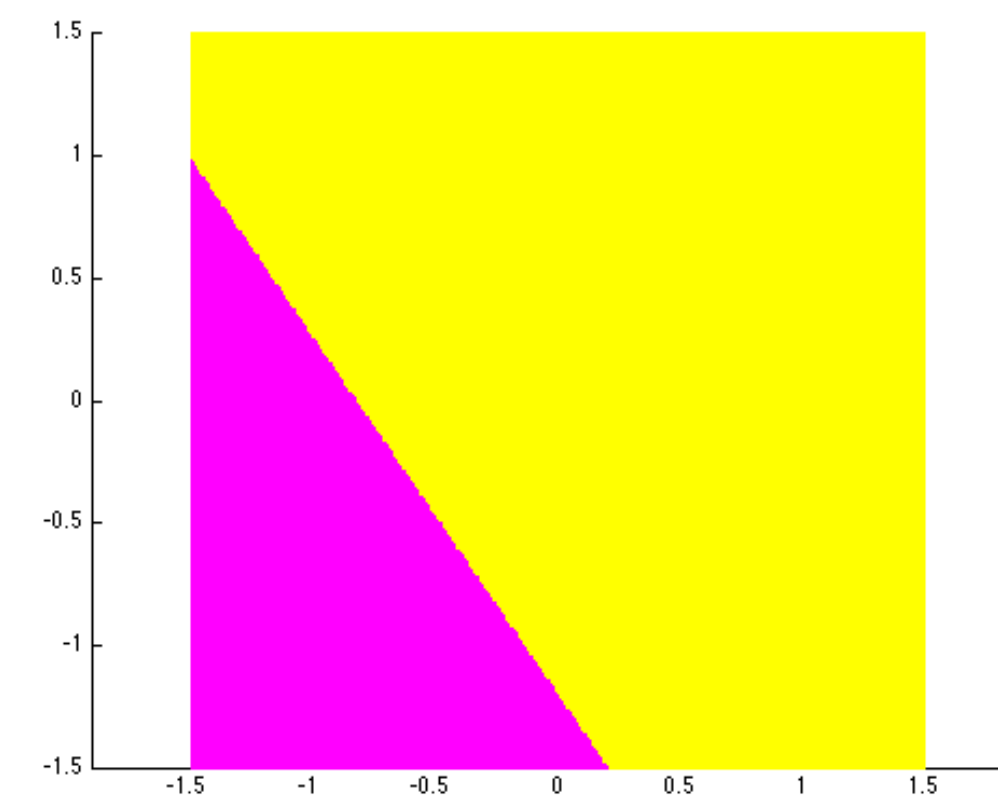
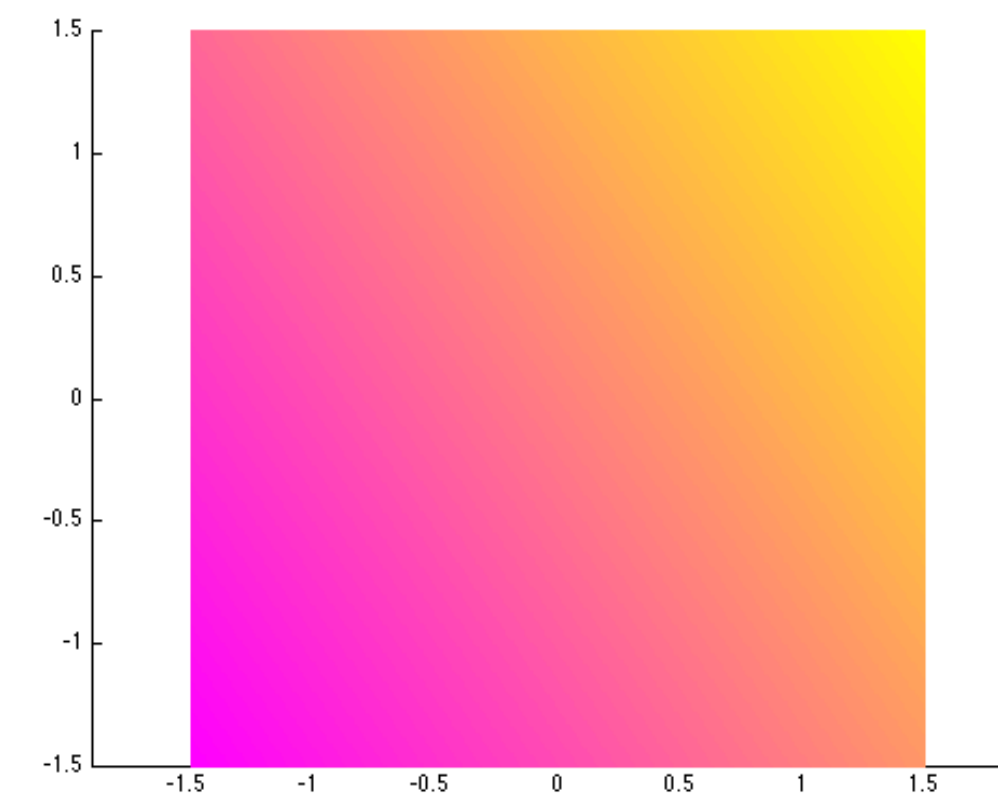


Implements a linear function $f(x_1, x_2)$

Threshold to get linear decision boundary

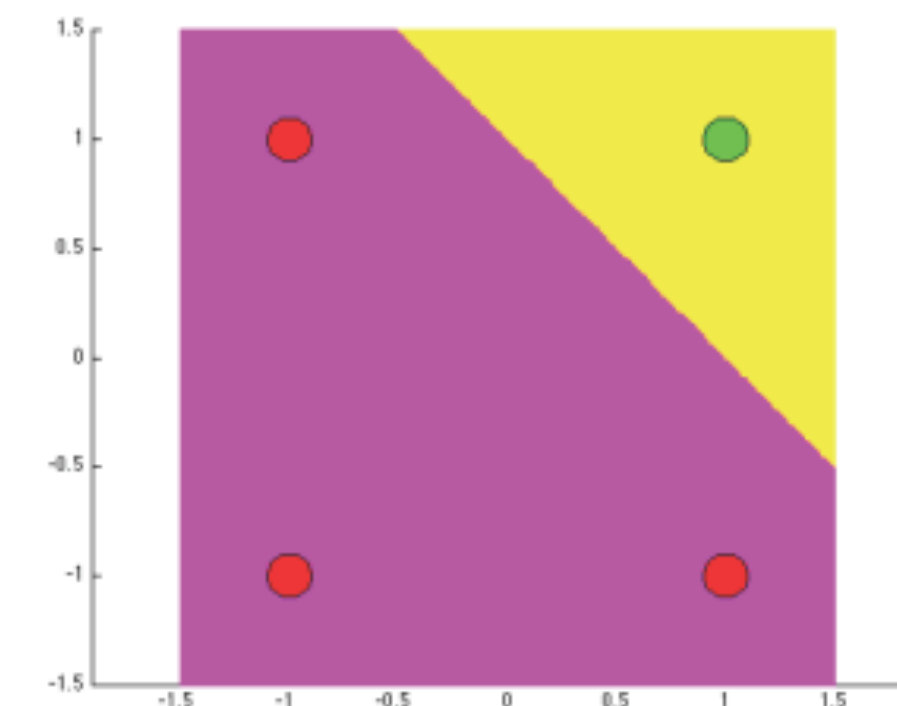
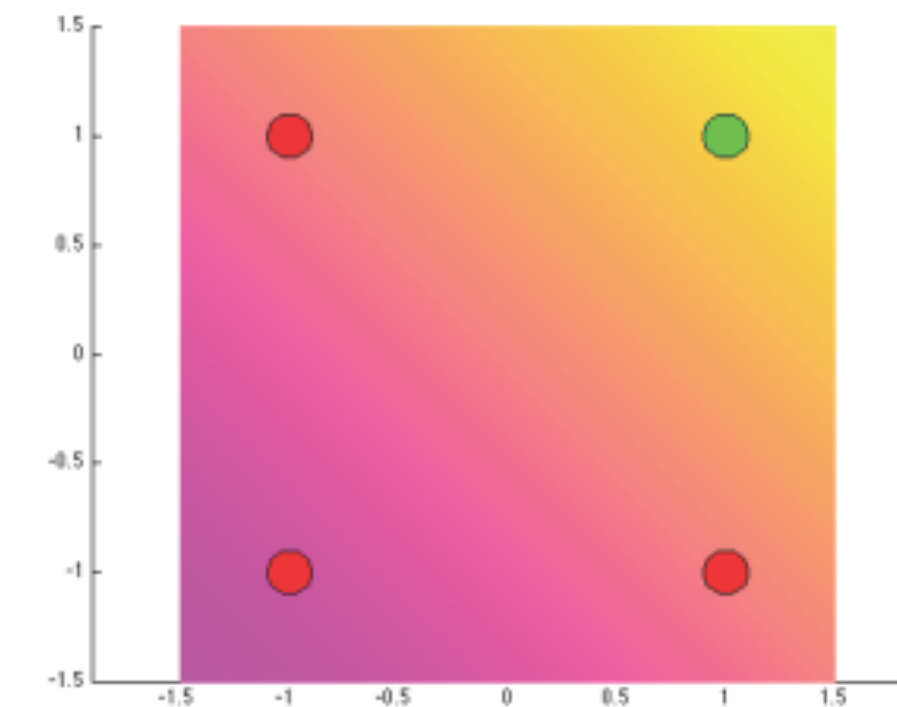
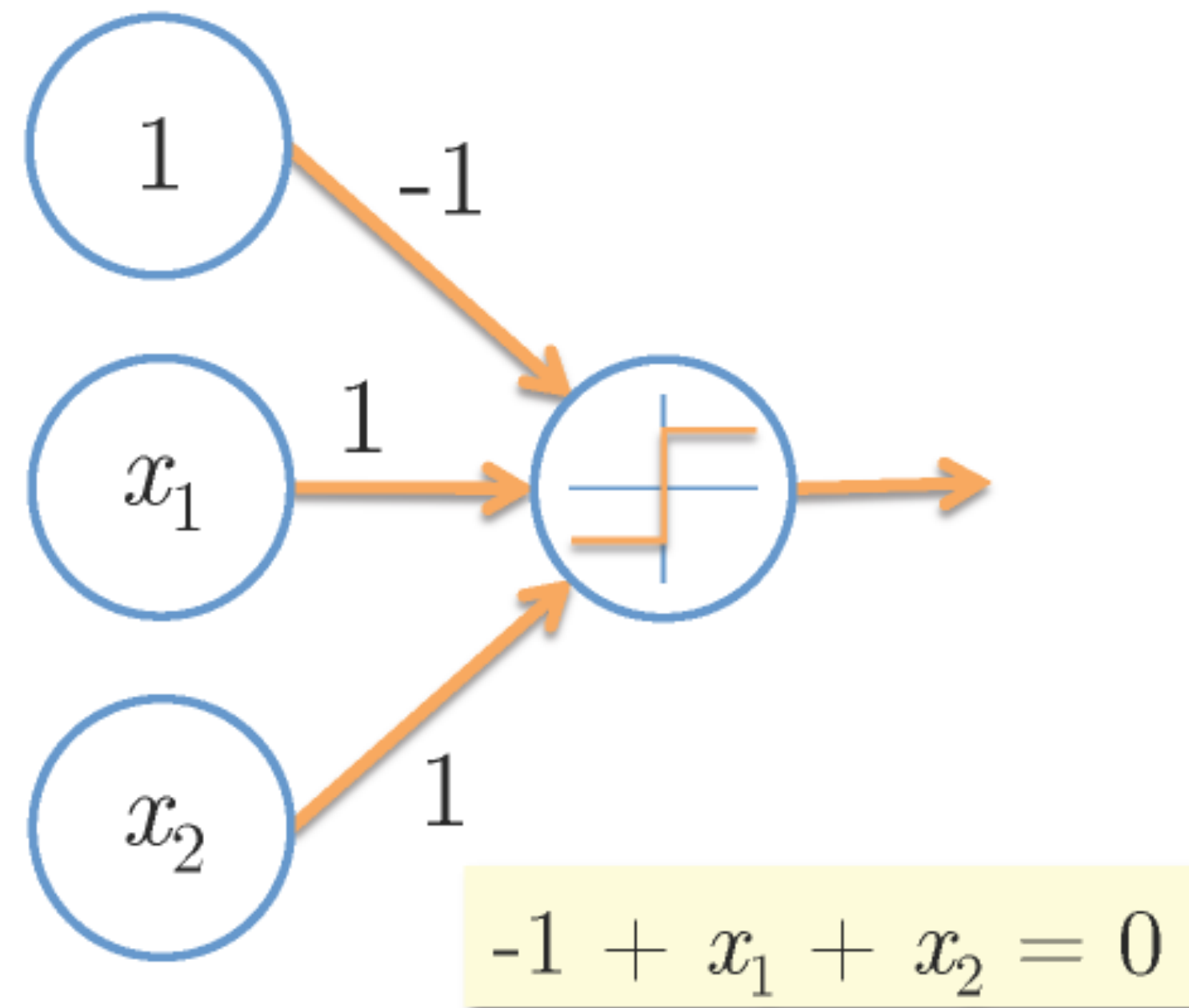
Example:

$$\mathbf{w} = [0.79, 0.96, 0.66]$$

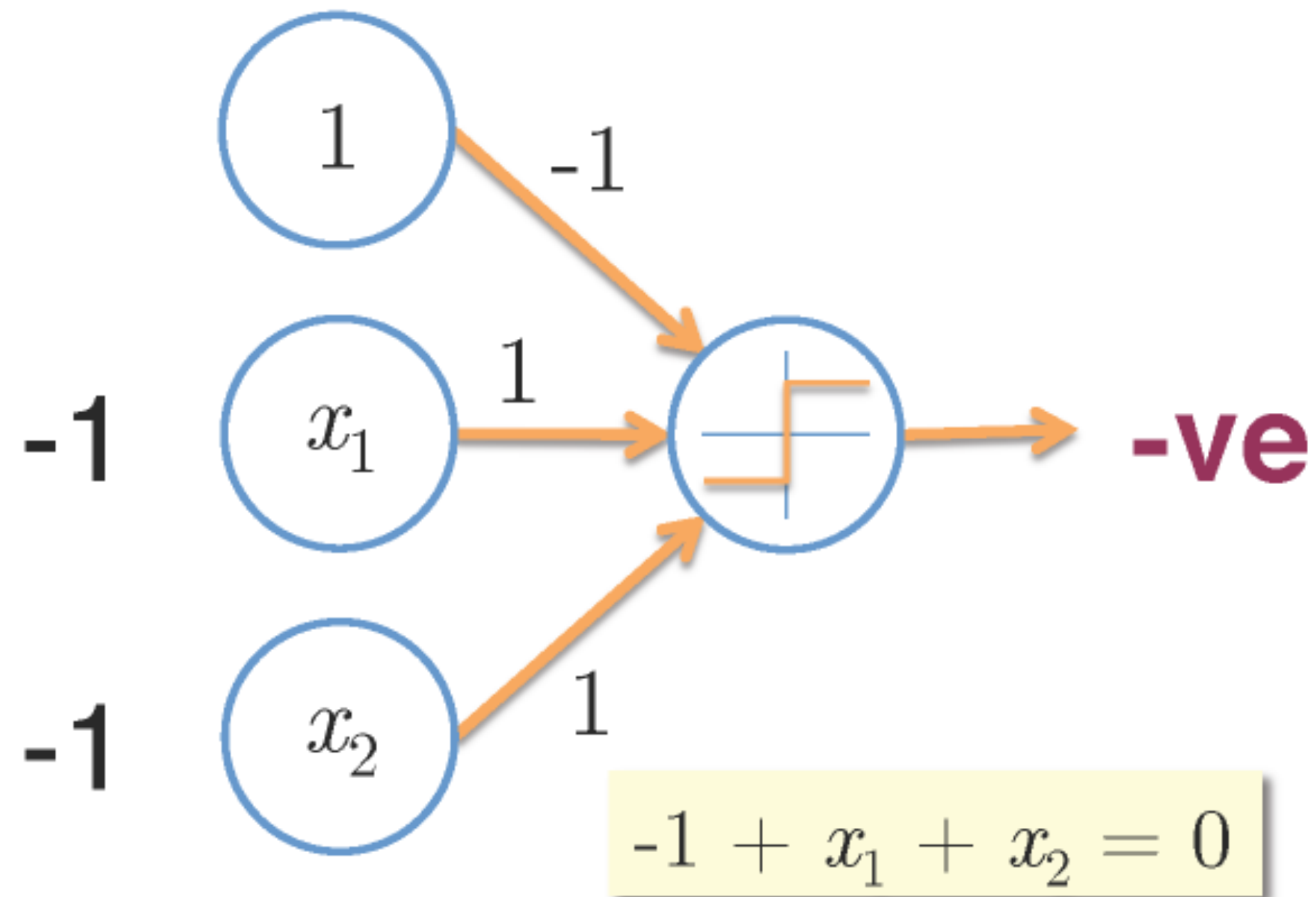


AND Function

x_1	x_2	AND(x_1, x_2)
-1	-1	-1
-1	1	-1
1	-1	-1
1	1	1

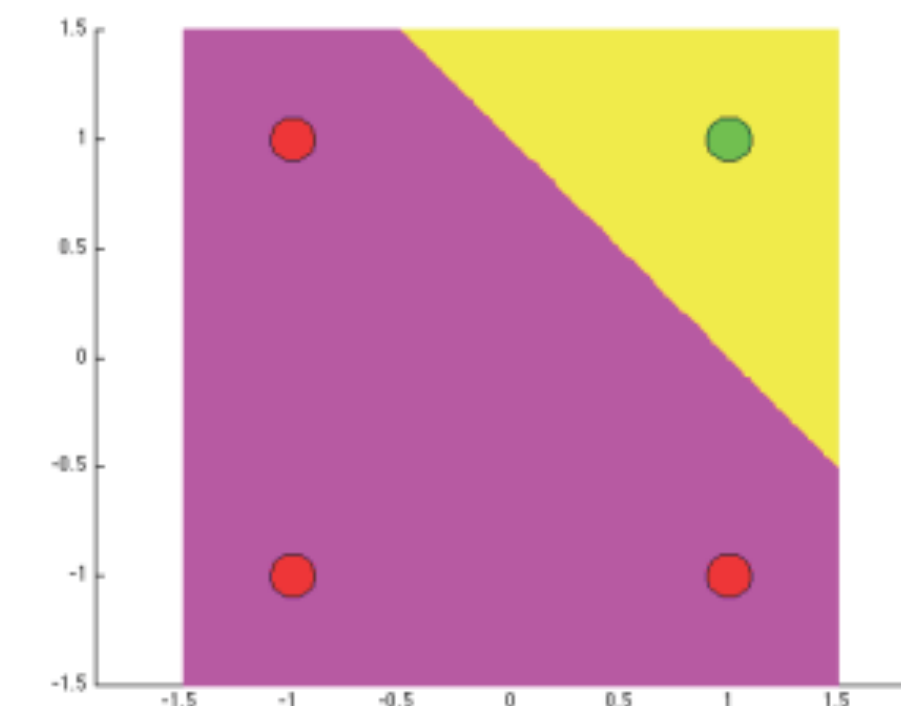
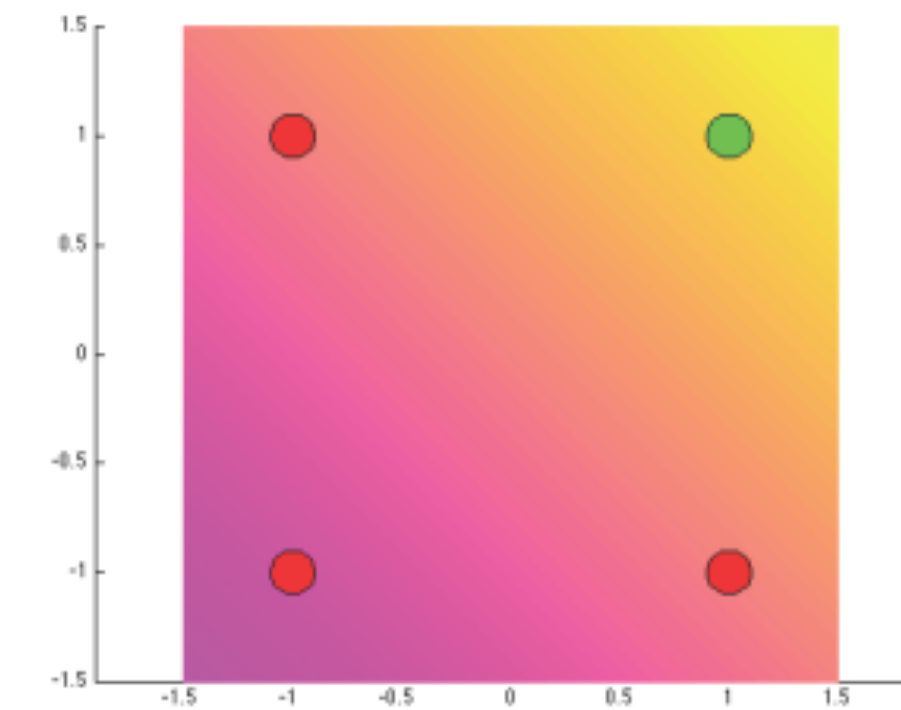


AND Function

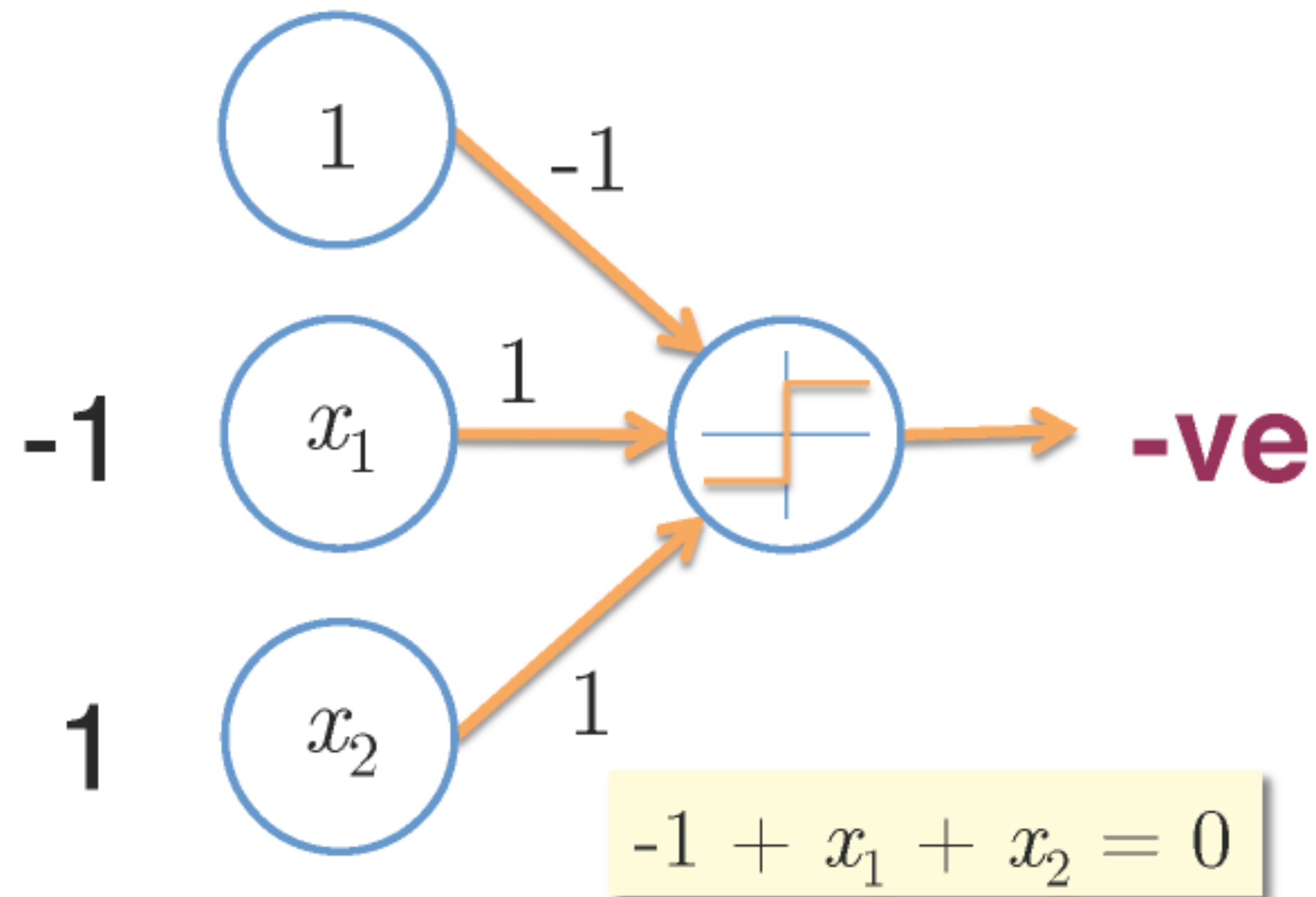


$$1 \times -1 + -1 \times 1 + -1 \times 1 = -3$$

x_1	x_2	AND(x_1, x_2)
-1	-1	-1
-1	1	-1
1	-1	-1
1	1	1

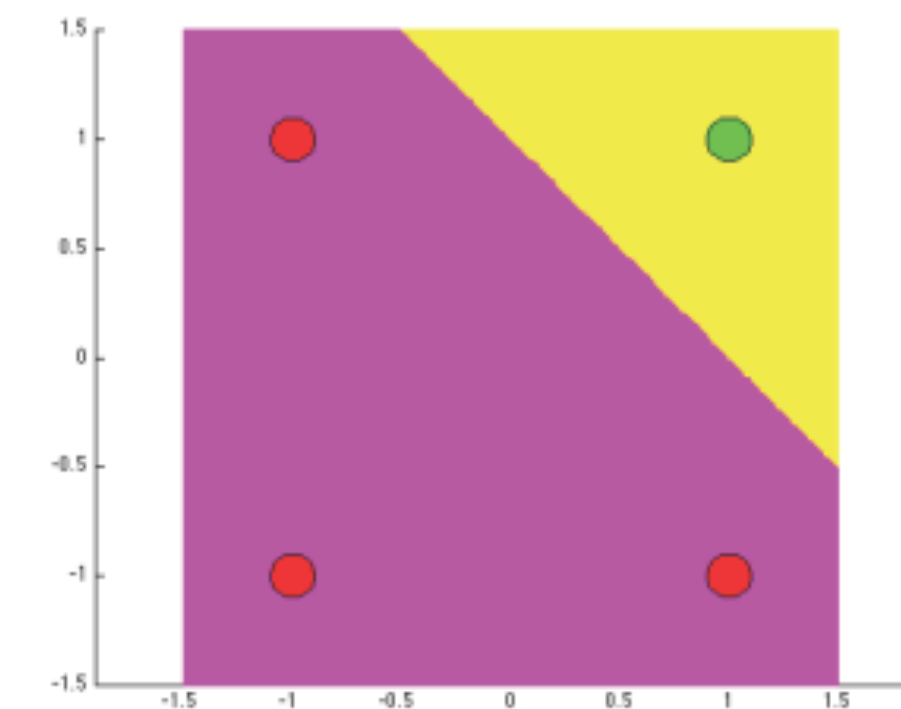
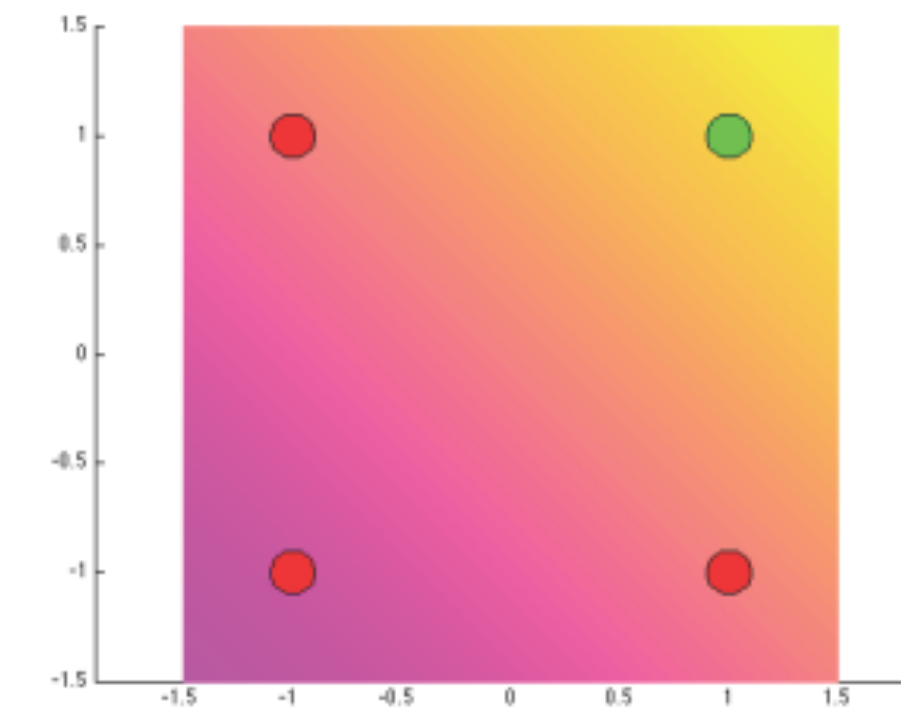


AND Function

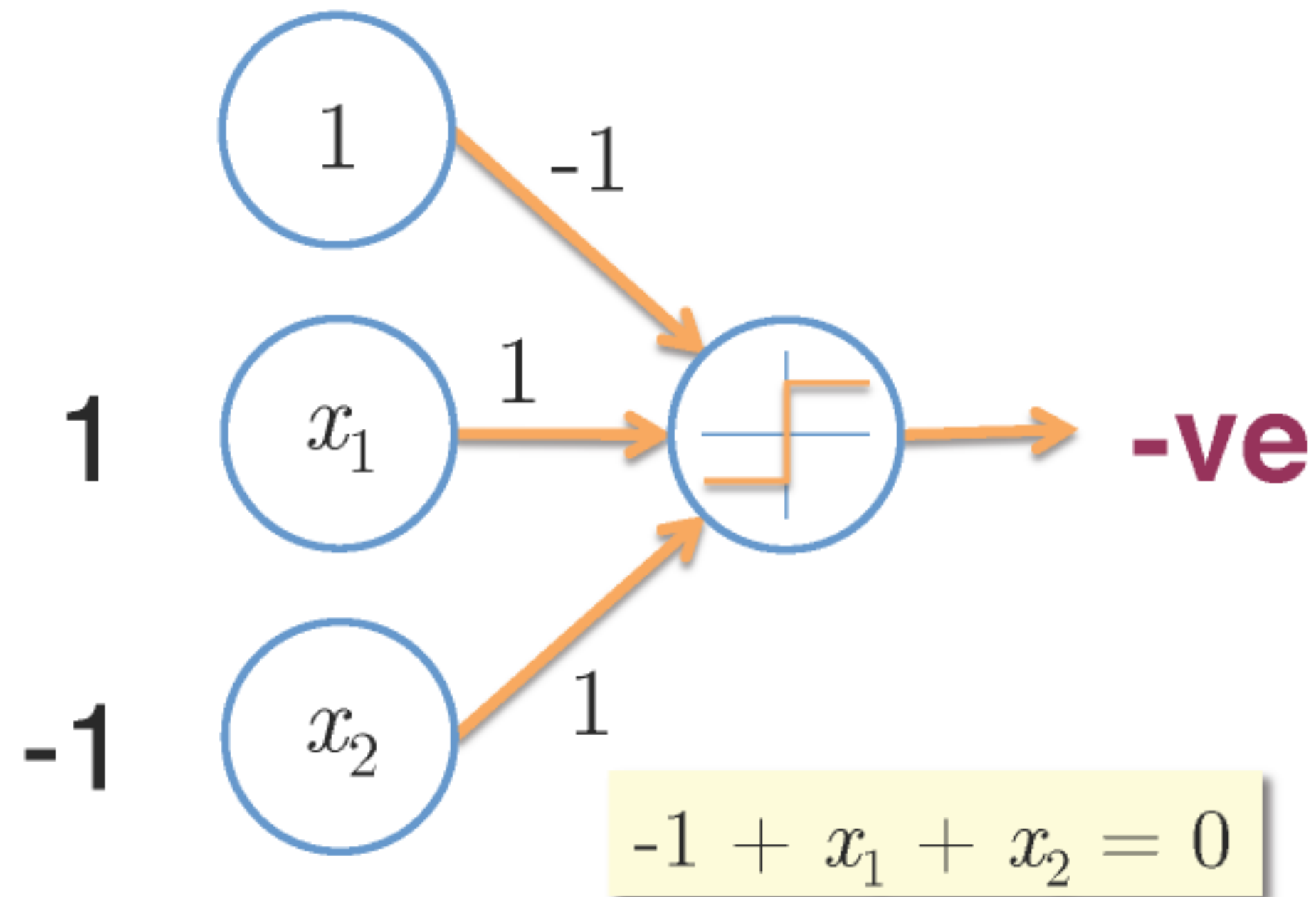


$$1 \times -1 + -1 \times 1 + 1 \times 1 = -1$$

x_1	x_2	AND(x_1, x_2)
-1	-1	-1
-1	1	-1
1	-1	-1
1	1	1

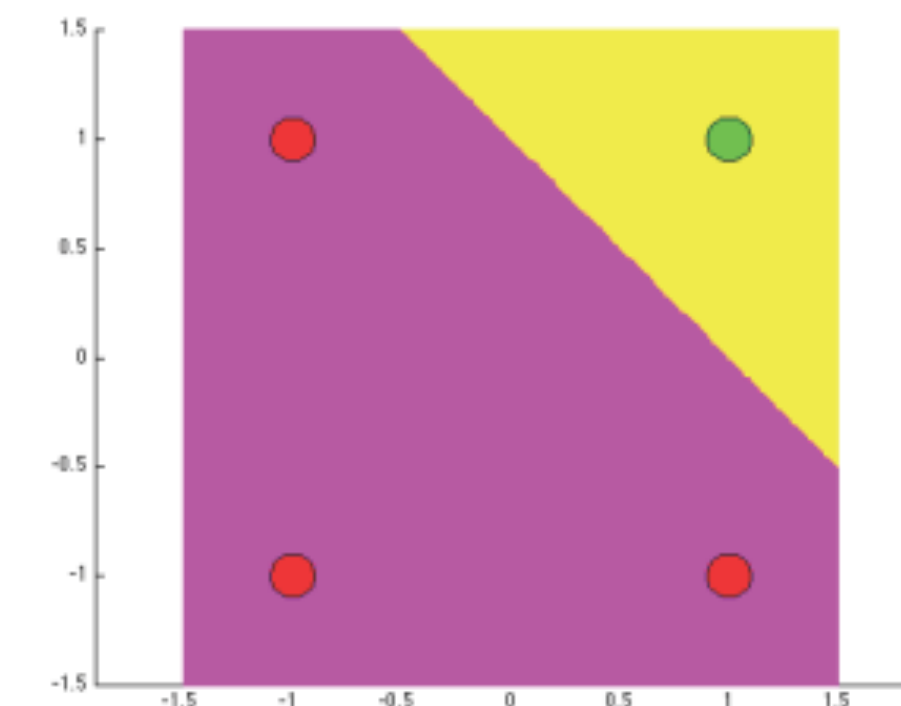
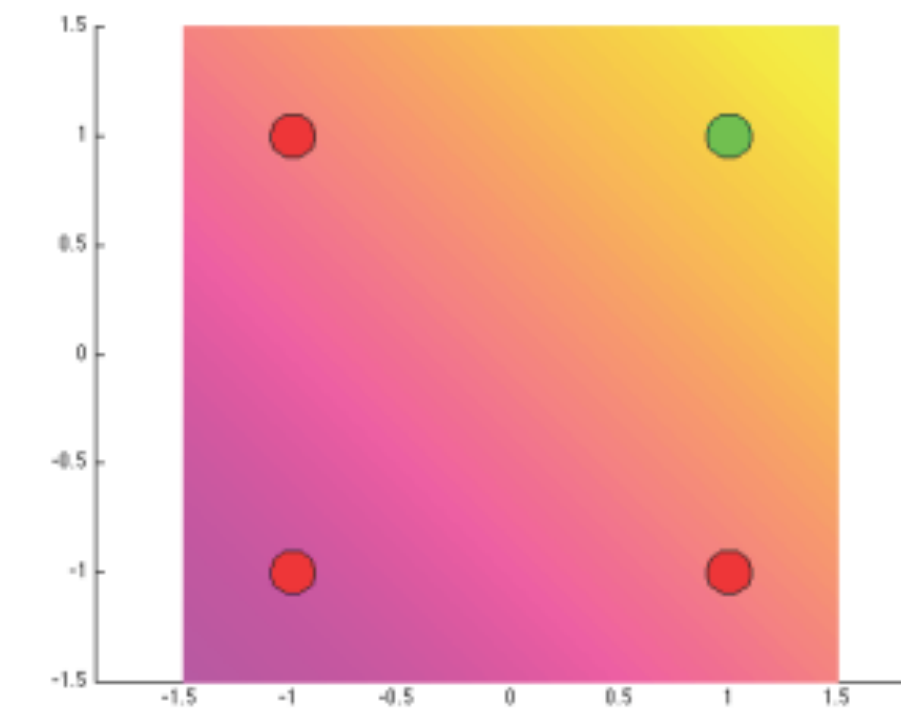


AND Function

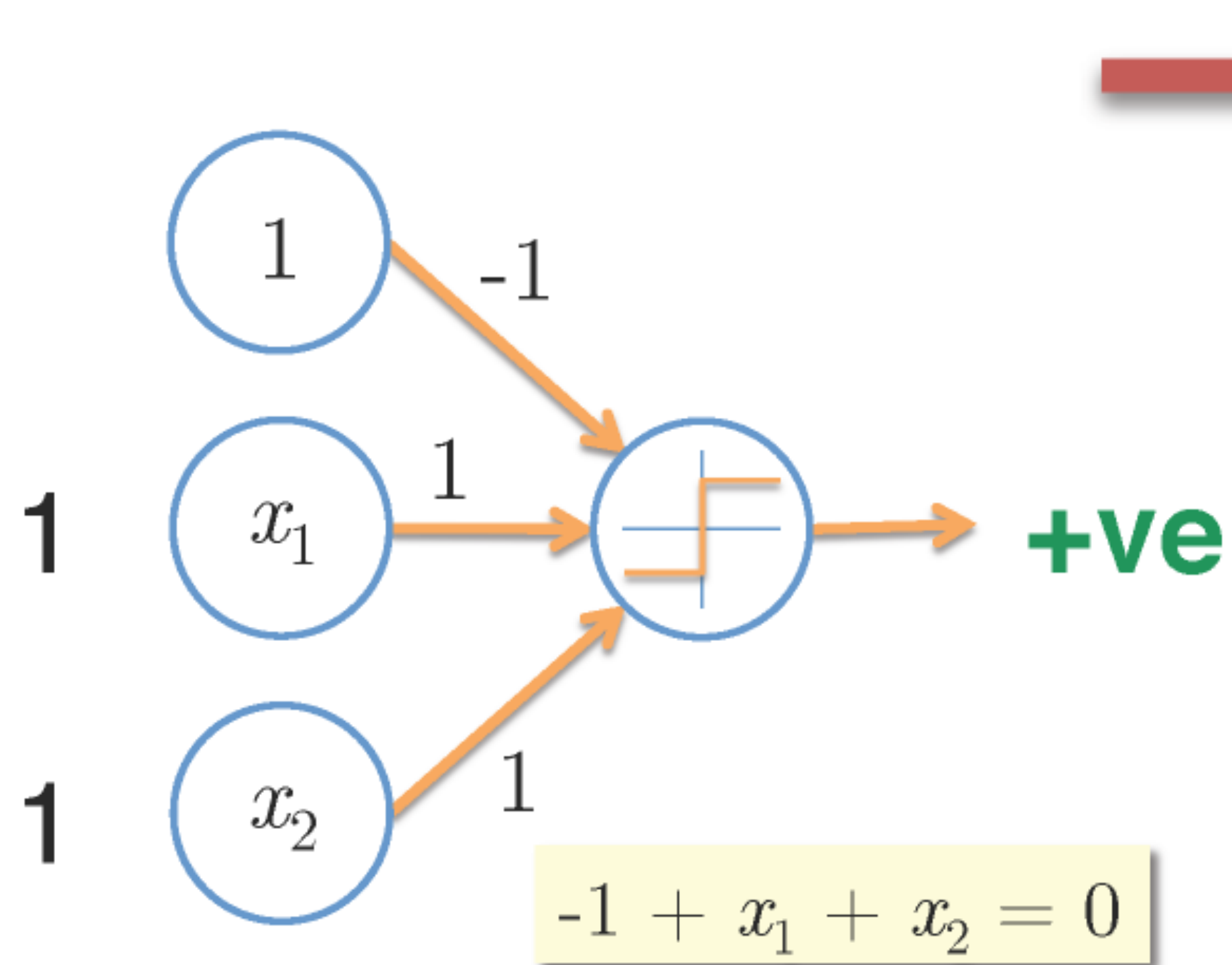


$$1 \times -1 + 1 \times 1 + -1 \times 1 = -1$$

x_1	x_2	AND(x_1, x_2)
-1	-1	-1
-1	1	-1
1	-1	-1
1	1	1

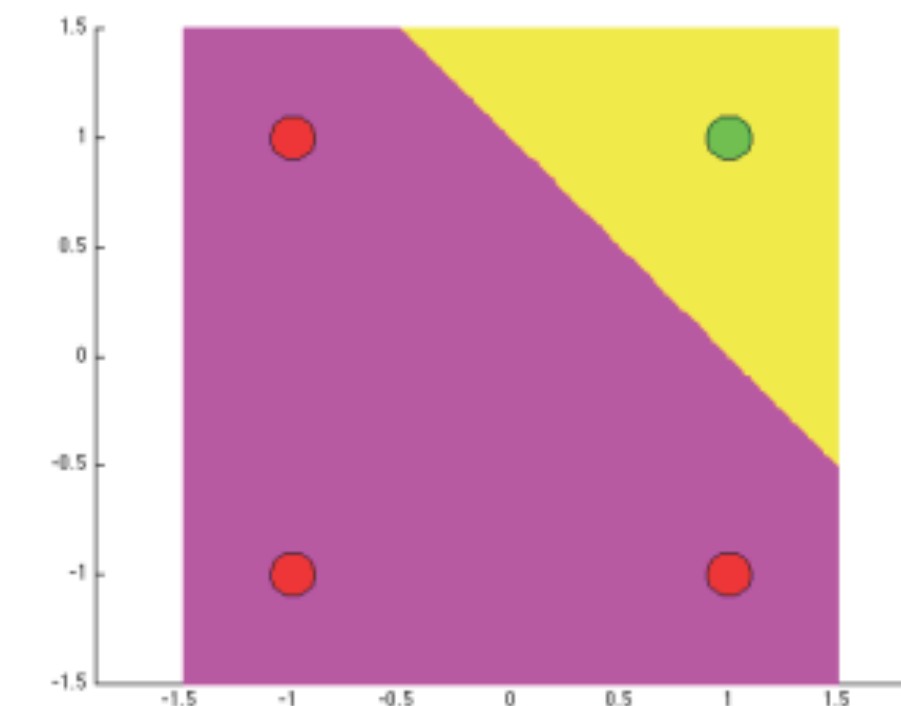
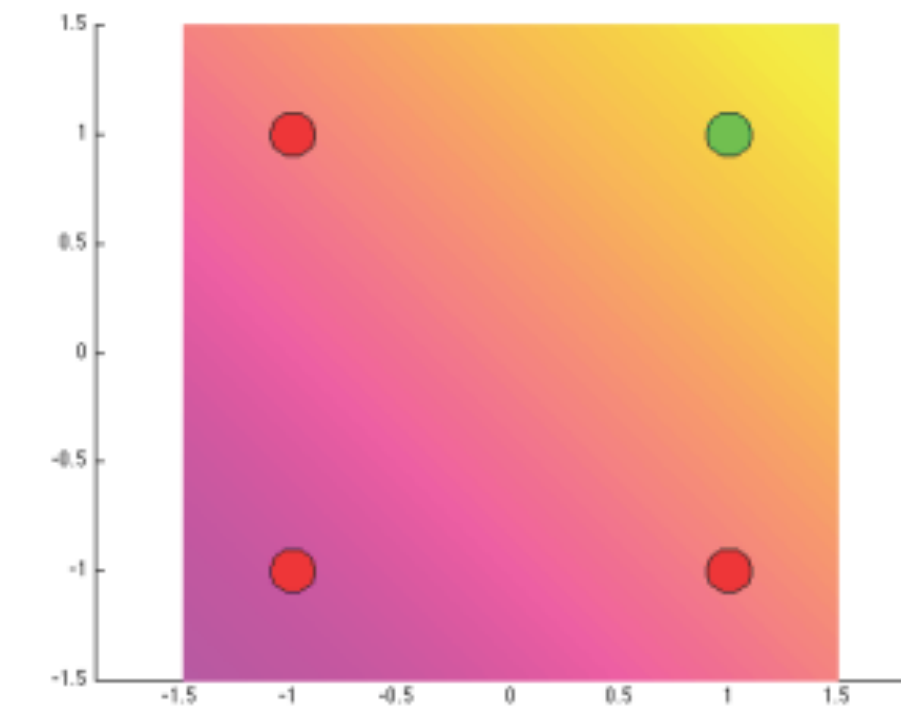


AND Function

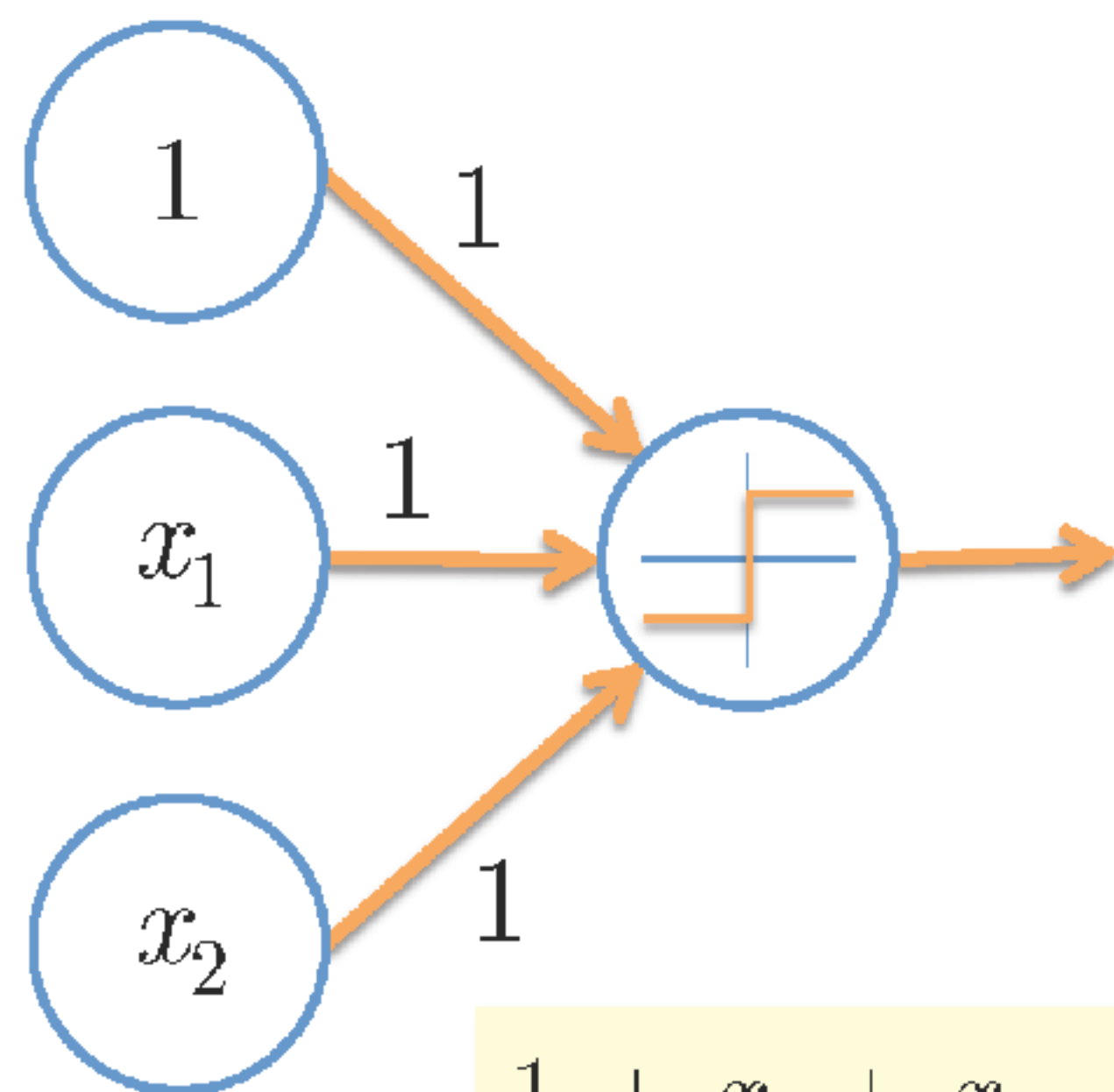


$$1 \times -1 + 1 \times 1 + 1 \times 1 = +1$$

x_1	x_2	AND(x_1, x_2)
-1	-1	-1
-1	1	-1
1	-1	-1
1	1	1



OR Function

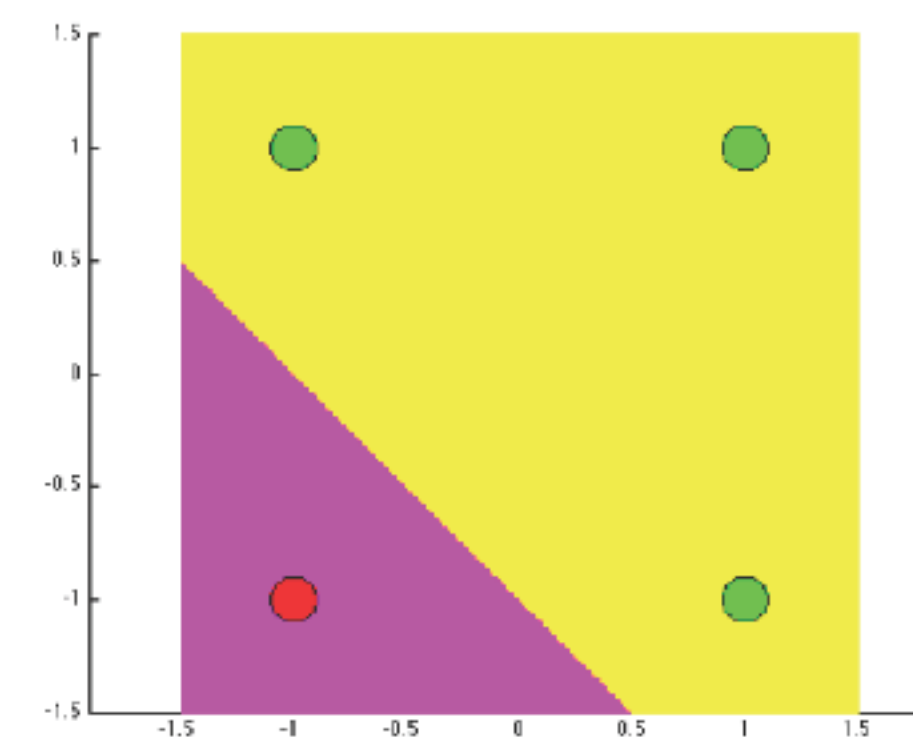
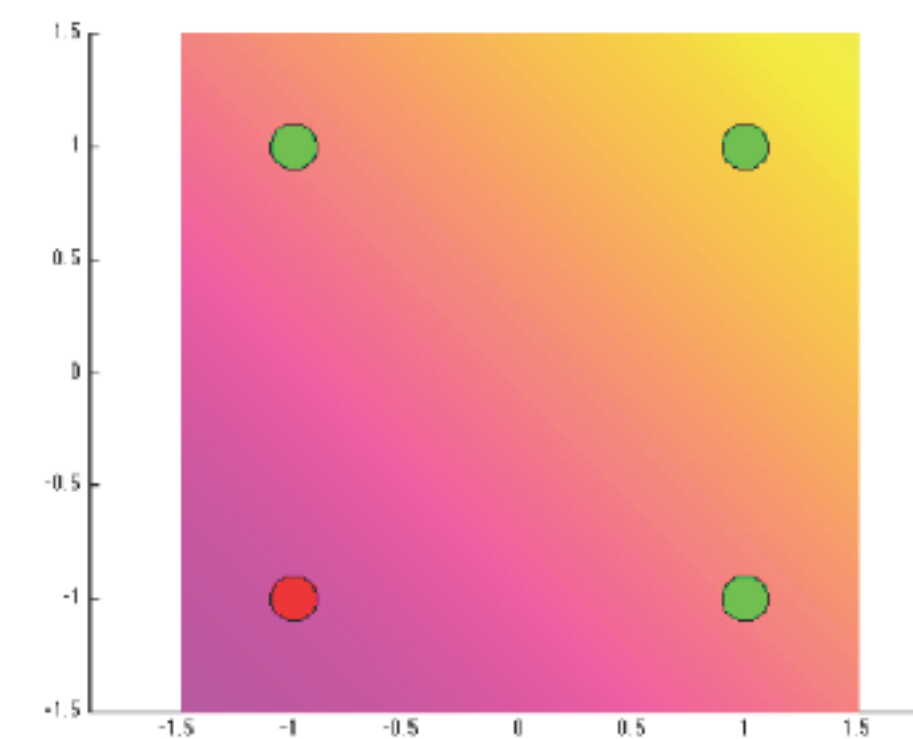


$$1 + x_1 + x_2 = 0$$

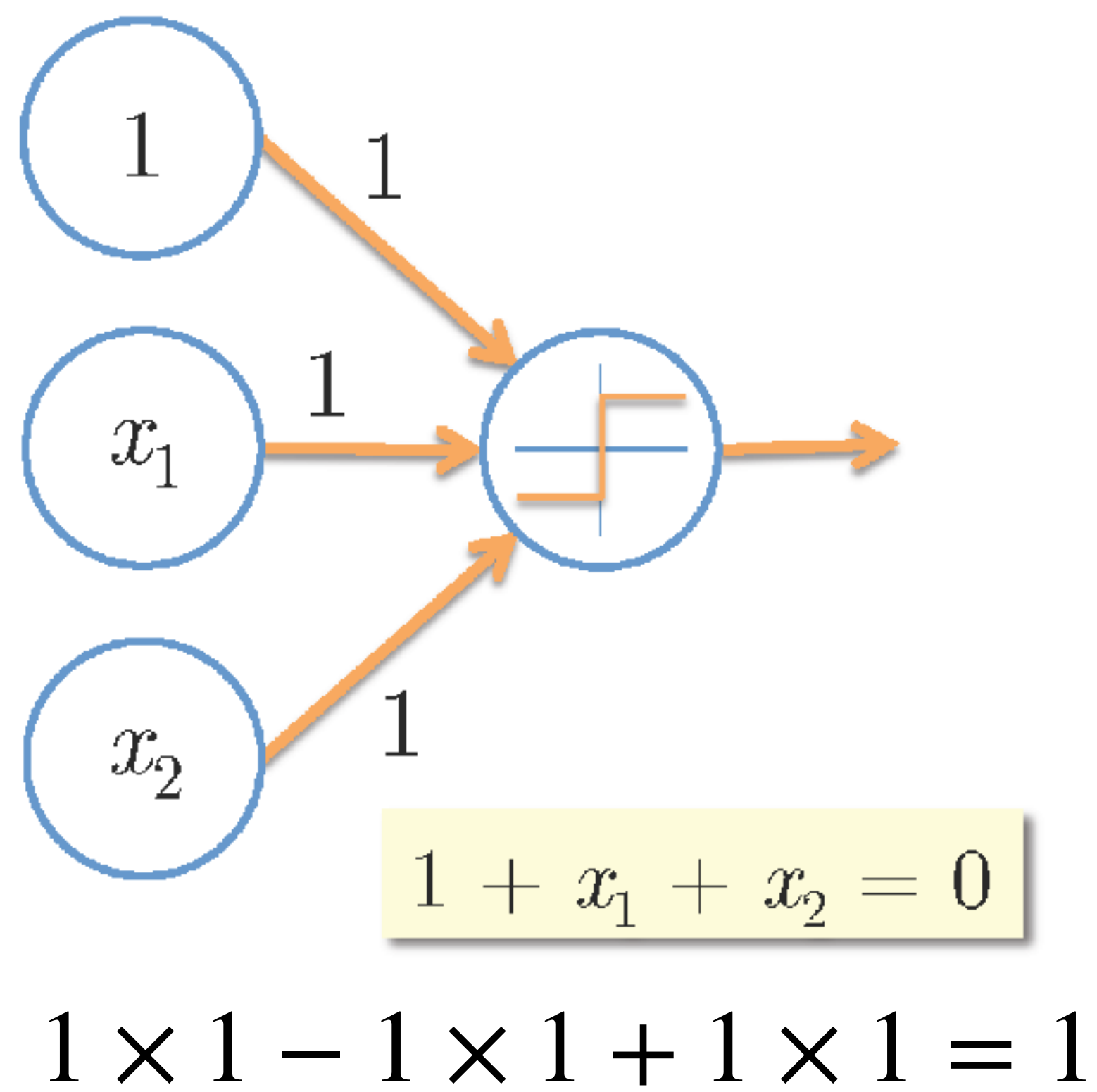
$$1 \times 1 - 1 \times 1 - 1 \times 1 = -1$$



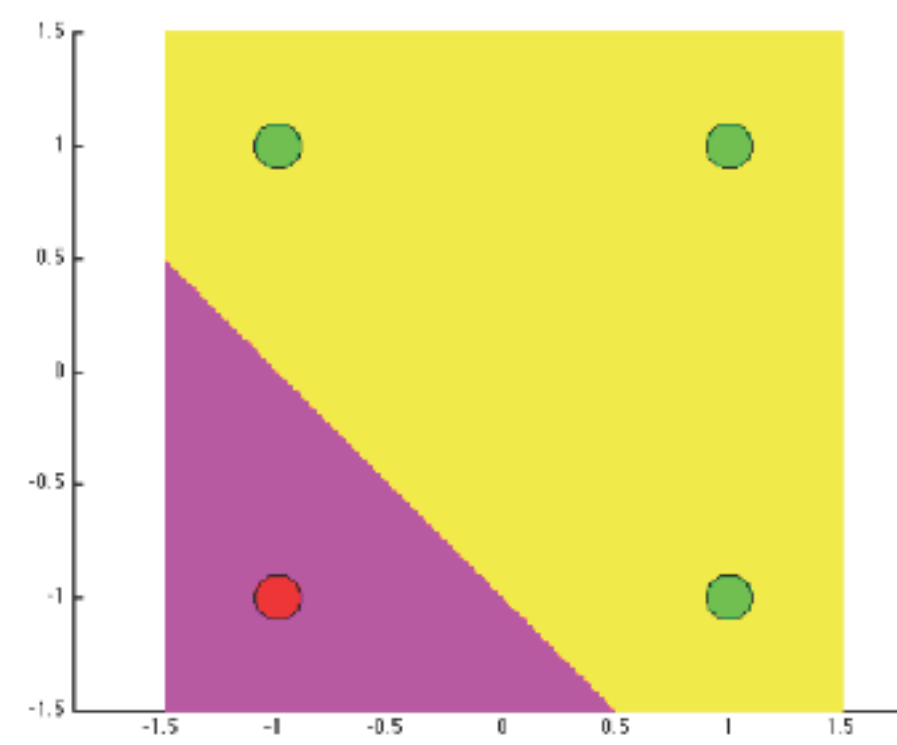
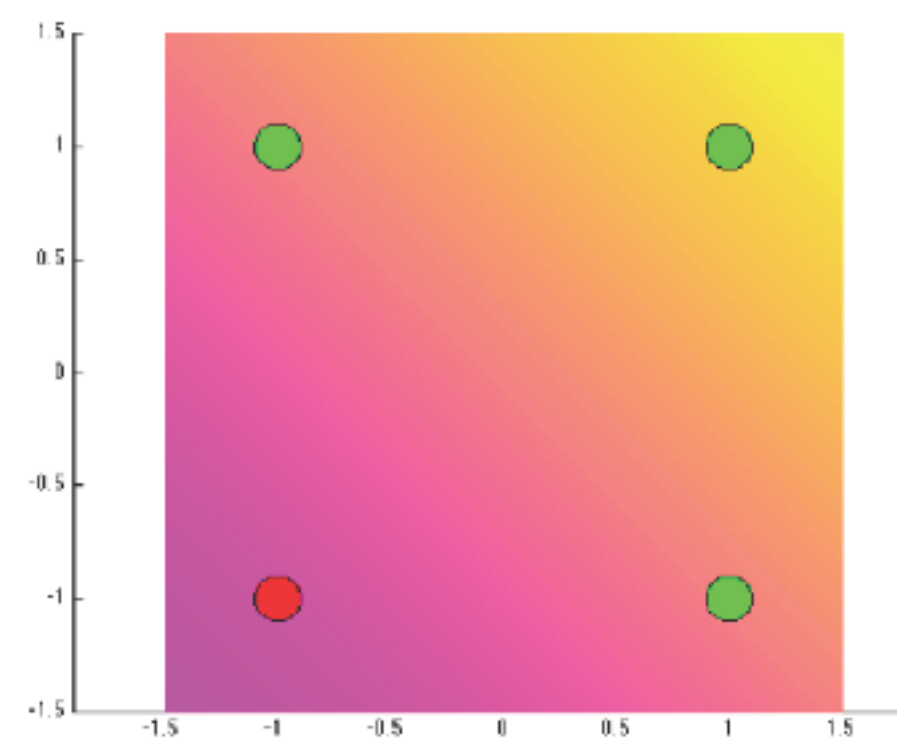
x_1	x_2	$\text{OR}(x_1, x_2)$
-1	-1	-1
-1	1	1
1	-1	1
1	1	1



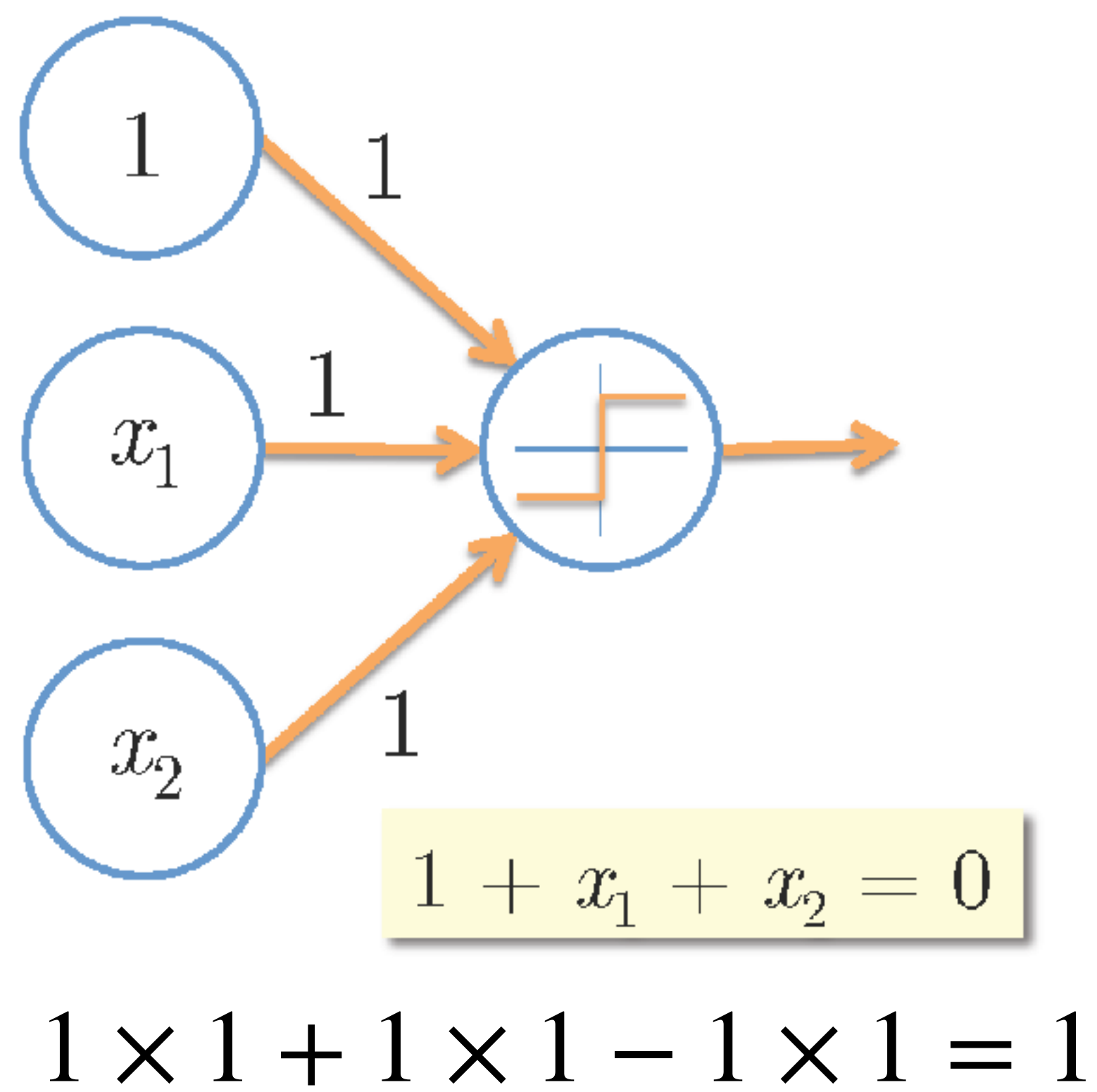
OR Function



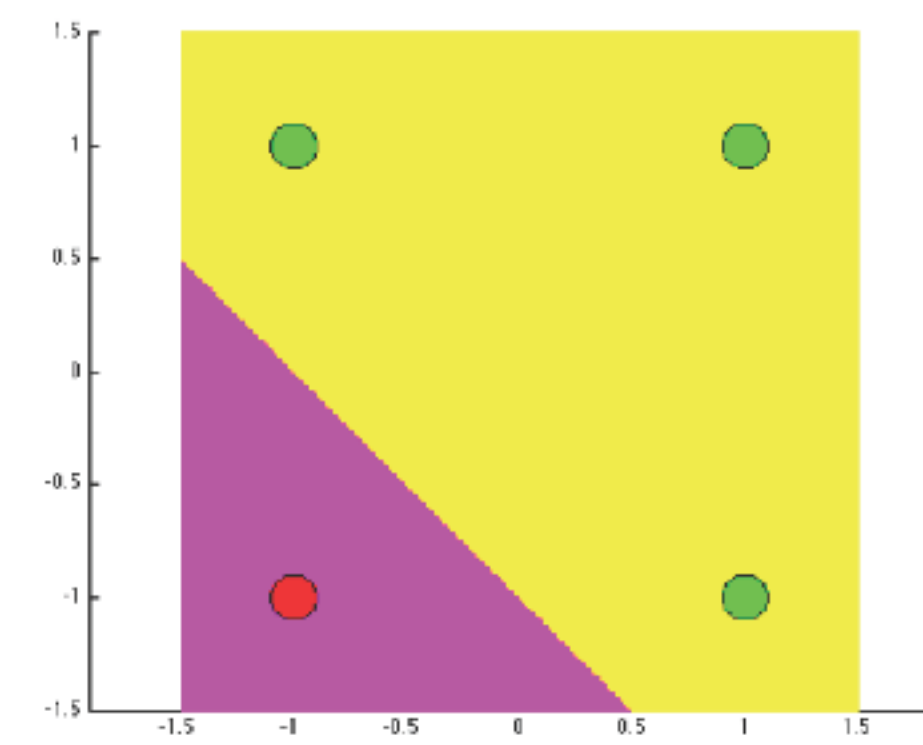
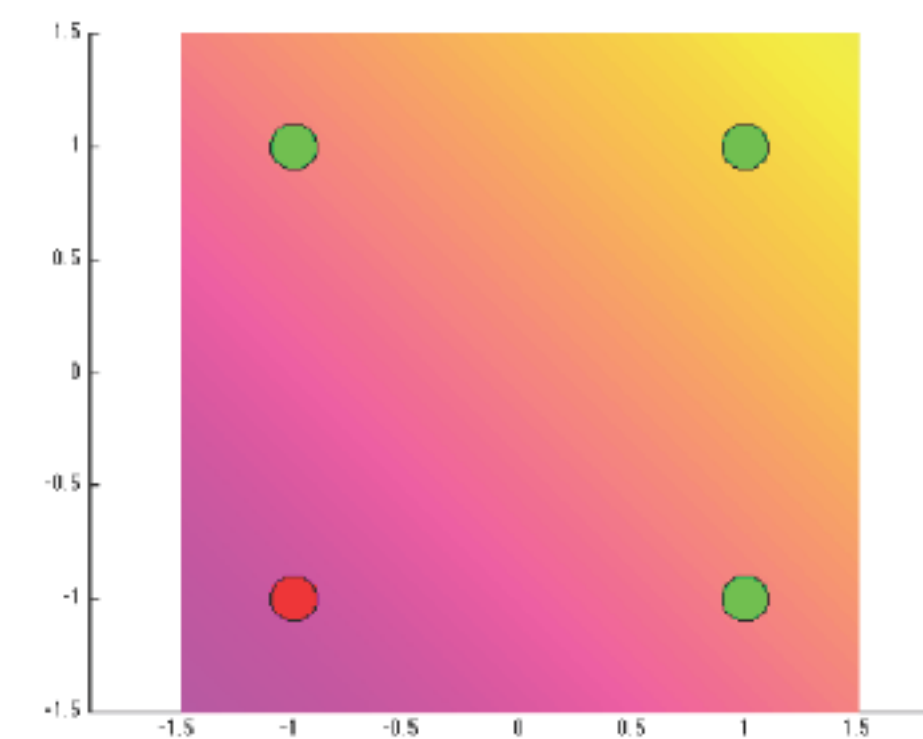
x_1	x_2	$\text{OR}(x_1, x_2)$
-1	-1	-1
-1	1	1
1	-1	1
1	1	1



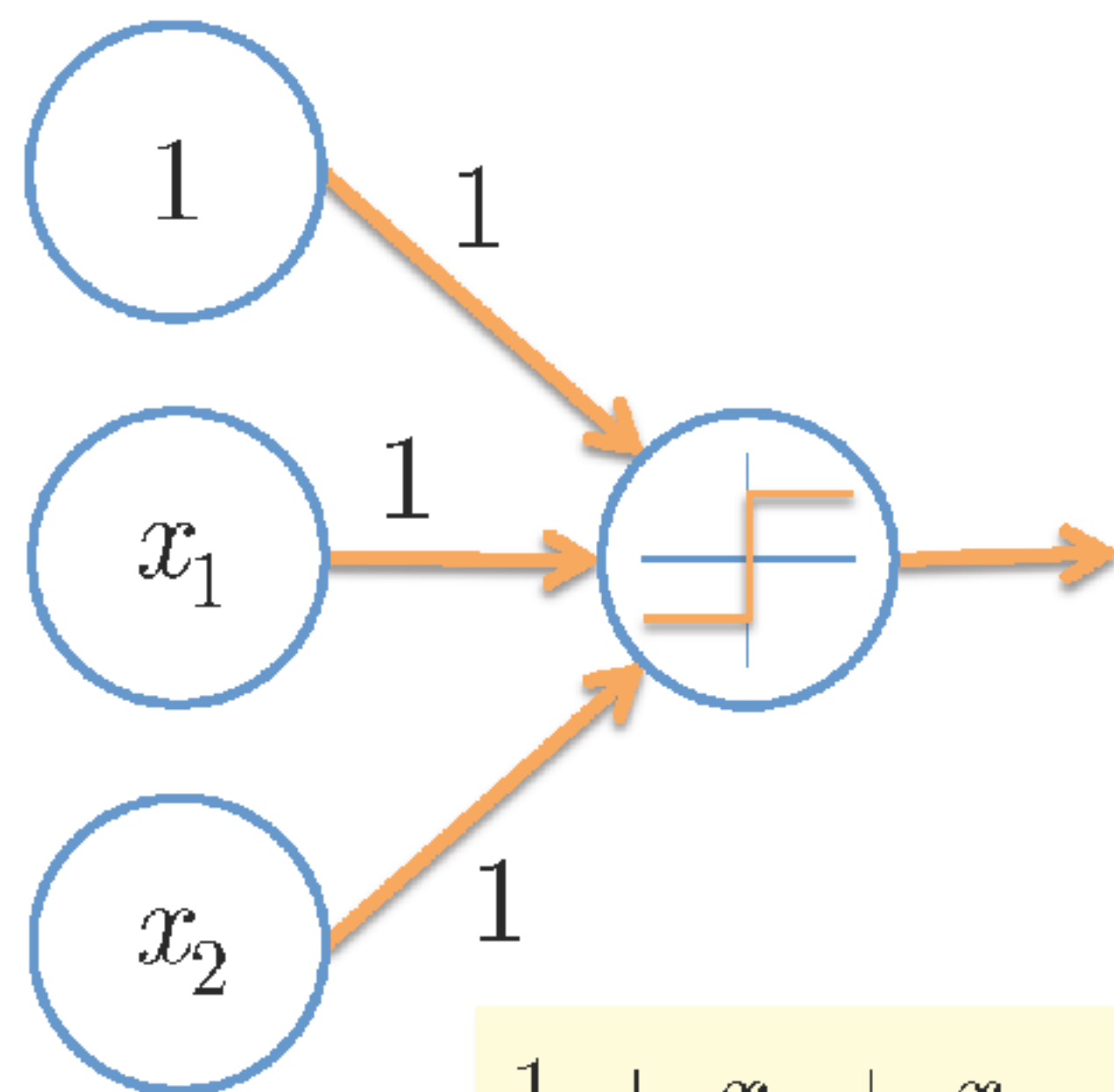
OR Function



x_1	x_2	$\text{OR}(x_1, x_2)$
-1	-1	-1
-1	1	1
1	-1	1
1	1	1



OR Function

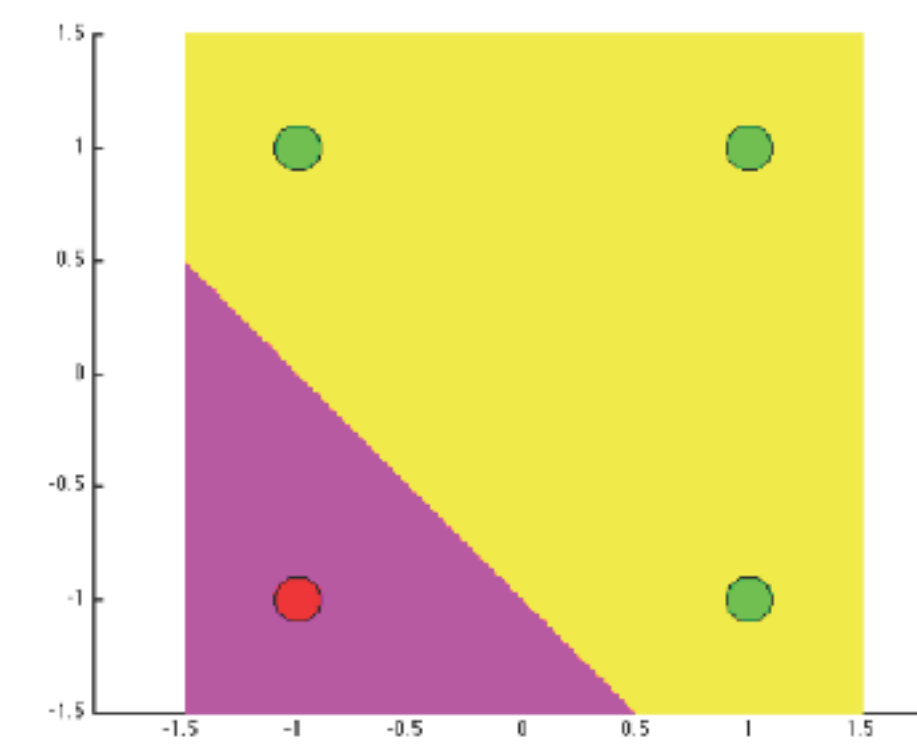
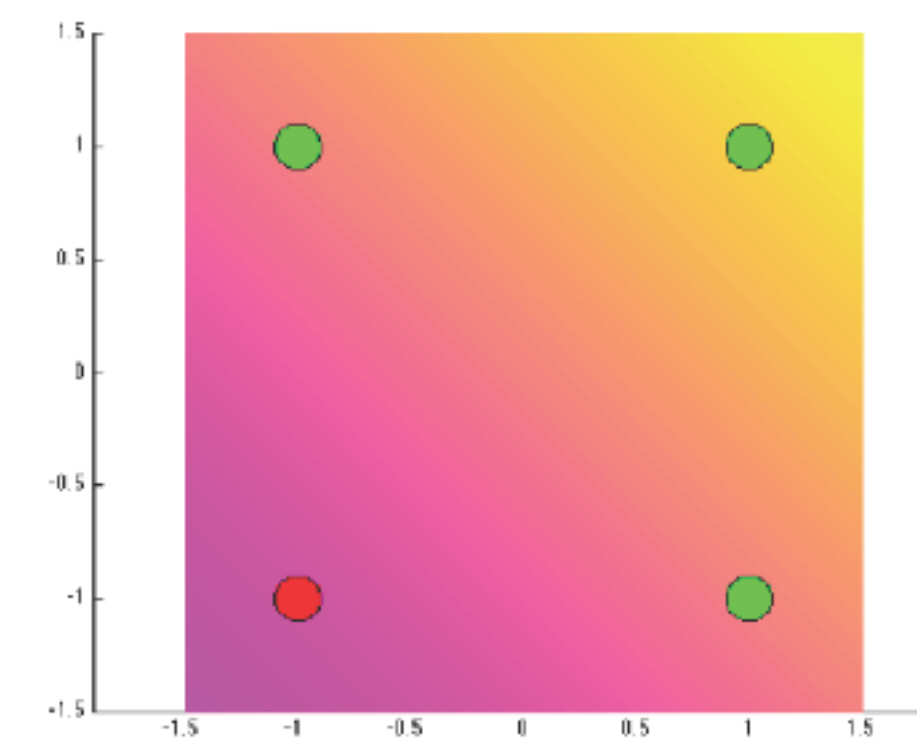


$$1 + x_1 + x_2 = 0$$

$$1 \times 1 + 1 \times 1 + 1 \times 1 = 3$$



x_1	x_2	$\text{OR}(x_1, x_2)$
-1	-1	-1
-1	1	1
1	-1	1
1	1	1

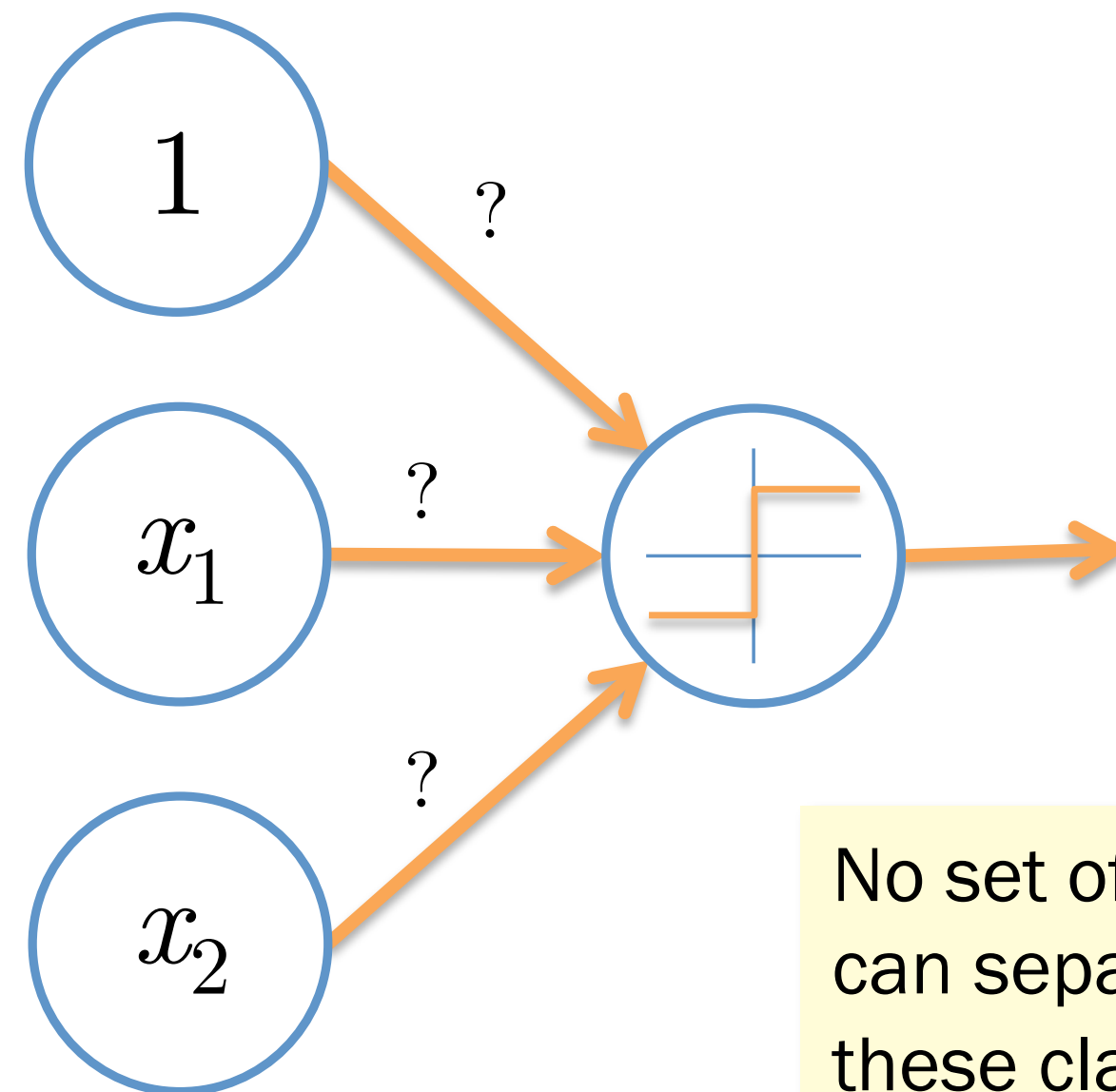


What to do in case of non-linear problem? XOR

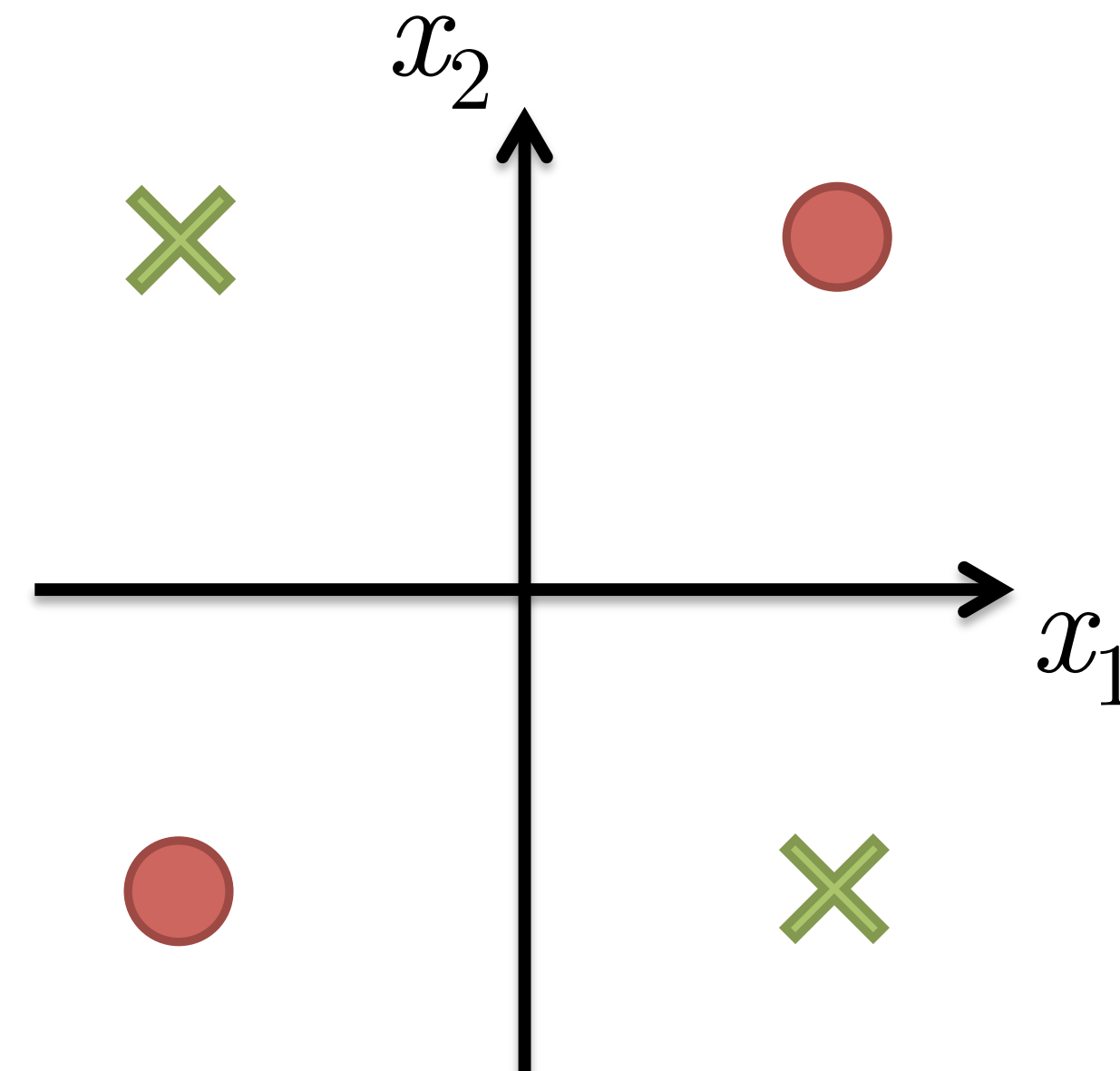
Idea: Stack a bunch of them together

Consider XOR

x_1	x_2	$\text{XOR}(x_1, x_2)$
-1	-1	-1
-1	1	1
1	-1	1
1	1	-1

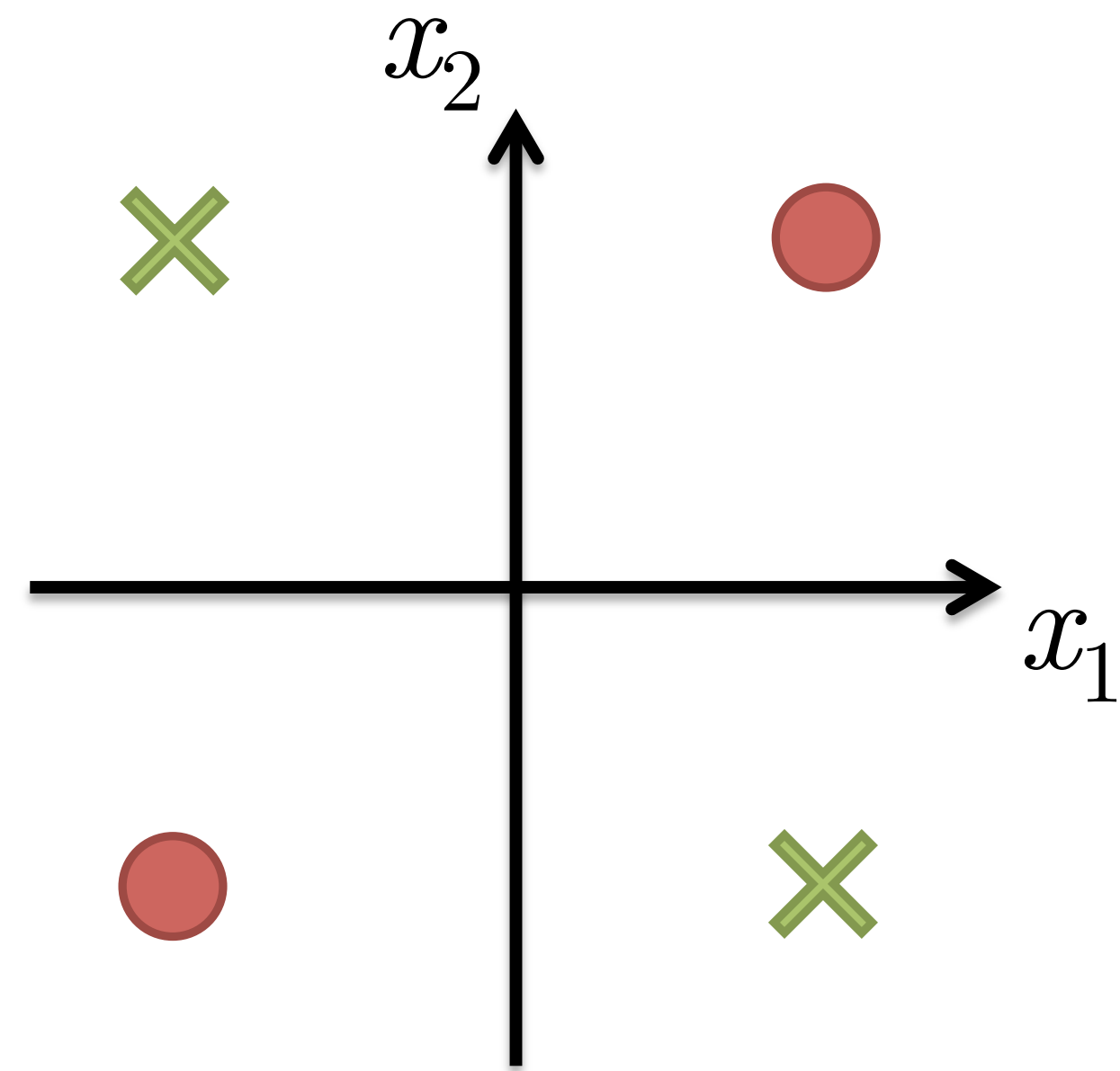
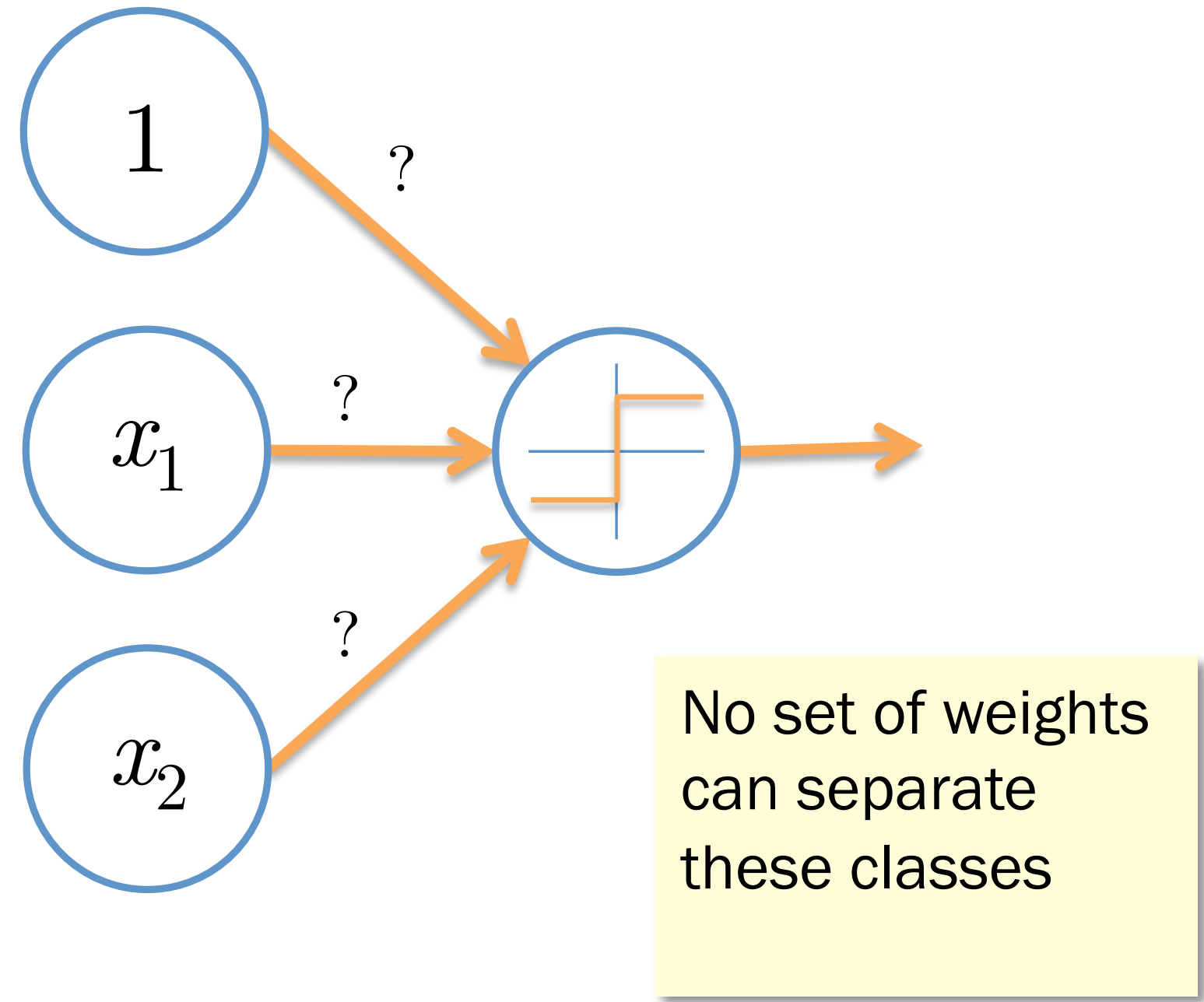


No set of weights
can separate
these classes



XOR Function

x_1	x_2	$\text{XOR}(x_1, x_2)$
-1	-1	-1
-1	1	1
1	-1	1
1	1	-1



XOR

$$\text{XOR}(x_1, x_2) = \text{OR}(\text{AND}(x_1, \neg x_2), \text{AND}(\neg x_1, x_2))$$

x_1	x_2	$\text{AND}(x_1, \neg x_2)$	$\text{AND}(\neg x_1, x_2)$	$\text{OR}(\text{AND}(x_1, \neg x_2), \text{AND}(\neg x_1, x_2))$
-1	-1			
-1	1			
1	-1			
1	1			

XOR

$$\text{XOR}(x_1, x_2) = \text{OR}(\text{AND}(x_1, \neg x_2), \text{AND}(\neg x_1, x_2))$$

x_1	x_2	$\text{AND}(x_1, \neg x_2)$	$\text{AND}(\neg x_1, x_2)$	$\text{OR}(\text{AND}(x_1, \neg x_2), \text{AND}(\neg x_1, x_2))$
-1	-1	-1		
-1	1	-1		
1	-1	1		
1	1	-1		

XOR

$$\text{XOR}(x_1, x_2) = \text{OR}(\text{AND}(x_1, \neg x_2), \text{AND}(\neg x_1, x_2))$$

x_1	x_2	$\text{AND}(x_1, \neg x_2)$	$\text{AND}(\neg x_1, x_2)$	$\text{OR}(\text{AND}(x_1, \neg x_2), \text{AND}(\neg x_1, x_2))$
-1	-1	-1	-1	
-1	1	-1	1	
1	-1	1	-1	
1	1	-1	-1	

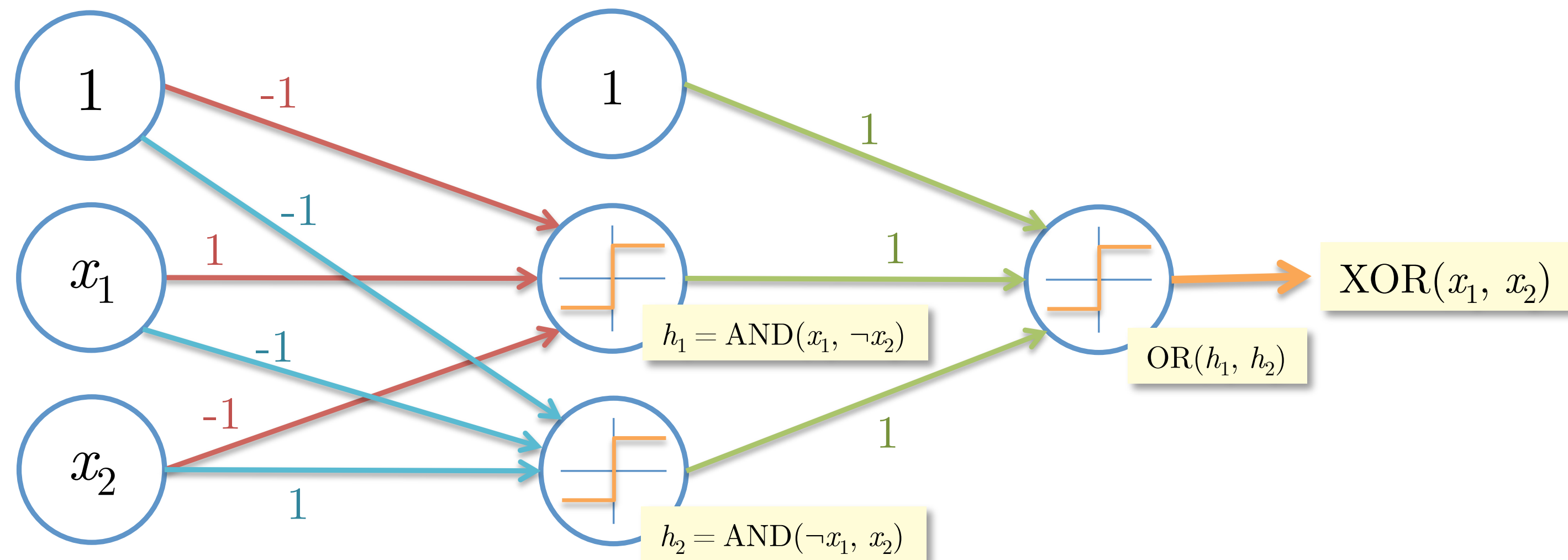
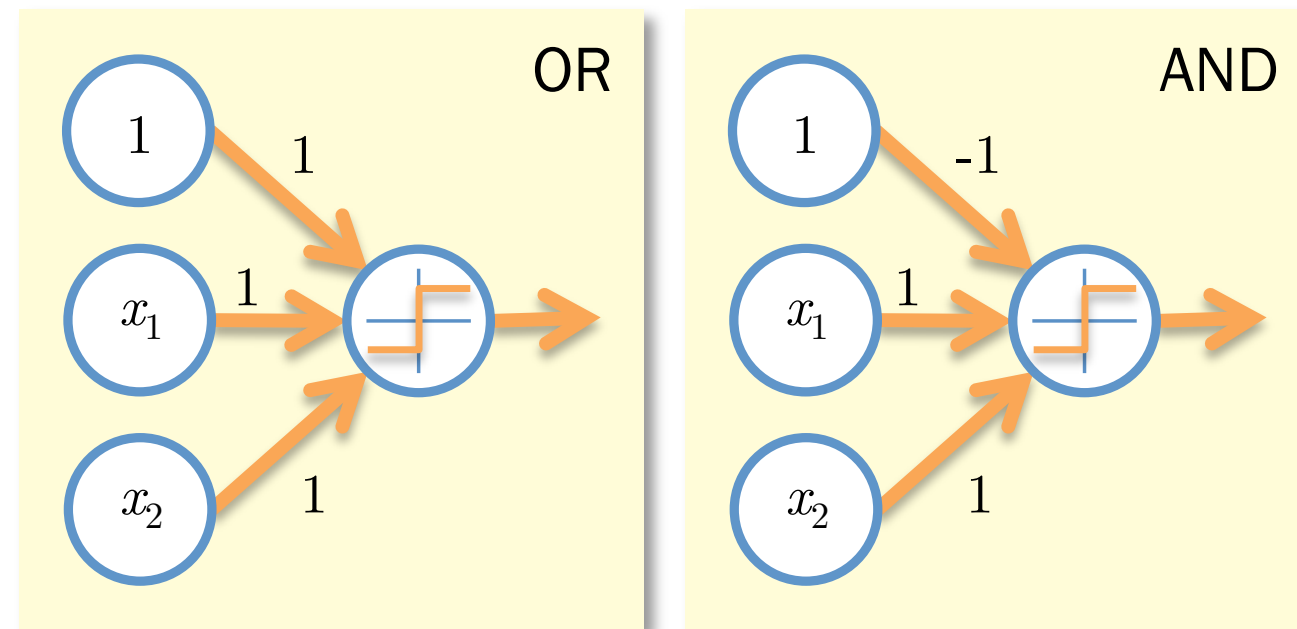
XOR

$$\text{XOR}(x_1, x_2) = \text{OR}(\text{AND}(x_1, \neg x_2), \text{AND}(\neg x_1, x_2))$$

x_1	x_2	$\text{AND}(x_1, \neg x_2)$	$\text{AND}(\neg x_1, x_2)$	$\text{OR}(\text{AND}(x_1, \neg x_2), \text{AND}(\neg x_1, x_2))$
-1	-1	-1	-1	-1
-1	1	-1	1	1
1	-1	1	-1	1
1	1	-1	-1	-1

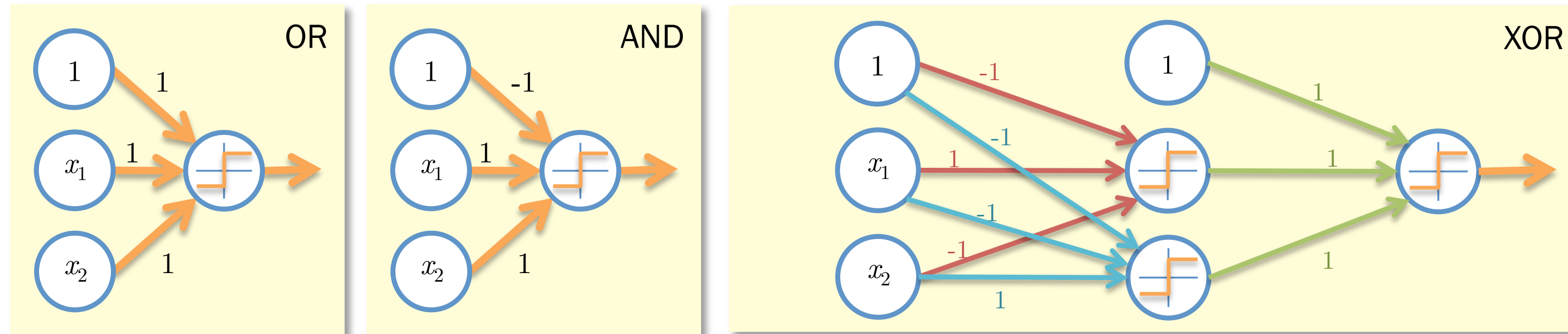
XOR

$$\text{XOR}(x_1, x_2) = \text{OR}(\text{AND}(x_1, \neg x_2), \text{AND}(\neg x_1, x_2))$$

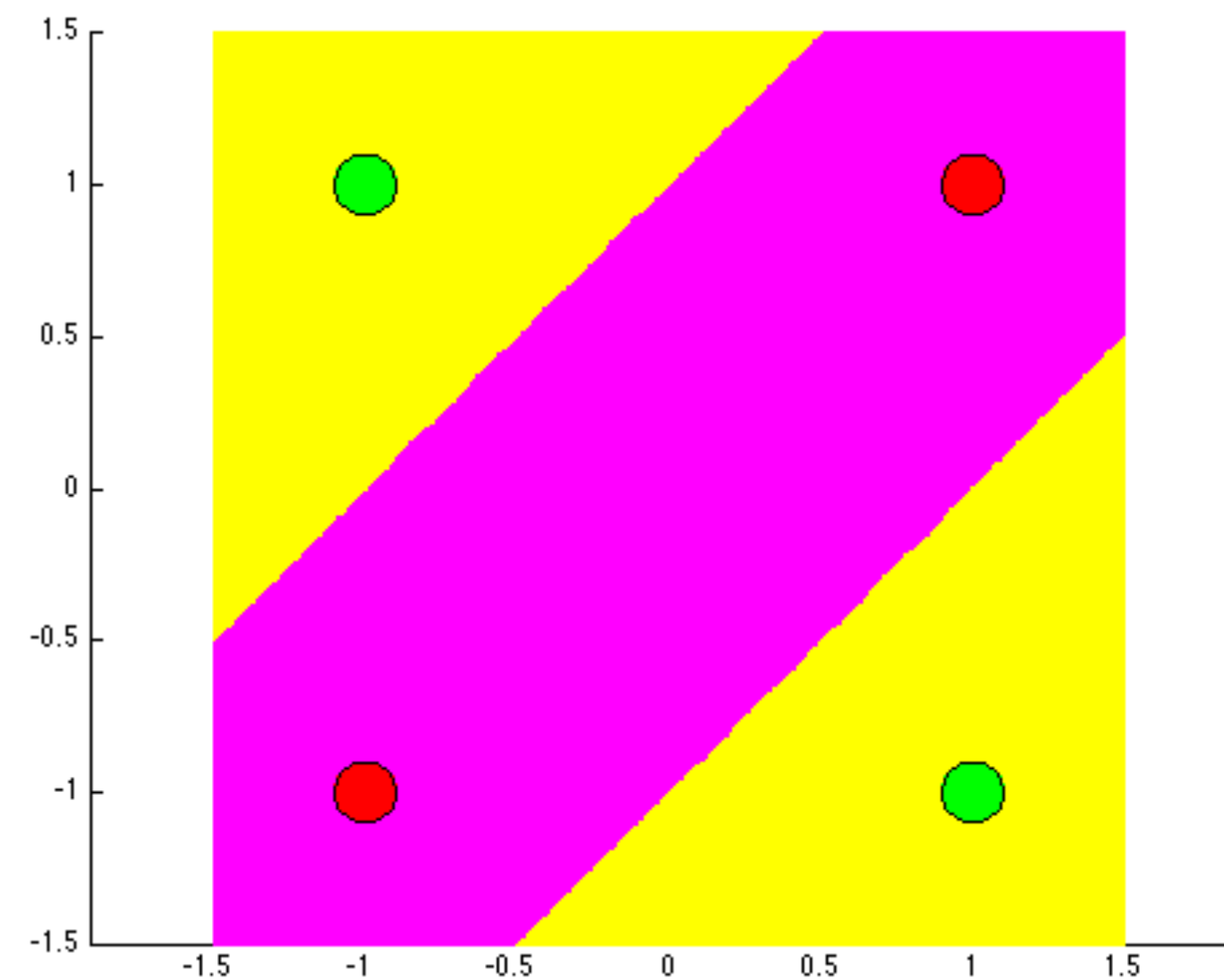


XOR

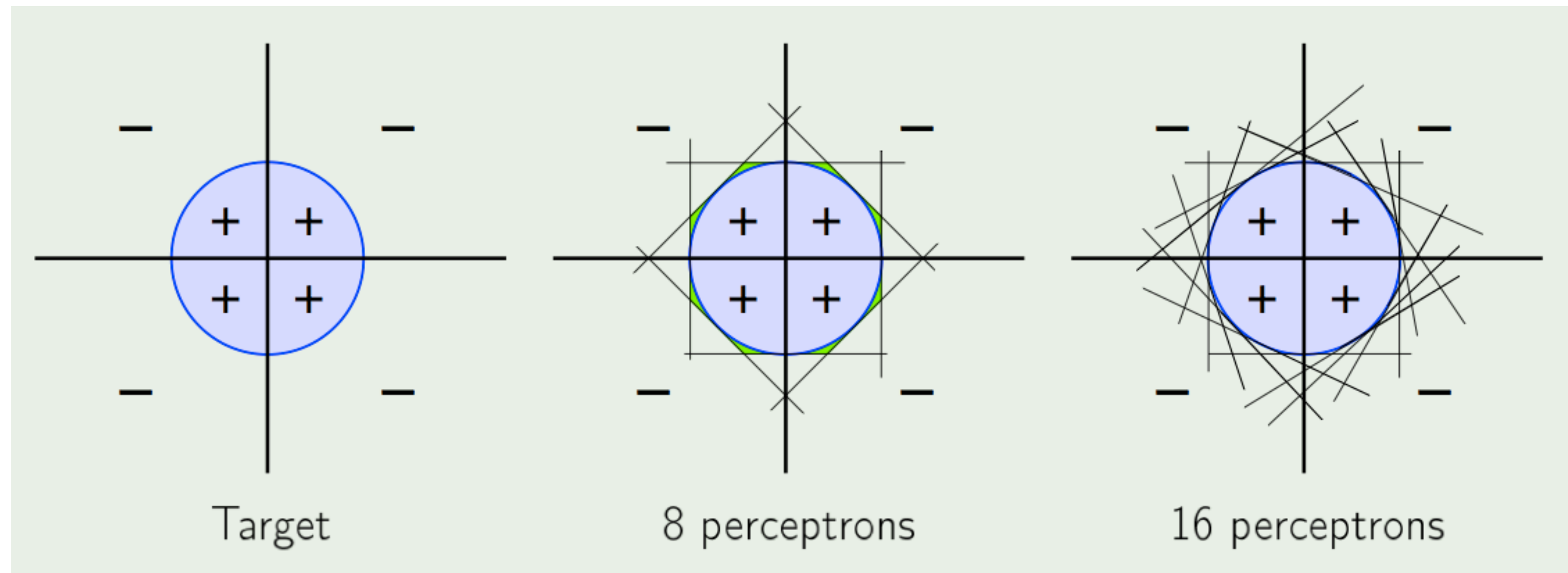
$$\text{XOR}(x_1, x_2) = \text{OR}(\text{AND}(x_1, \neg x_2), \text{AND}(\neg x_1, x_2))$$



By combining two Perceptrons, we are able to create a **non-linear** decision boundary

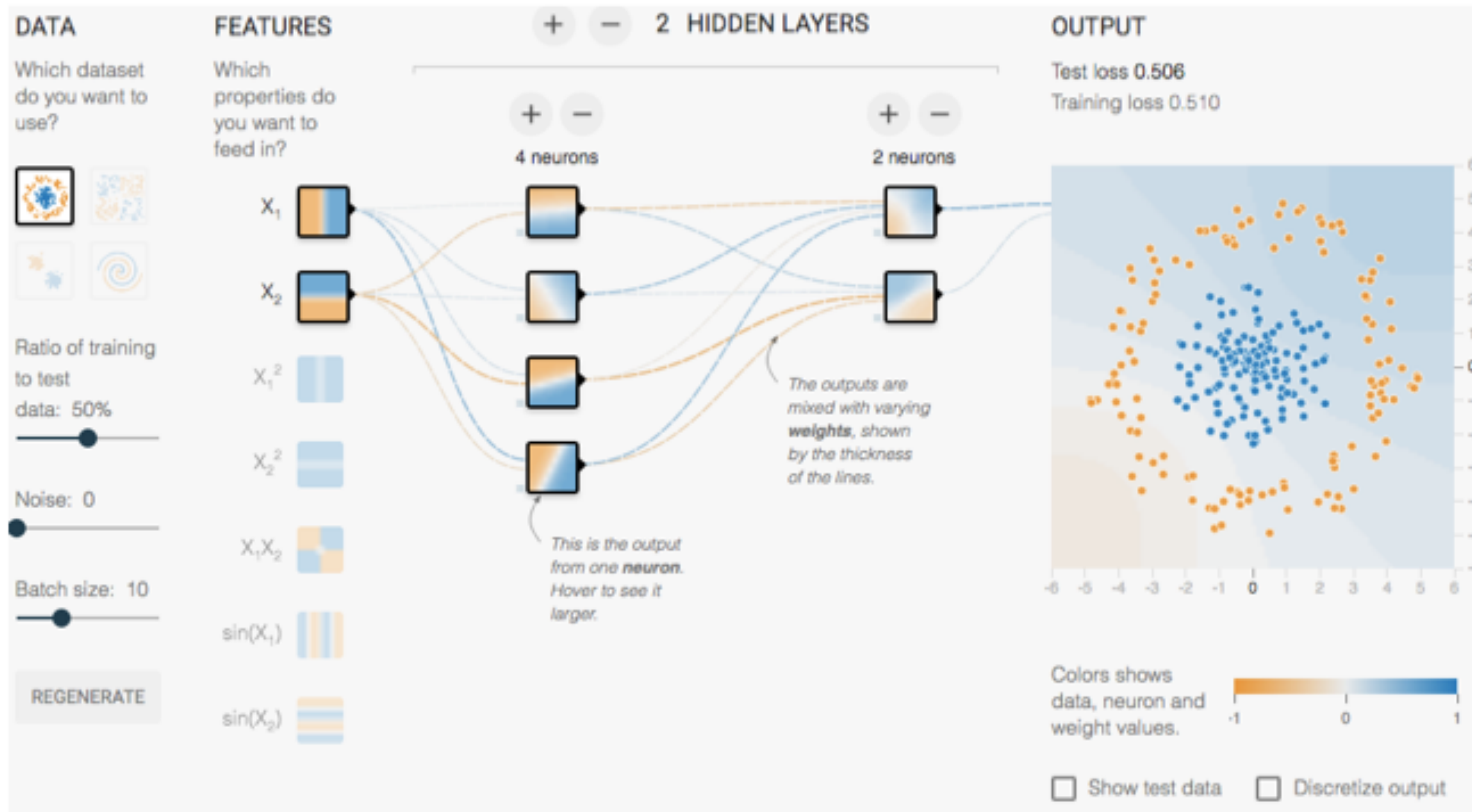


A Powerful Model



Demo:

<http://playground.tensorflow.org/>

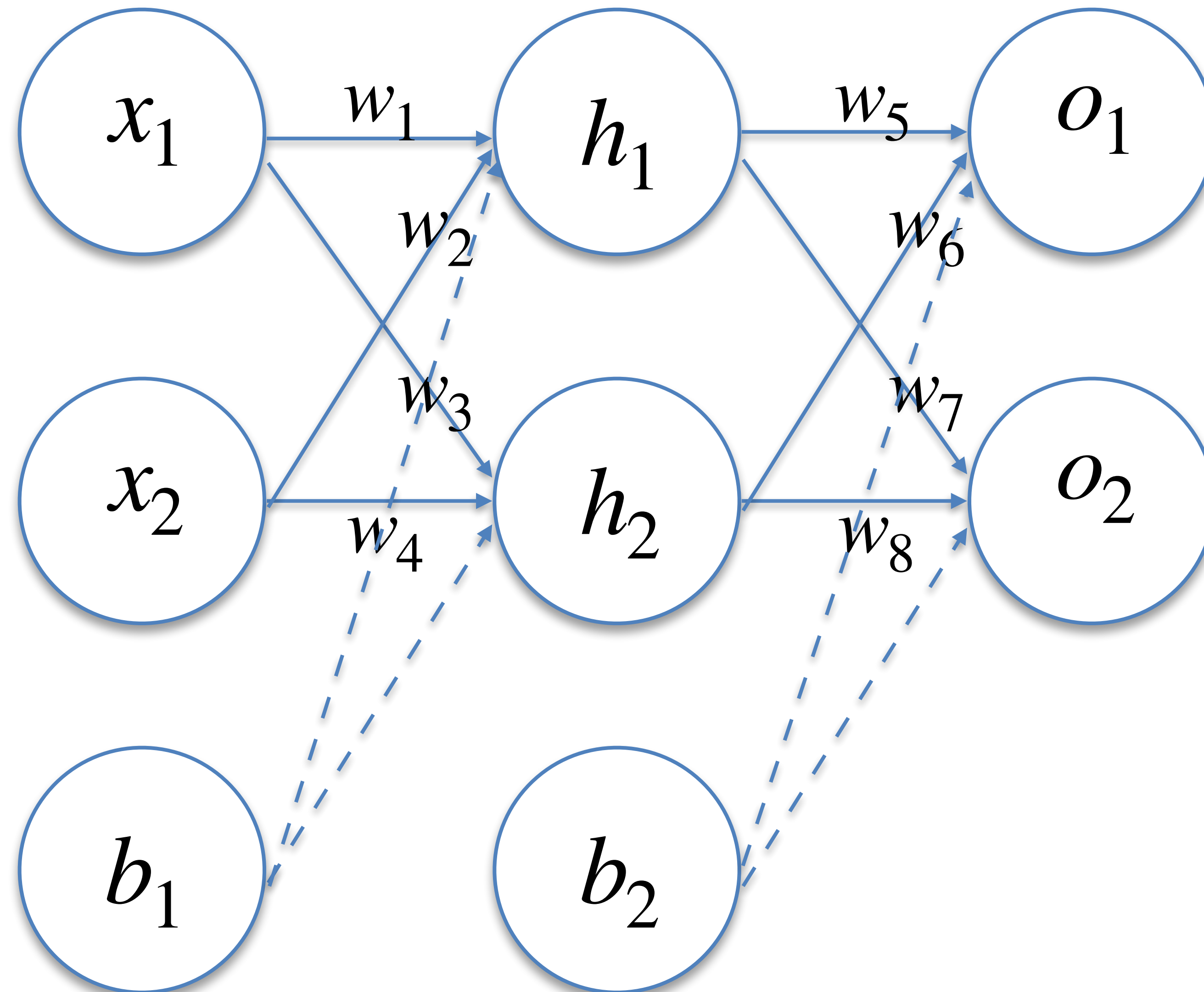


Readings

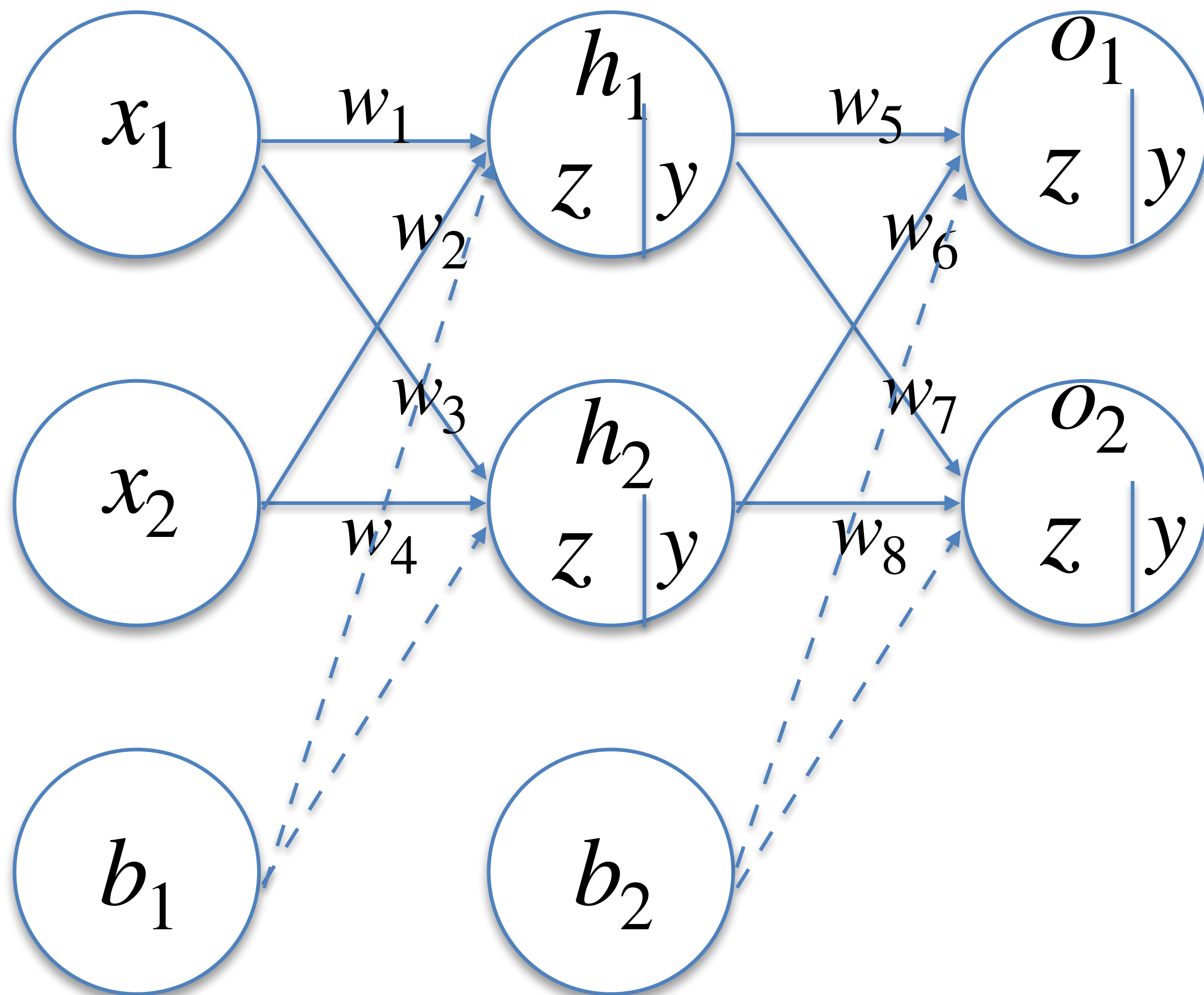


- Gradient Descent
- Forward and Backward pass of Neural Network
- [Nikhil Buduma CH1] Neural, Perceptron, Regression, Logistic Regression
- [Nikhil Buduma CH2] Training feedforward NN, back prop., Gradient Descent

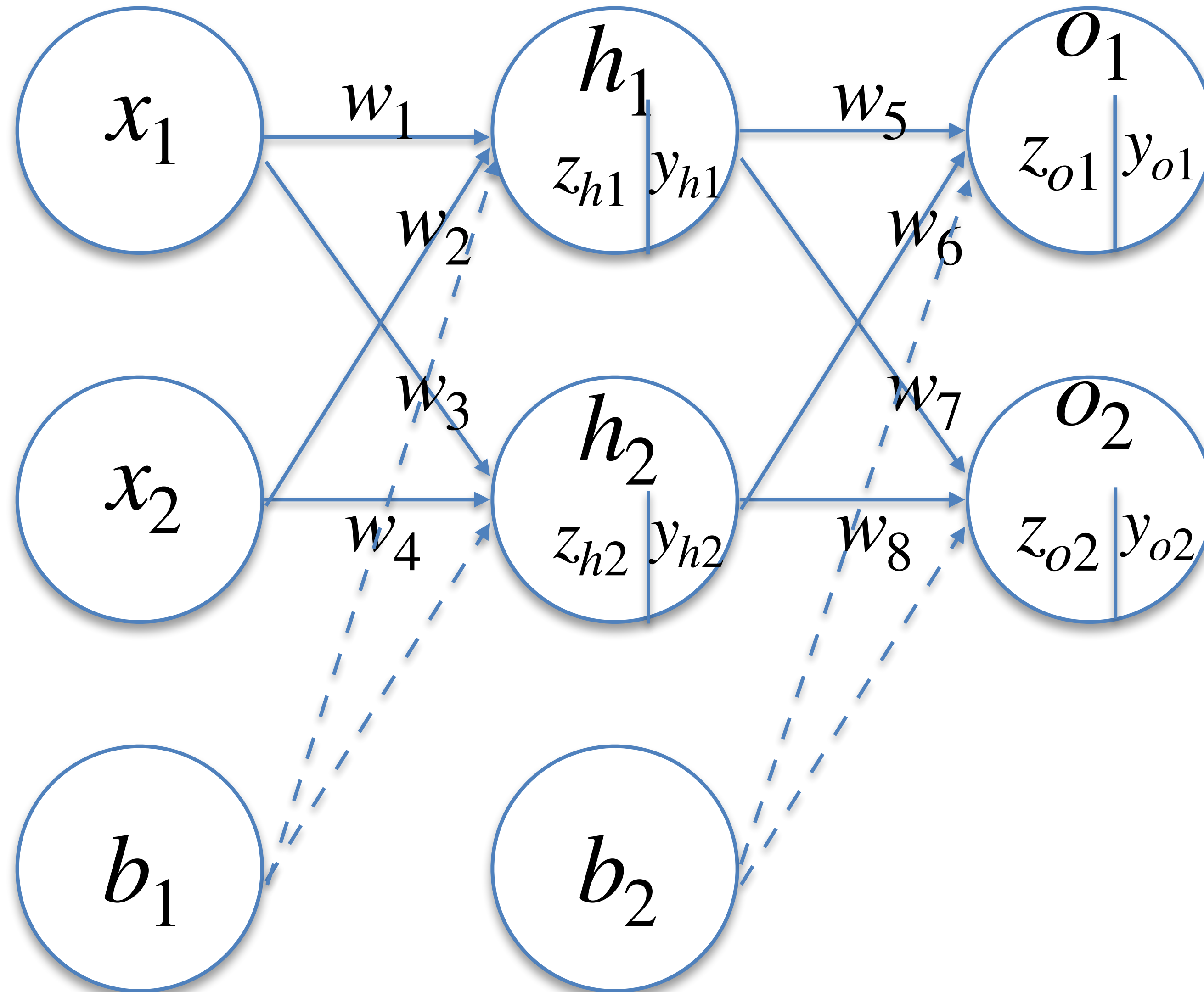
Multilayer Networks



Multilayer Networks

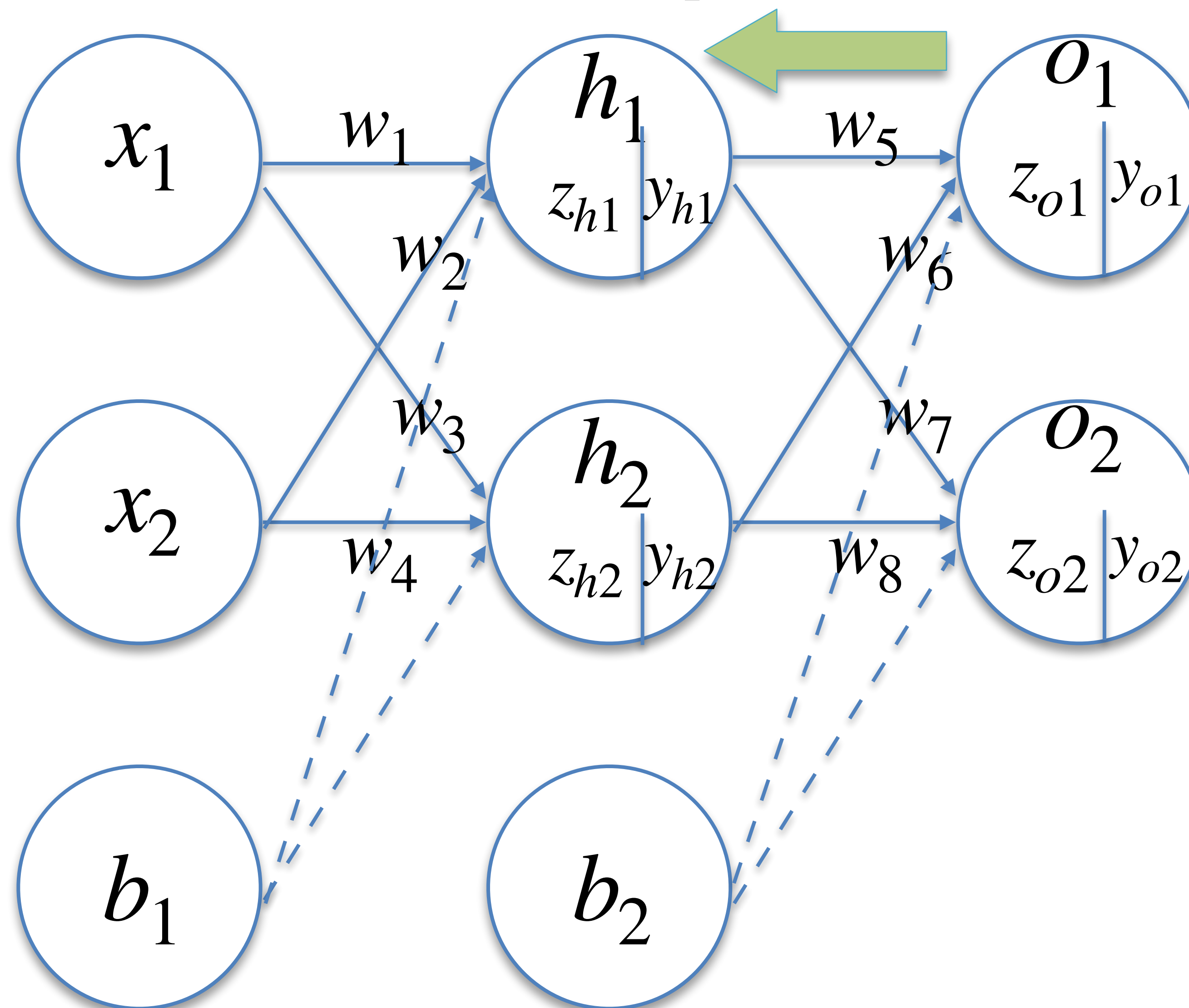


Multilayer Networks

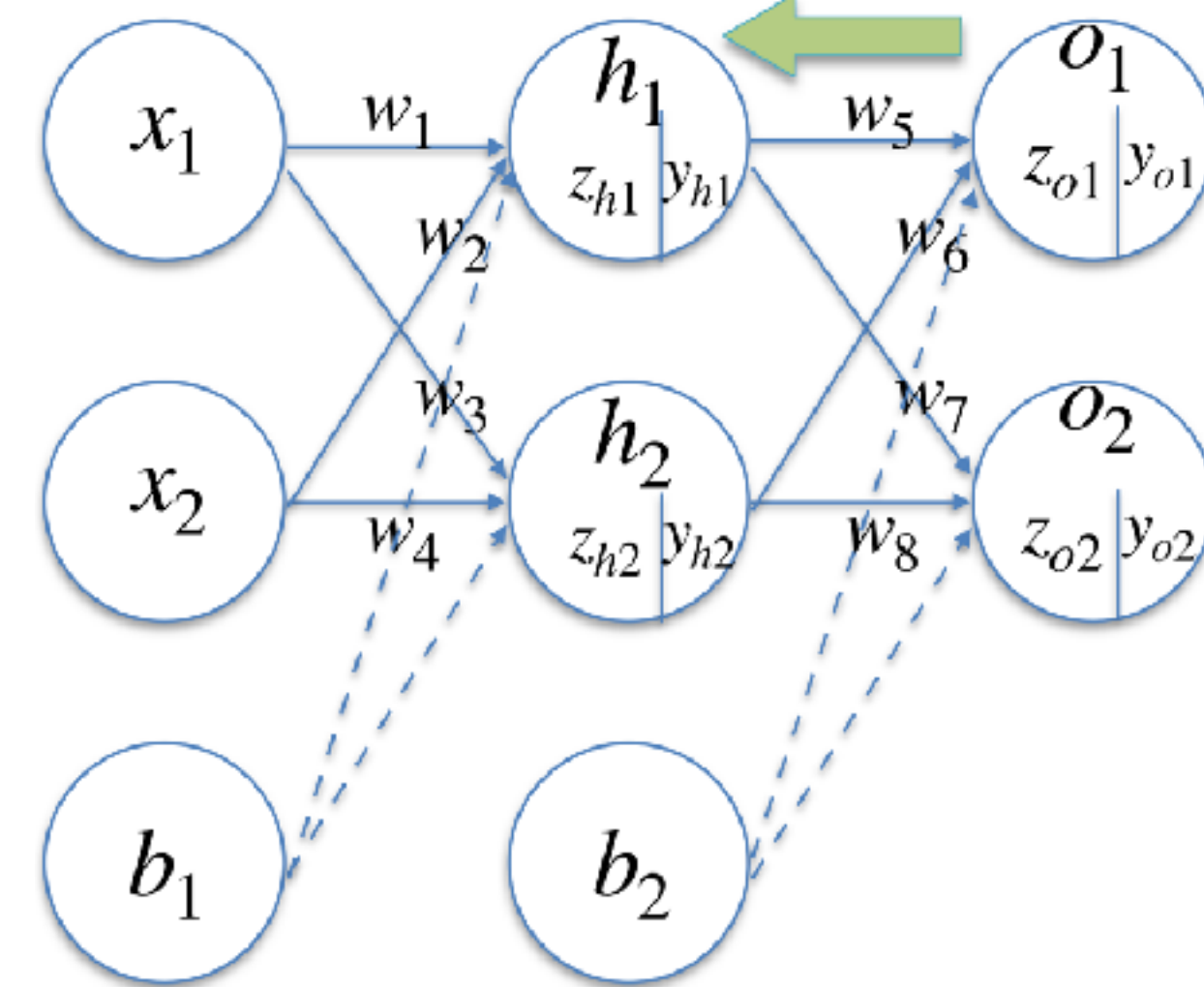


Key Computation: Back-Prop

$$\frac{\partial E}{\partial w_5} = ?$$



Key Computation: Back-Prop

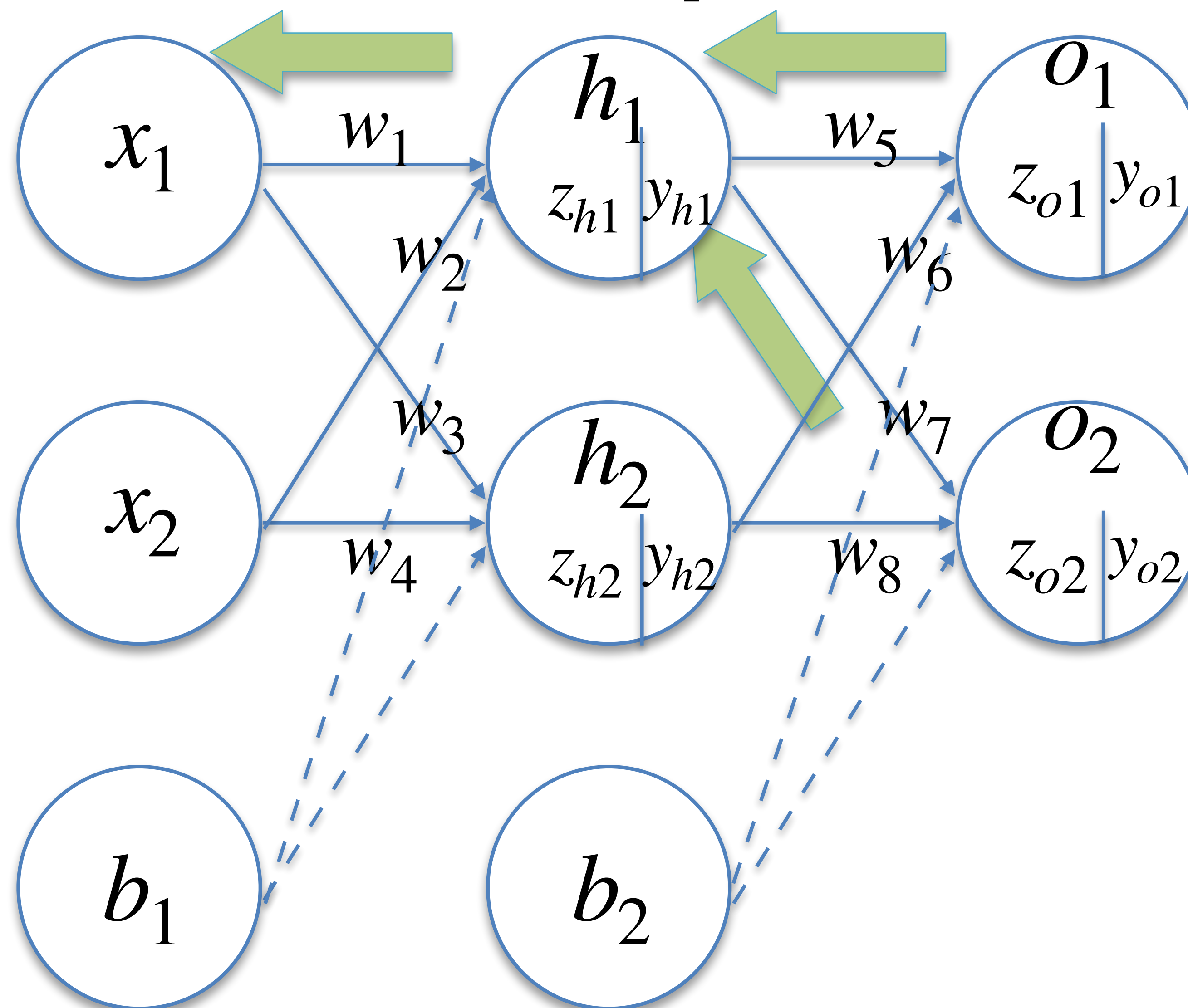


$$\frac{\partial E}{\partial w_5} \leftarrow \frac{\partial z_{o1}}{\partial w_5} \leftarrow \frac{\partial y_{o1}}{\partial z_{o1}} \leftarrow \left(\frac{\partial E_{o1}}{\partial y_{o1}} + \frac{\partial E_{o2}}{\partial y_{o1}} \right) \leftarrow E_T$$

$$\frac{\partial E}{\partial w_5} \leftarrow y_{h1} \leftarrow y_{o1}(1 - y_{o1}) \leftarrow -(t_{o1} - y_{o1}) \leftarrow E_T$$

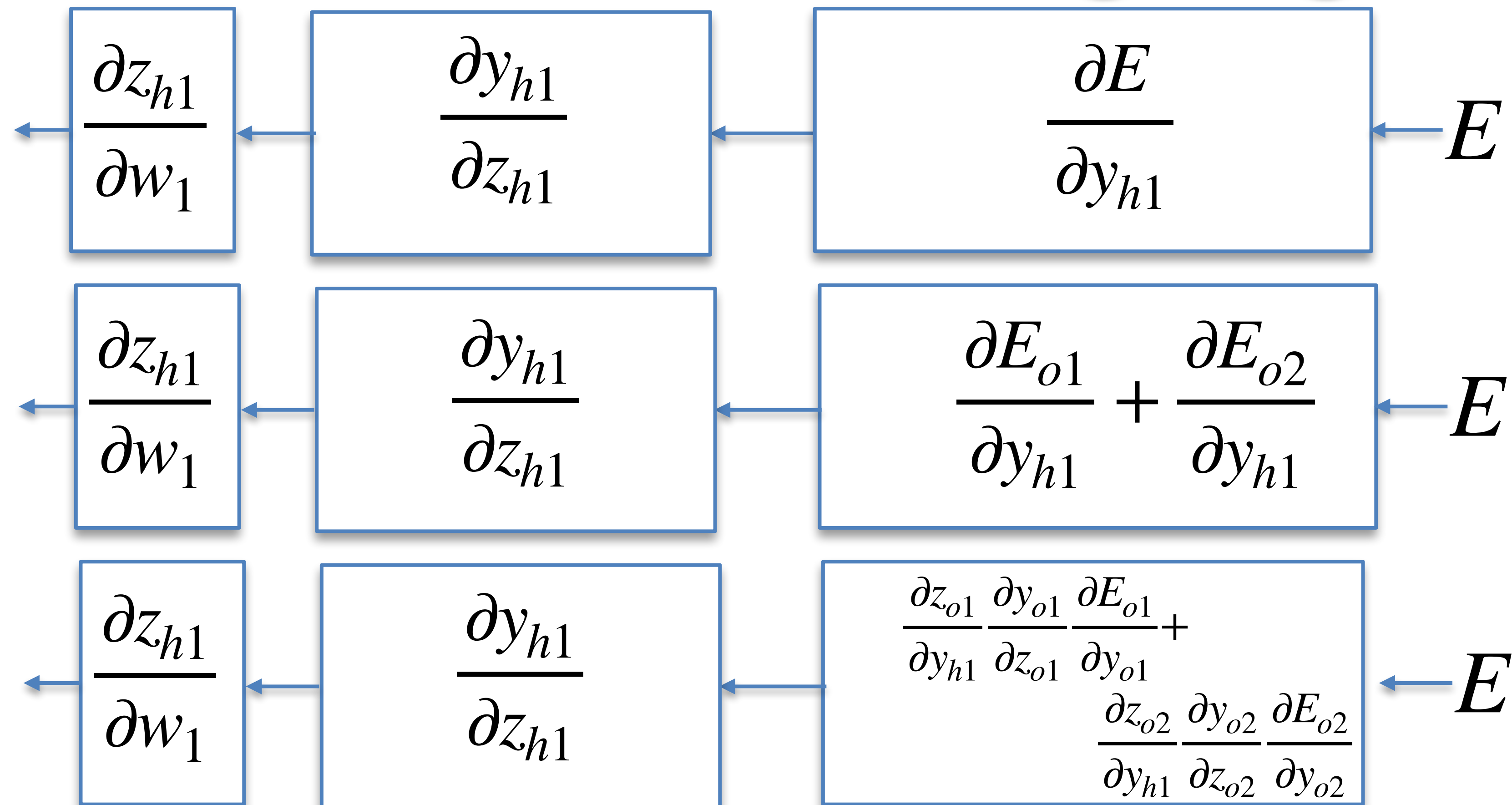
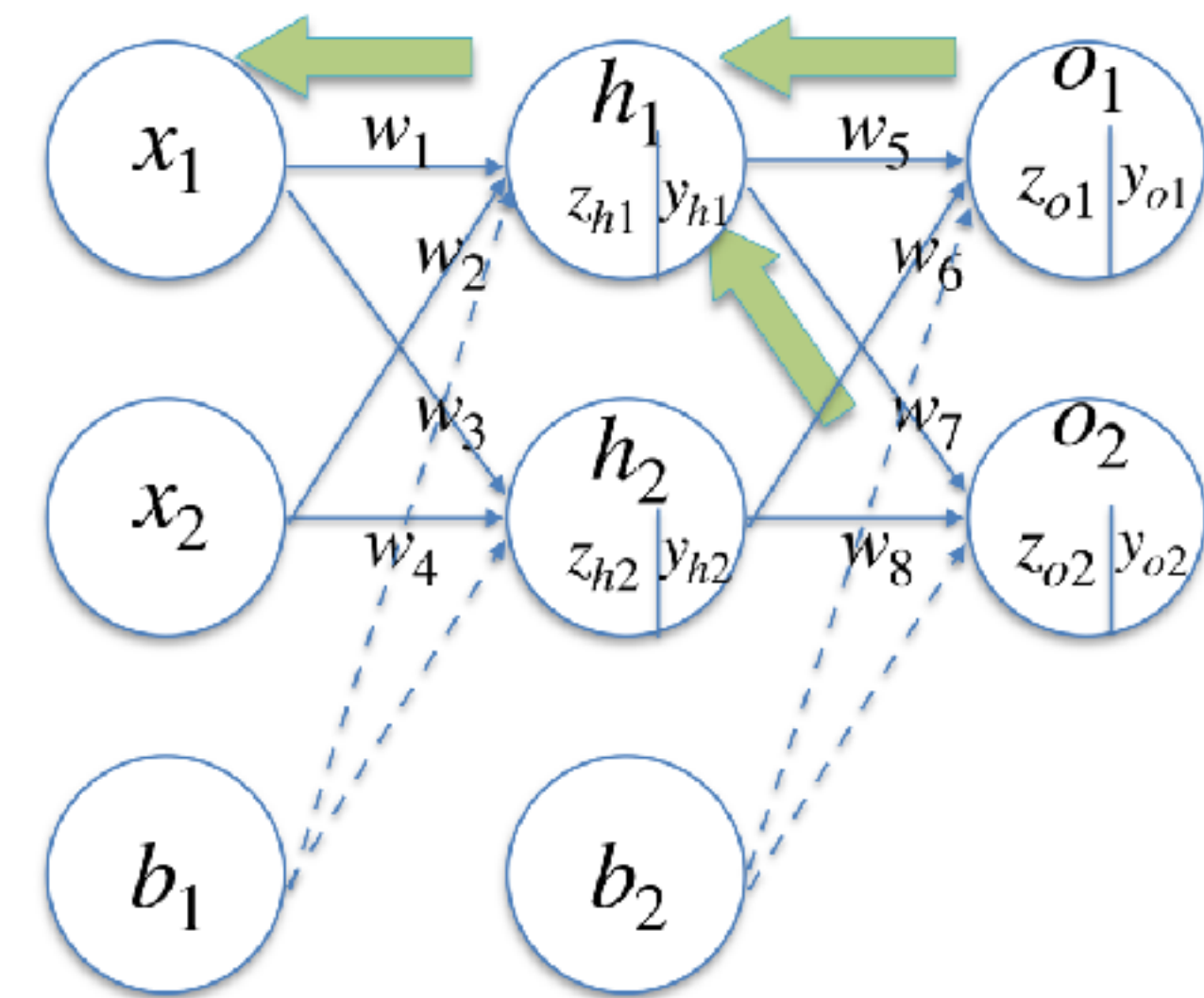
Key Computation: Back-Prop

$$\frac{\partial E}{\partial w_1} = ?$$



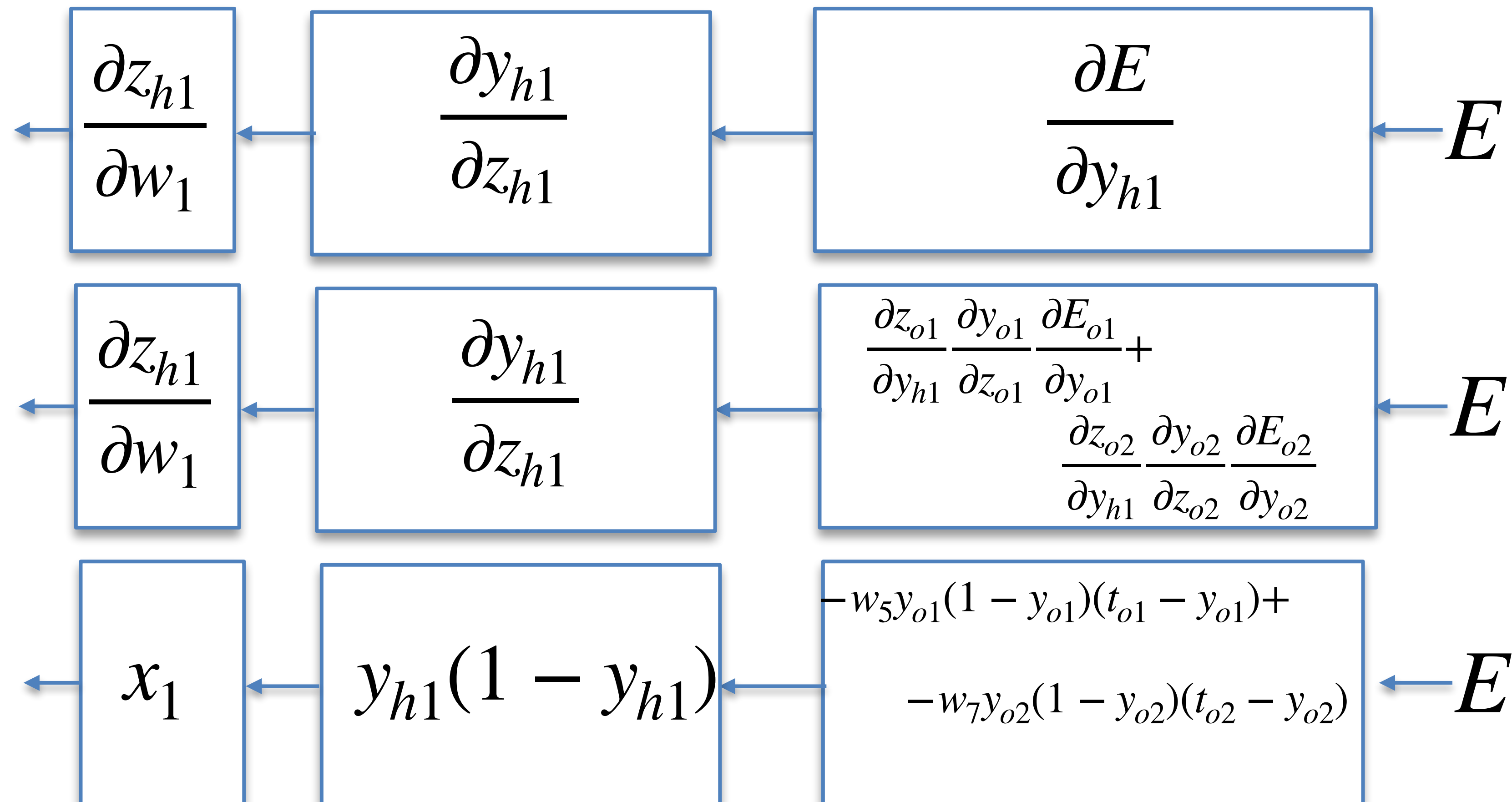
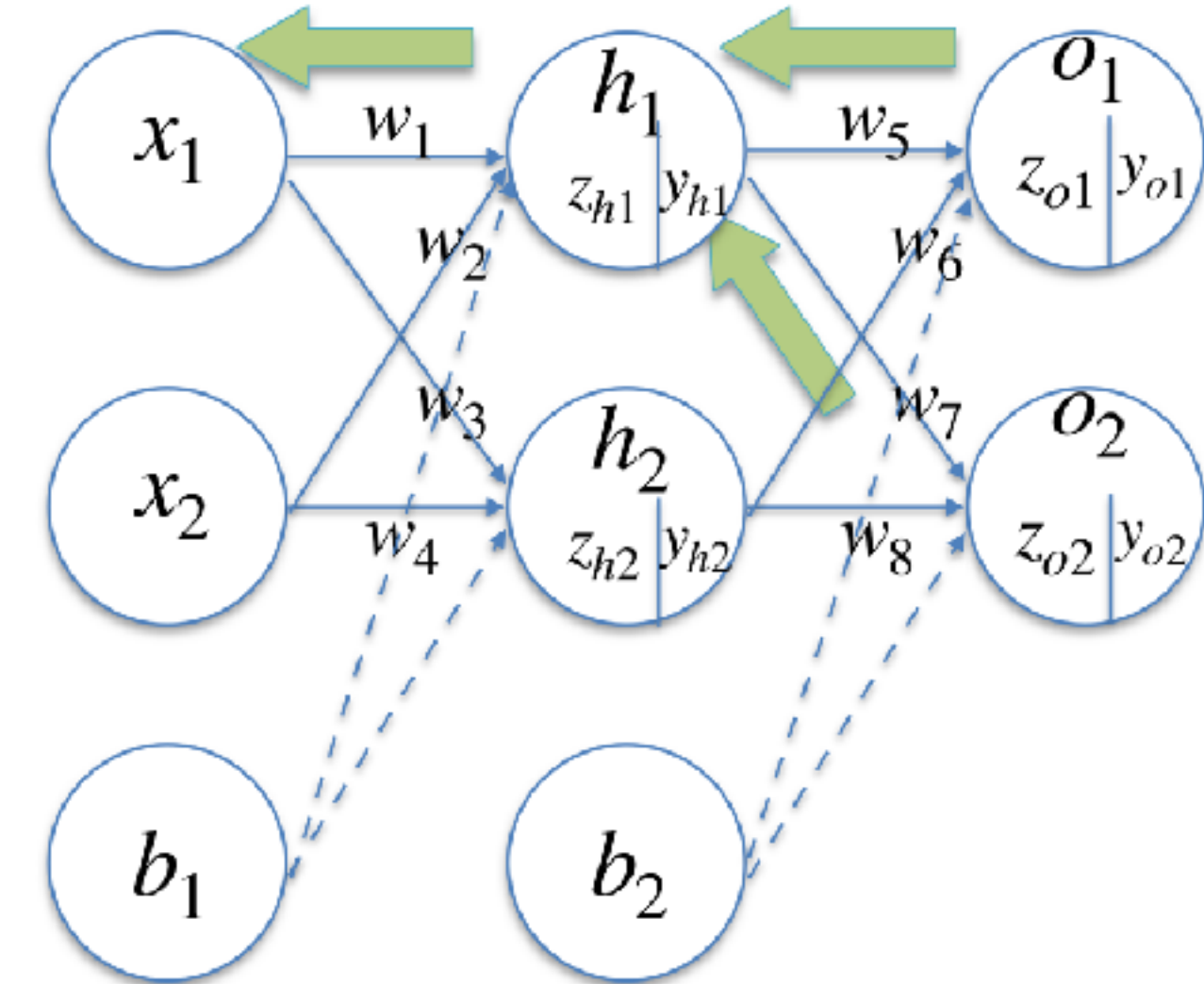
Key Computation: Back-Prop

$$\frac{\partial E}{\partial w_1} = ?$$



Key Computation: Back-Prop

$$\frac{\partial E}{\partial w_1} = ?$$



Gradient Descent

- $\Delta w_j \leftarrow \text{Initialize to ZEROS}$
- For each iteration
 - For each training example $(\mathbf{x}^{(i)}, t^{(i)})$
 - For each weight w_j

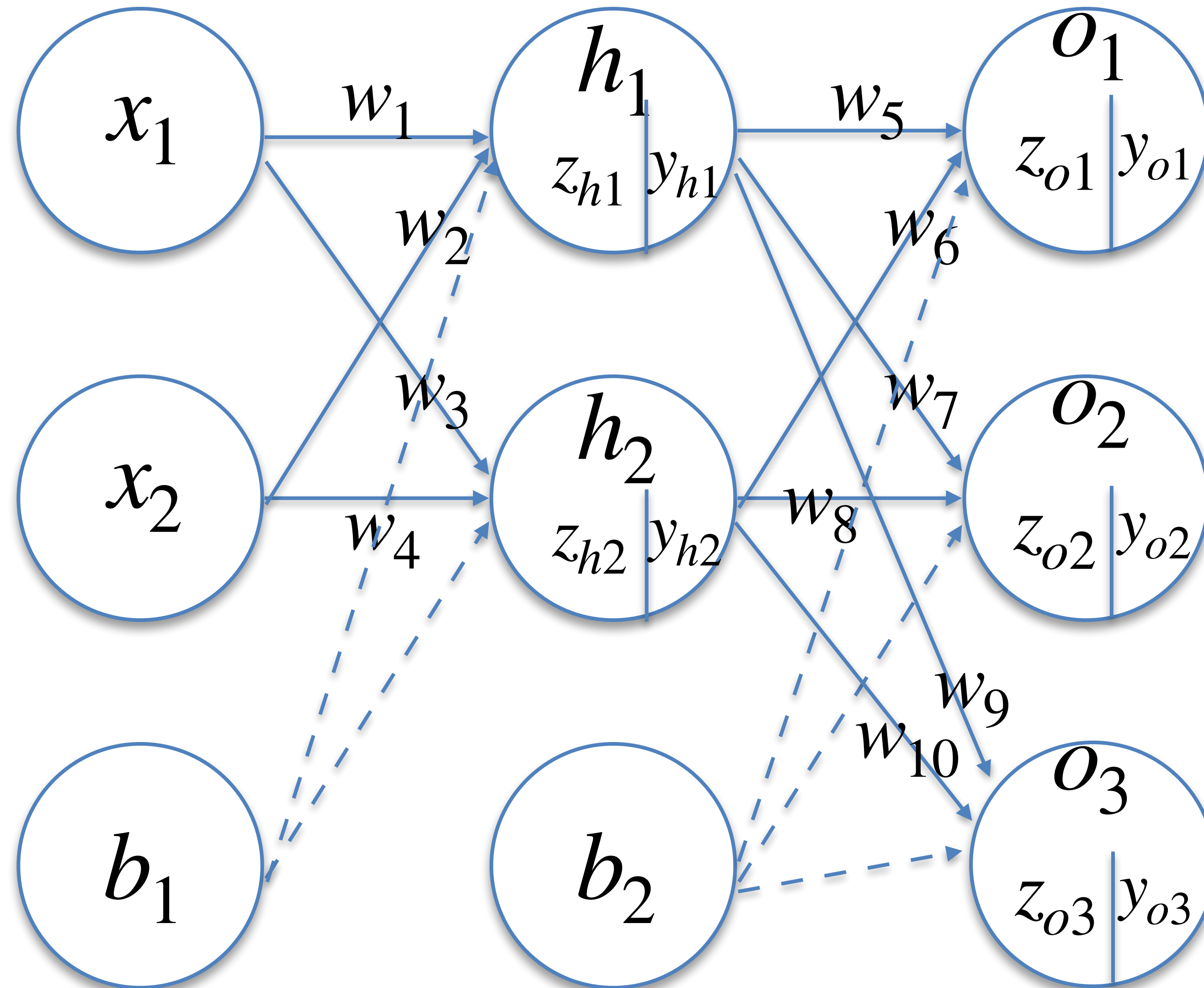
$$\Delta w_j \leftarrow \Delta w_j + \eta x_1 y_{h1} (1 - y_{h1}) [-w_5 y_{o1} (1 - y_{o1}) (t_{o1} - y_{o1}) - w_7 y_{o2} (1 - y_{o2}) (t_{o2} - y_{o2})]$$

- For each weight w_j

$$w_j \leftarrow w_j + \Delta w_j$$

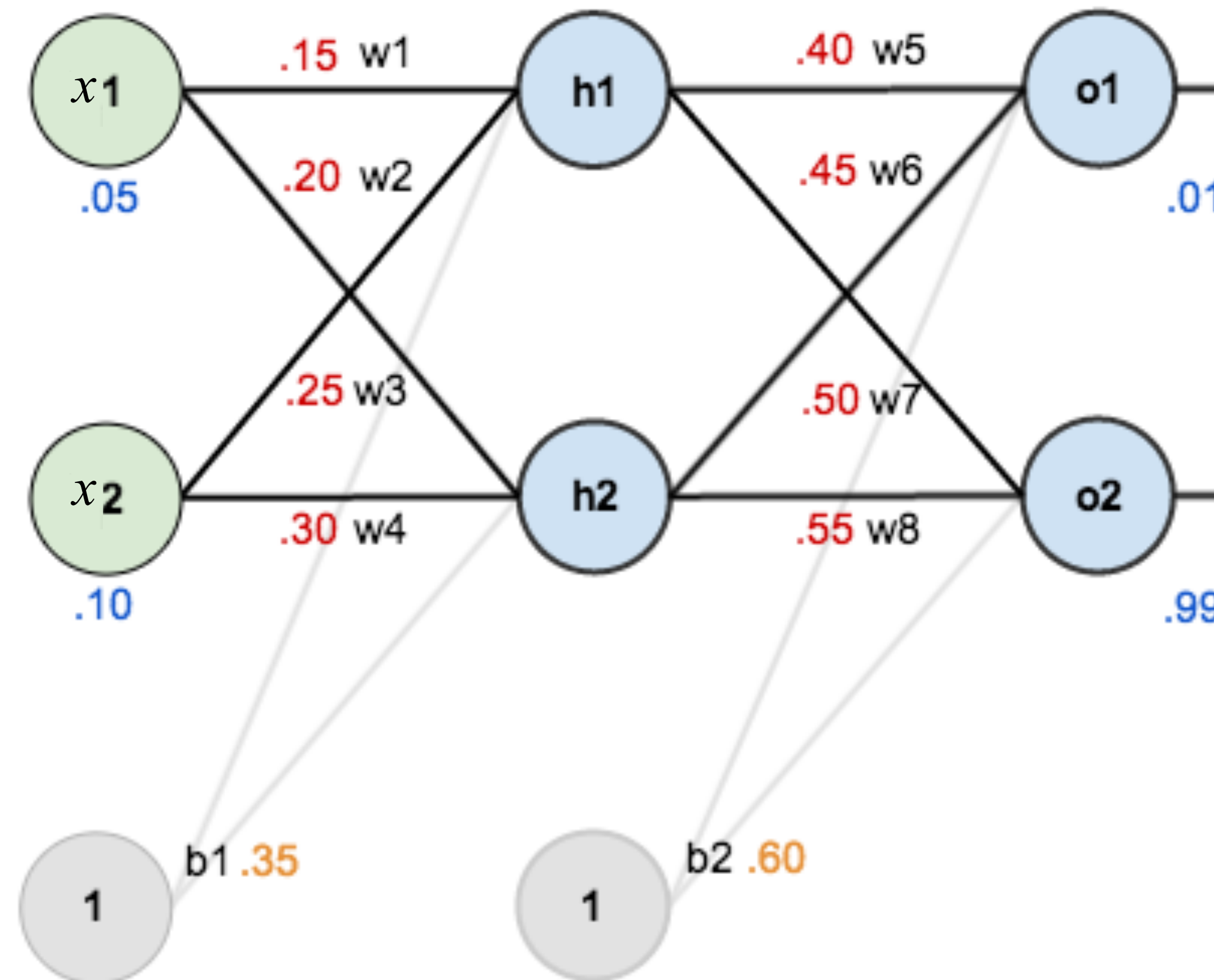
Key Computation: Back-Prop

$$\frac{\partial E}{\partial w_1} = ?$$

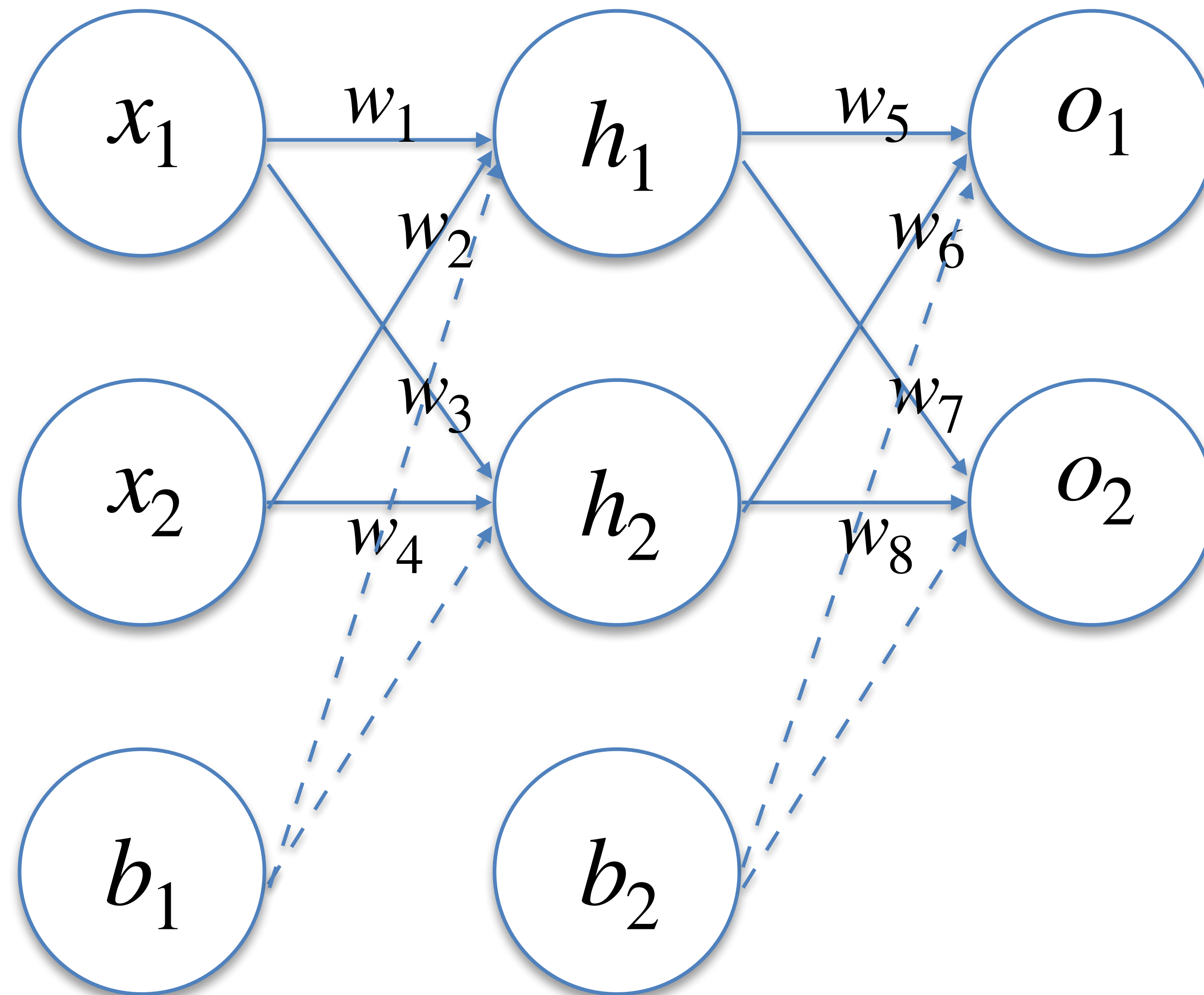


Numerical Example

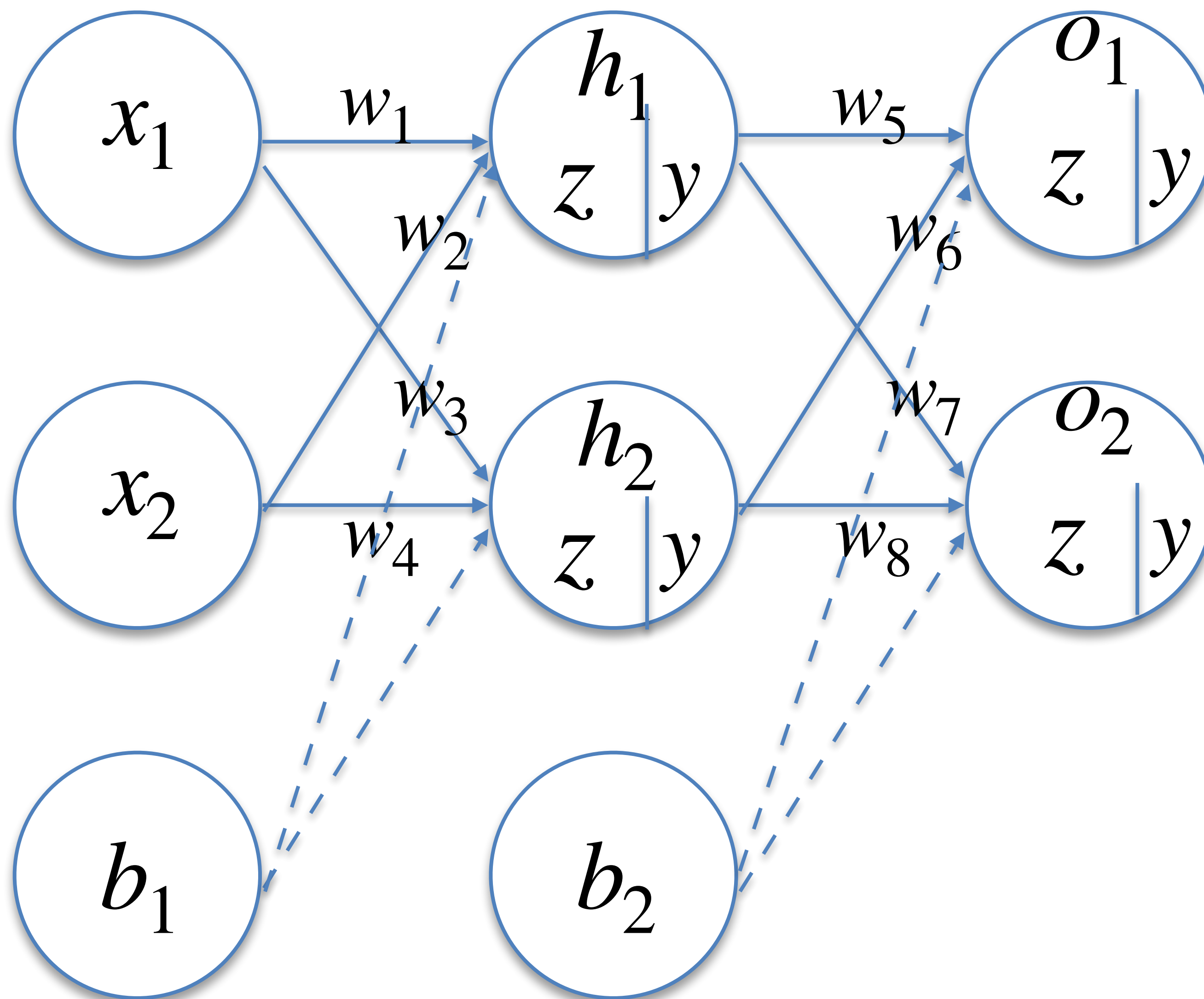
A Step by Step Backpropagation Example



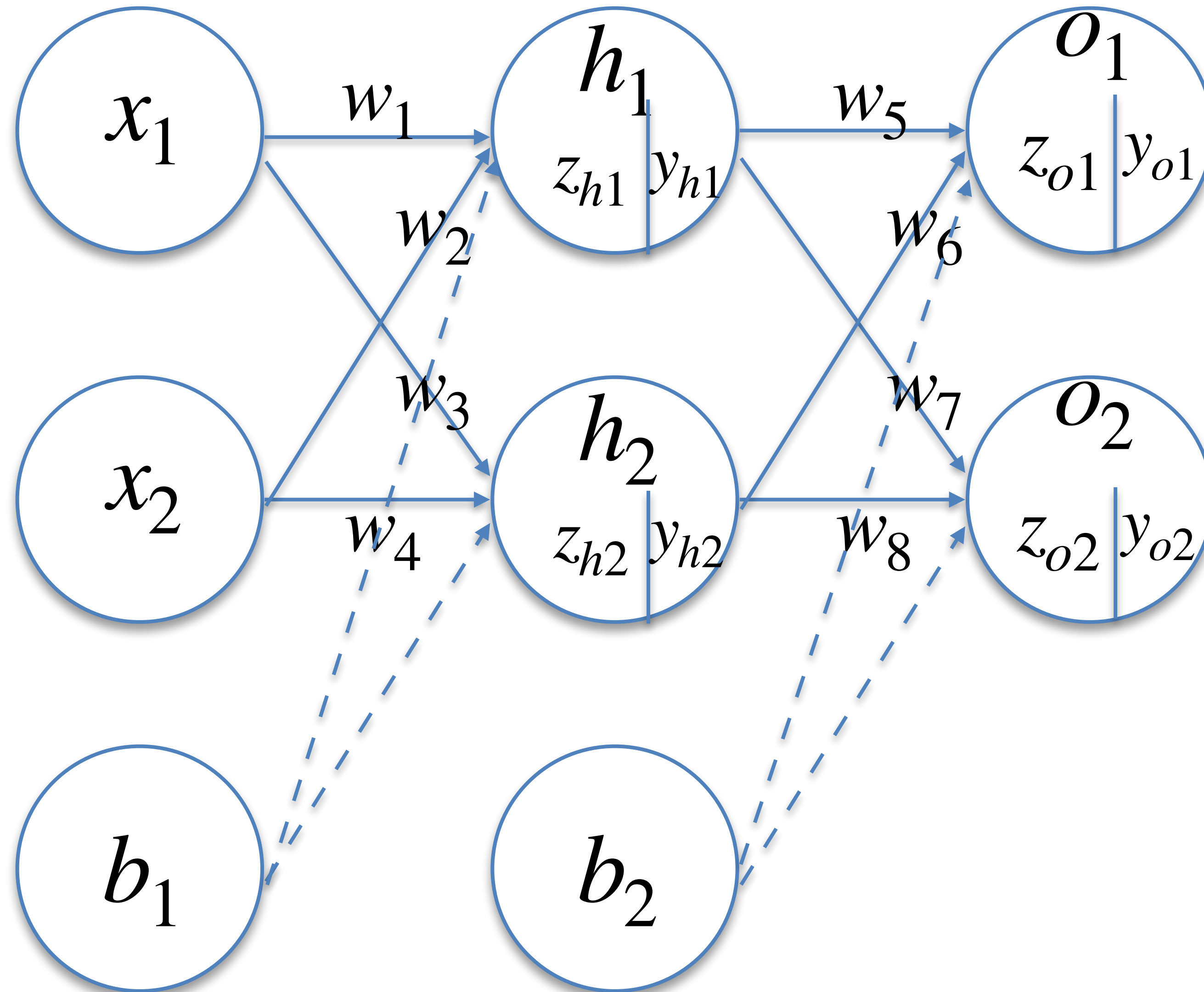
Multilayer Networks



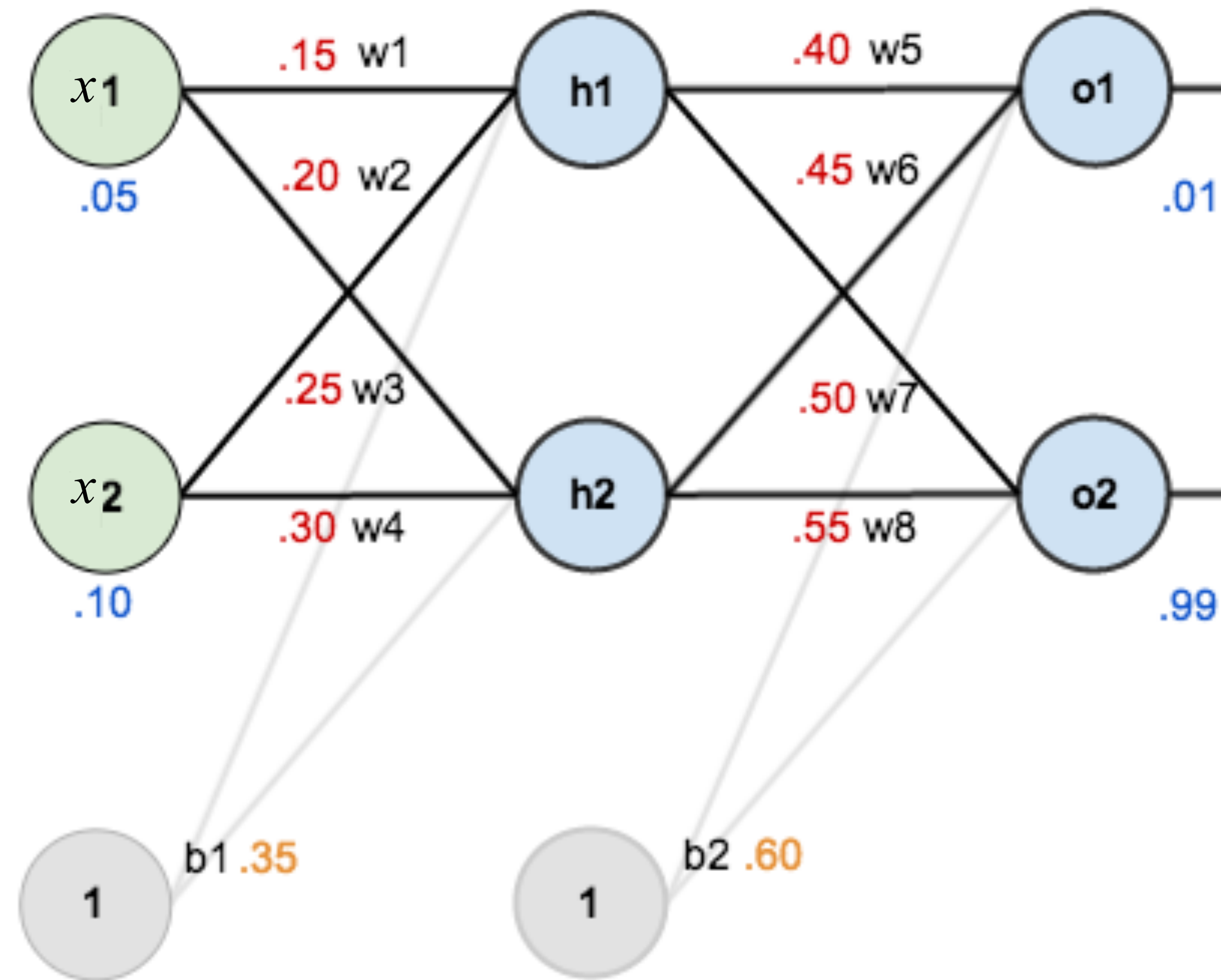
Multilayer Networks



Multilayer Networks



A Step by Step Backpropagation Example



A Step by Step Backpropagation Example

The goal of **backpropagation** is to **optimize** the **weights** so that the neural network can **learn** how to correctly **map** arbitrary inputs to outputs.

For the rest of this tutorial we're going to work with a single training set: given **inputs** 0.05 and 0.10, we want the neural network to **output** 0.01 and 0.99.

A Step by Step Backpropagation Example

The Forward Pass

Total net input for h_1 : $z_{h1} = w_1 * x_1 + w_2 * x_2 + b_1 * 1$

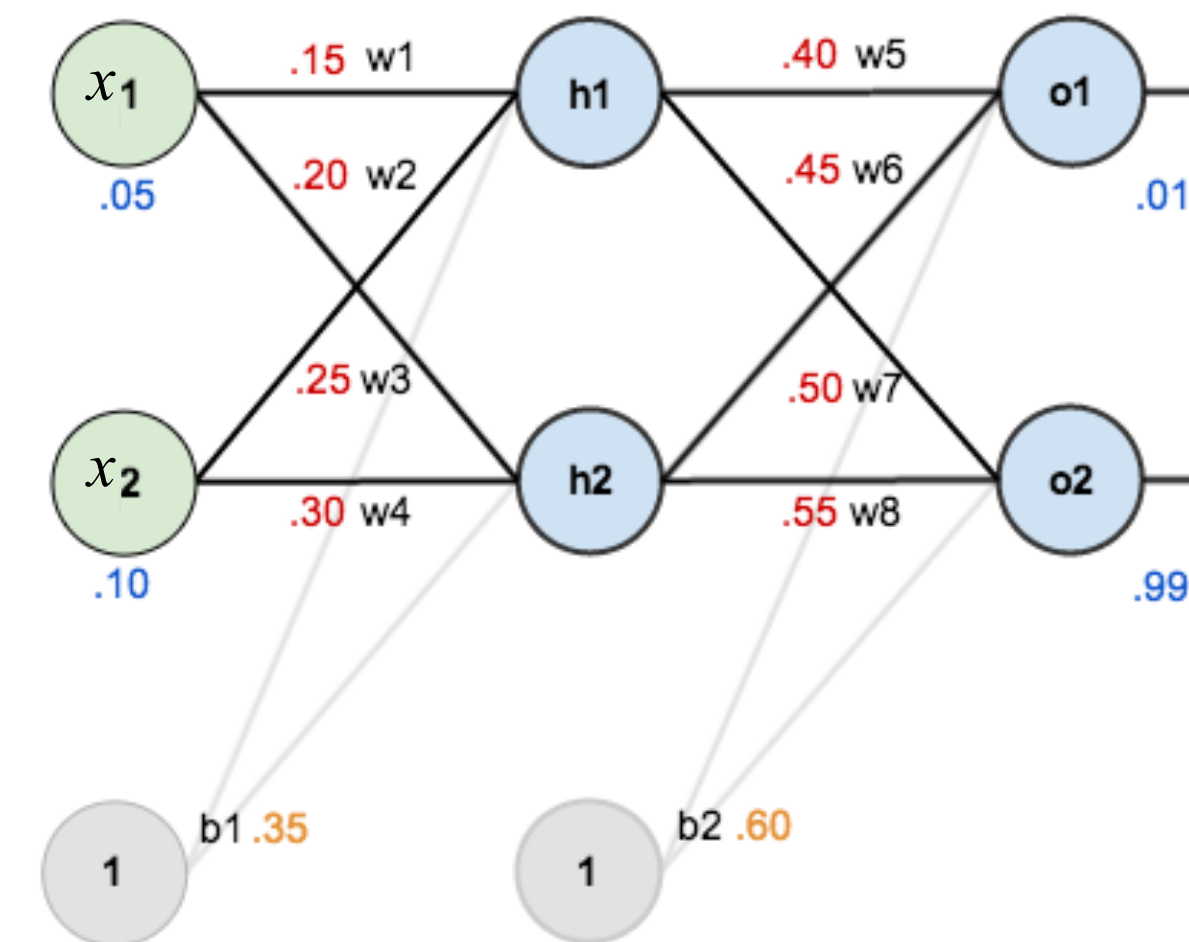
$$z_{h1} = 0.15 * 0.05 + 0.2 * 0.1 + 0.35 * 1 = 0.3775$$

We then squash it using the logistic function to get the output of :

$$y_{h1} = \frac{1}{1 + e^{-z_{h1}}} = \frac{1}{1 + e^{-0.3775}} = 0.593269992$$

Carrying out the same process for h_2 we get:

$$y_{h2} = 0.596884378$$



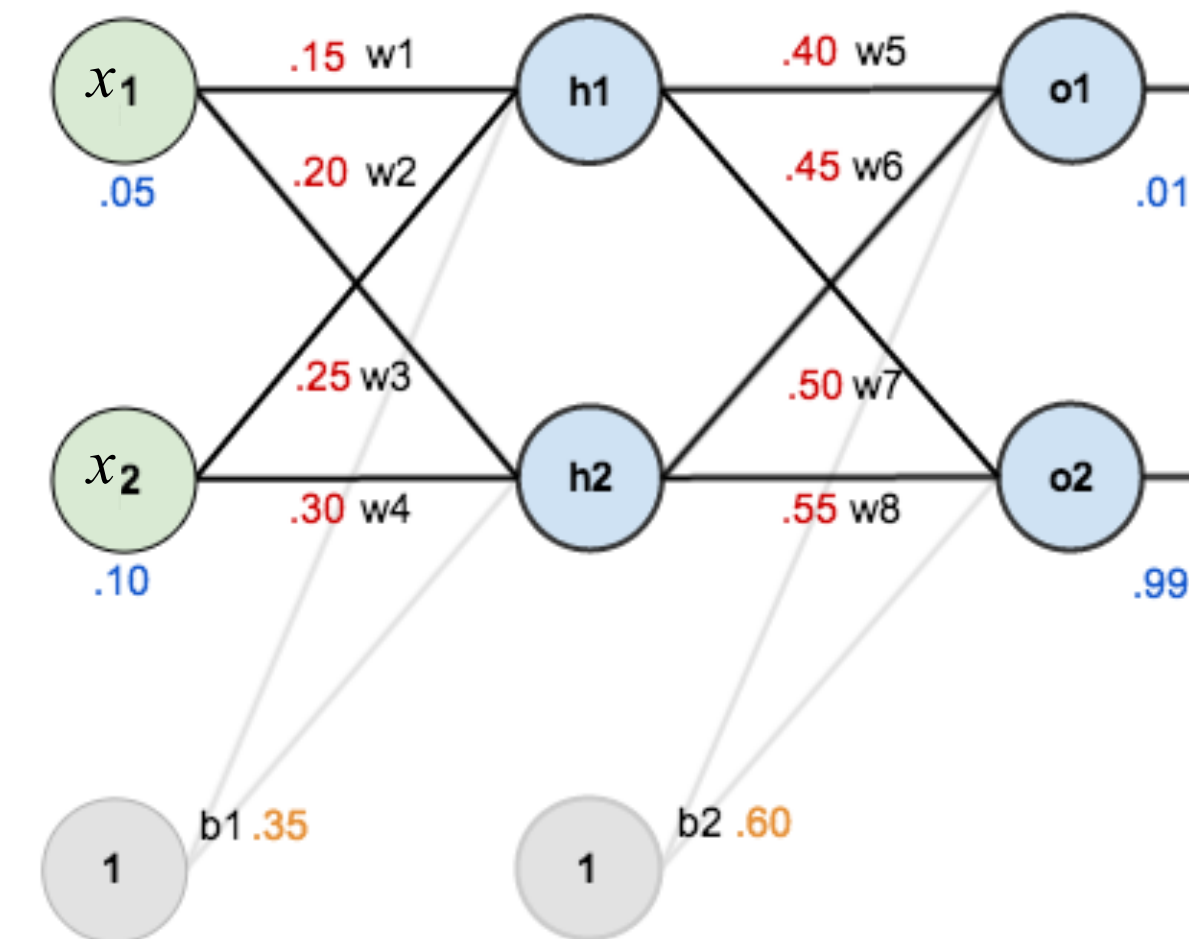
A Step by Step Backpropagation Example

Output for o_1 :

$$z_{o1} = w_5 * y_{h1} + w_6 * y_{h2} + b_2 * 1$$
$$z_{o1} = 0.4 * 0.593269992 + 0.45 * 0.596884378 + 0.6 * 1$$
$$= 1.105905967$$

$$y_{o1} = \frac{1}{1 + e^{-z_{o1}}} = \frac{1}{1 + e^{-1.105905967}} = 0.75136507$$

For o_2 we get: $y_{o2} = 0.772928465$



A Step by Step Backpropagation Example

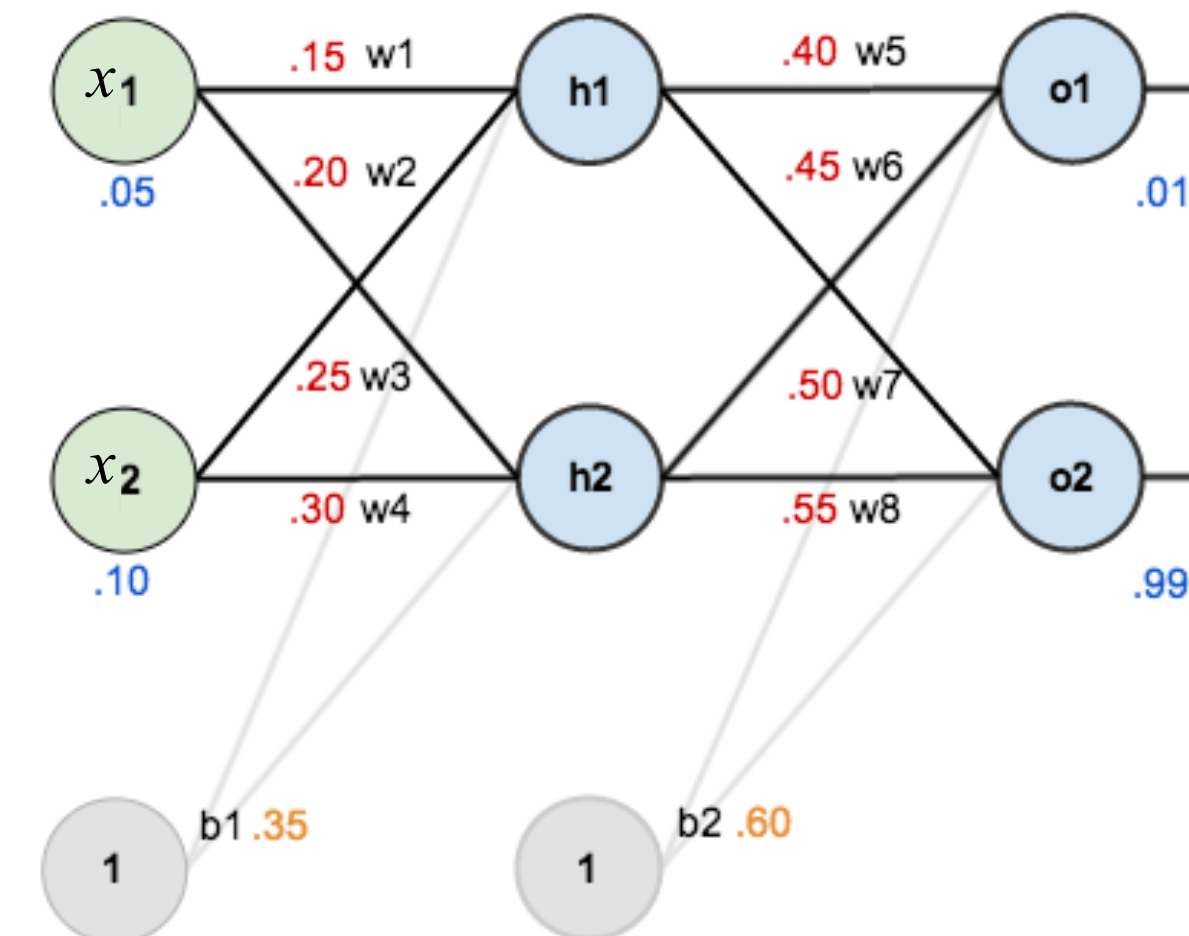
Calculating the Error

$$E_T = \sum \frac{1}{2}(\text{target} - \text{output})^2$$

$$E_{o2} = 0.023560026$$

$$E_{o1} = \frac{1}{2}(t_{o1} - y_{o1})^2 = \frac{1}{2}(0.01 - 0.75136507)^2 = 0.274811083$$

$$E_T = E_{o1} + E_{o2} = 0.274811083 + 0.023560026 = 0.298371109$$



A Step by Step Backpropagation Example

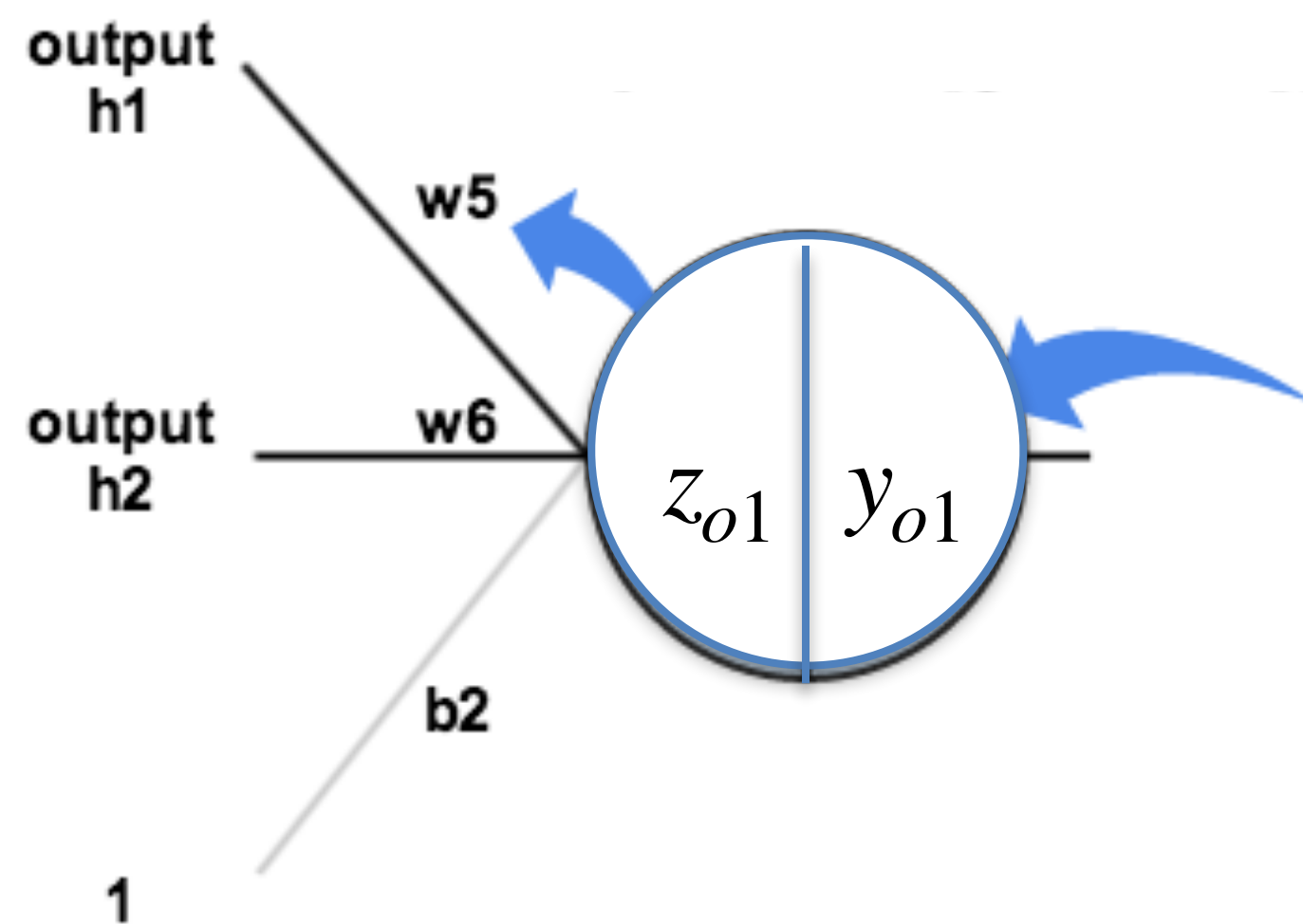
The Backward Pass

Output Layer

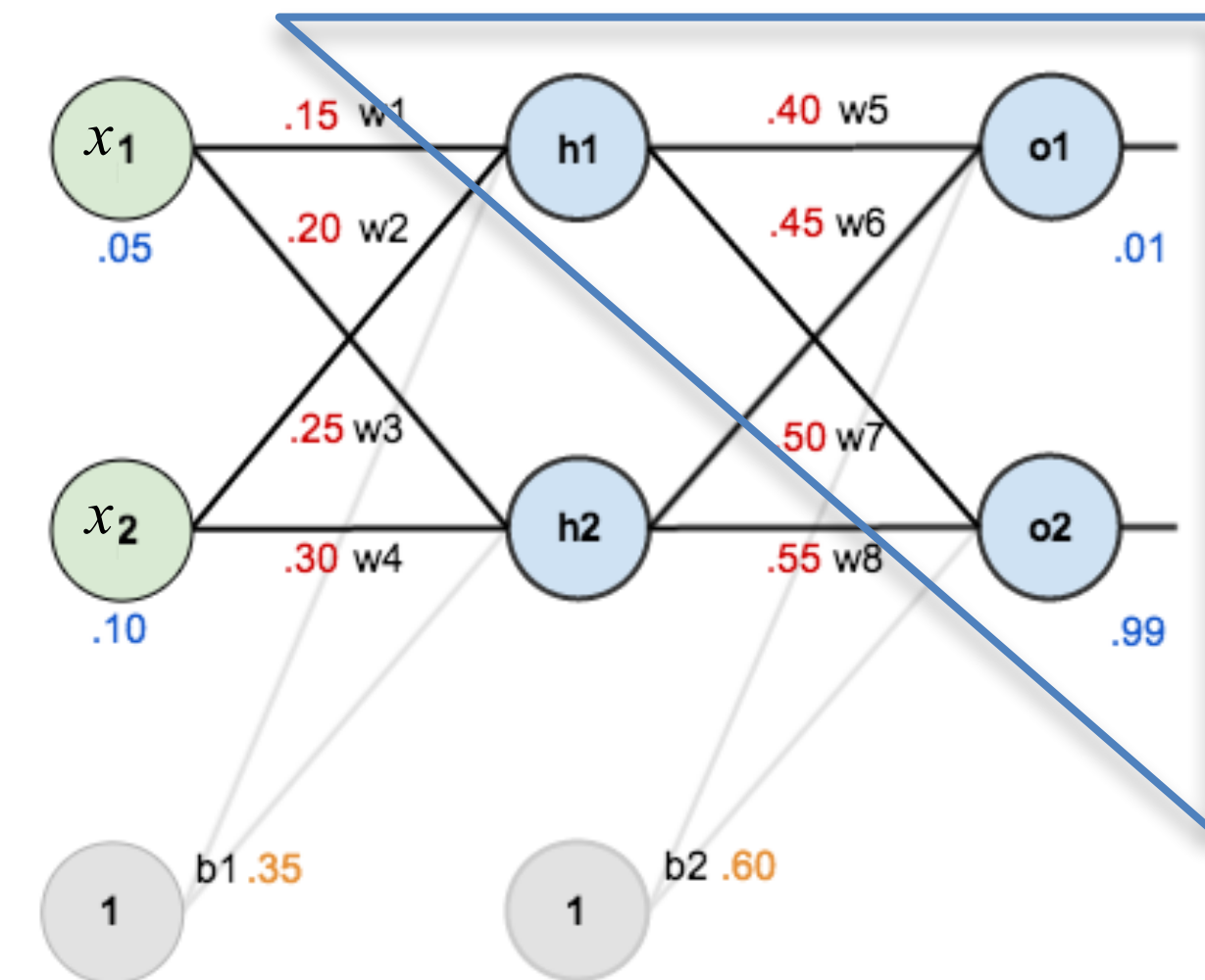
Consider w_5 . We want to know how much a change in affects the total error, aka $\frac{\partial E_T}{\partial w_5}$.

By applying the chain rule we know that:
$$\frac{\partial E_T}{\partial w_5} = \frac{\partial z_{o1}}{\partial w_5} \frac{\partial y_{o1}}{\partial z_{o1}} \frac{\partial E_T}{\partial y_{o1}}$$

Visually, here's what we're doing:



$$E_{o1} = \frac{1}{2}(t_{o1} - y_{o1})^2$$
$$E_T = E_{o1} + E_{o2}$$



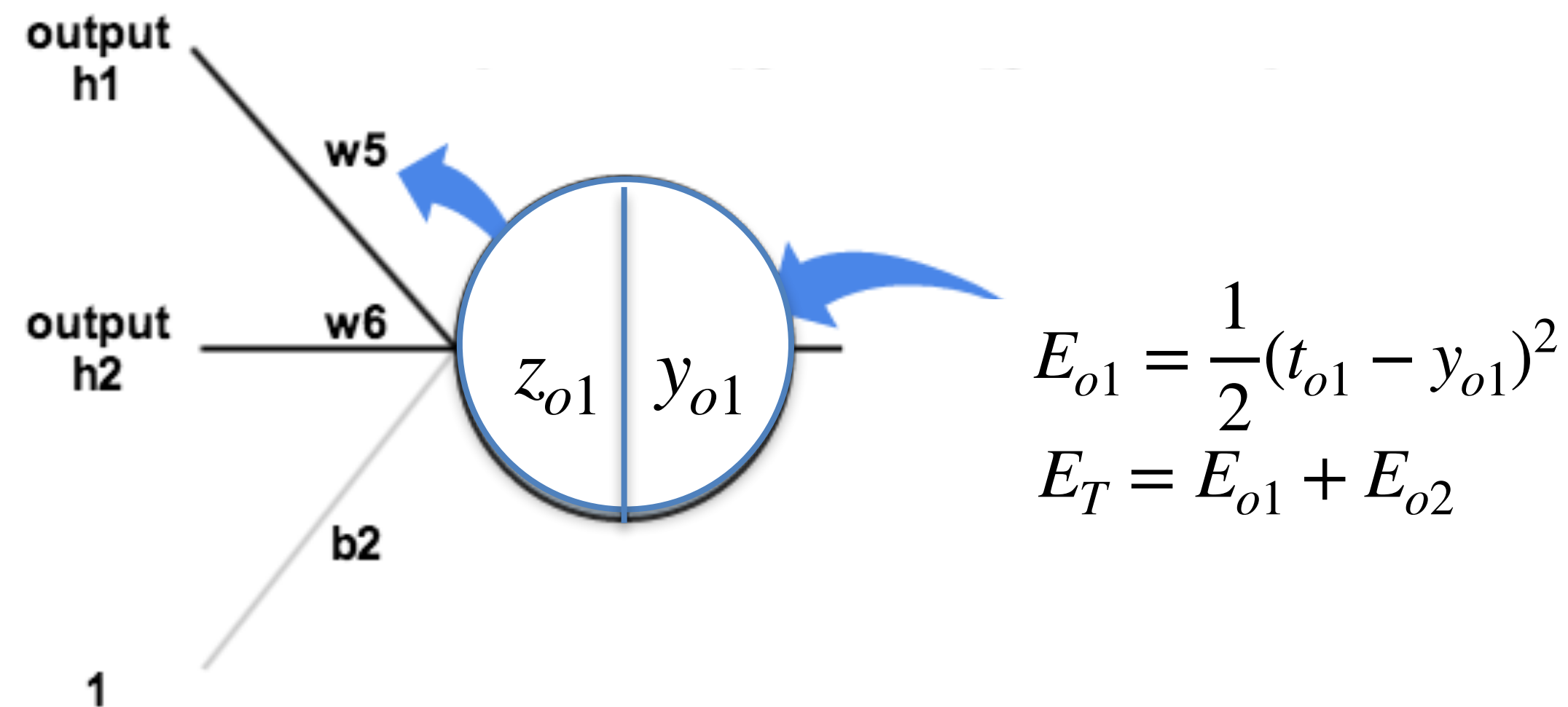
A Step by Step Backpropagation Example

The Backward Pass

$$\frac{\partial E_T}{\partial w_5} = \frac{\partial z_{o1}}{\partial w_5} \frac{\partial y_{o1}}{\partial z_{o1}} \frac{\partial E_T}{\partial y_{o1}}$$

how much does the total error change w.r.t the output?

$$E_T = \frac{1}{2}(t_{o1} - y_{o1})^2 + \frac{1}{2}(t_{o2} - y_{o2})^2$$
$$\frac{\partial E_T}{\partial y_{o1}} = 2 * \frac{1}{2}(t_{o1} - y_{o1})^{2-1} * (-1) + (-1) * 0$$
$$= -(0.01 - 0.75136507) = 0.74136507$$



A Step by Step Backpropagation Example

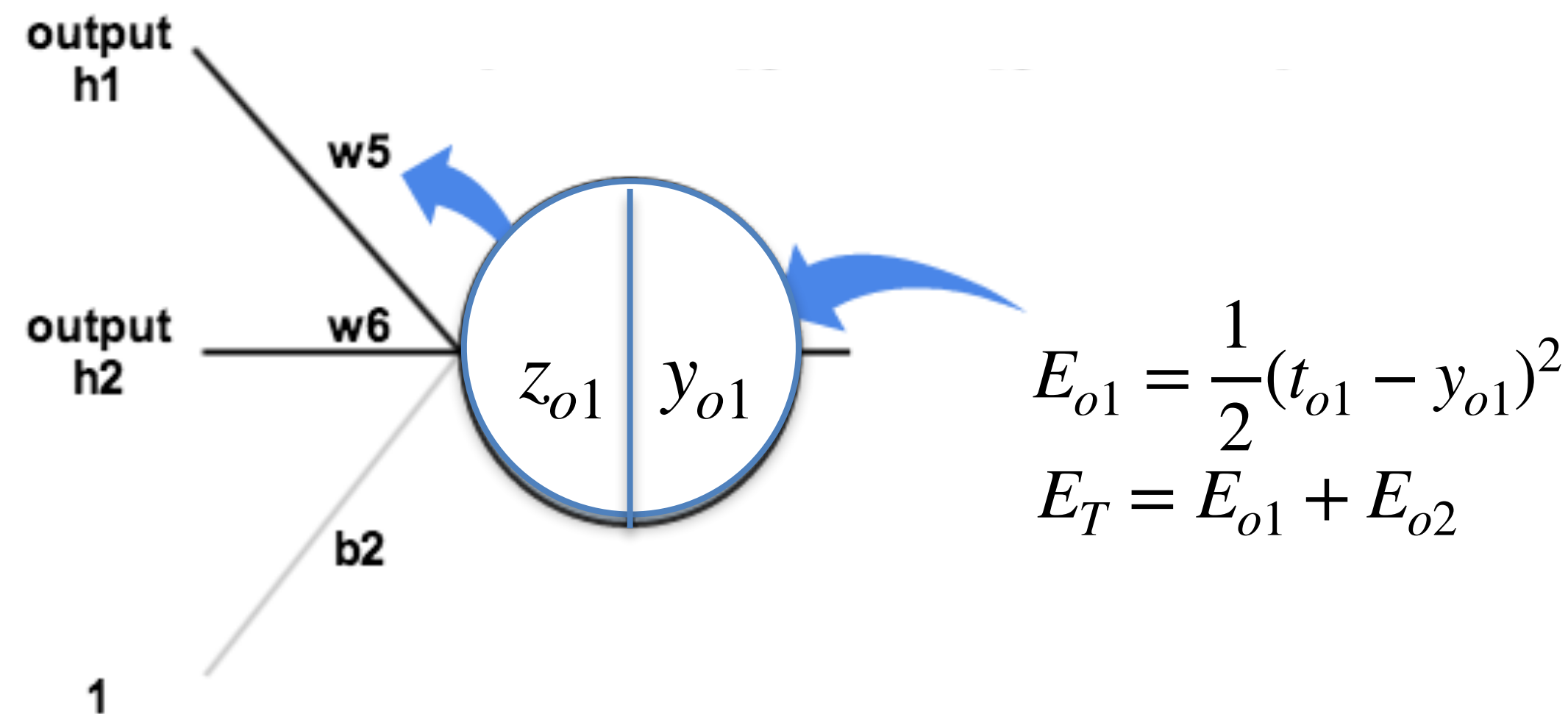
The Backward Pass

$$\frac{\partial E_T}{\partial w_5} = \frac{\partial z_{o1}}{\partial w_5} \frac{\partial y_{o1}}{\partial z_{o1}} \frac{\partial E_T}{\partial y_{o1}}$$

how much does the output o_1 change w.r.t its total net input?

$$y_{o1} = \frac{1}{1 + e^{-z_{o1}}}$$

$$\frac{\partial y_{o1}}{\partial z_{o1}} = y_{o1}(1 - y_{o1}) = 0.75136507(1 - 0.75136507) = 0.186815602$$



A Step by Step Backpropagation Example

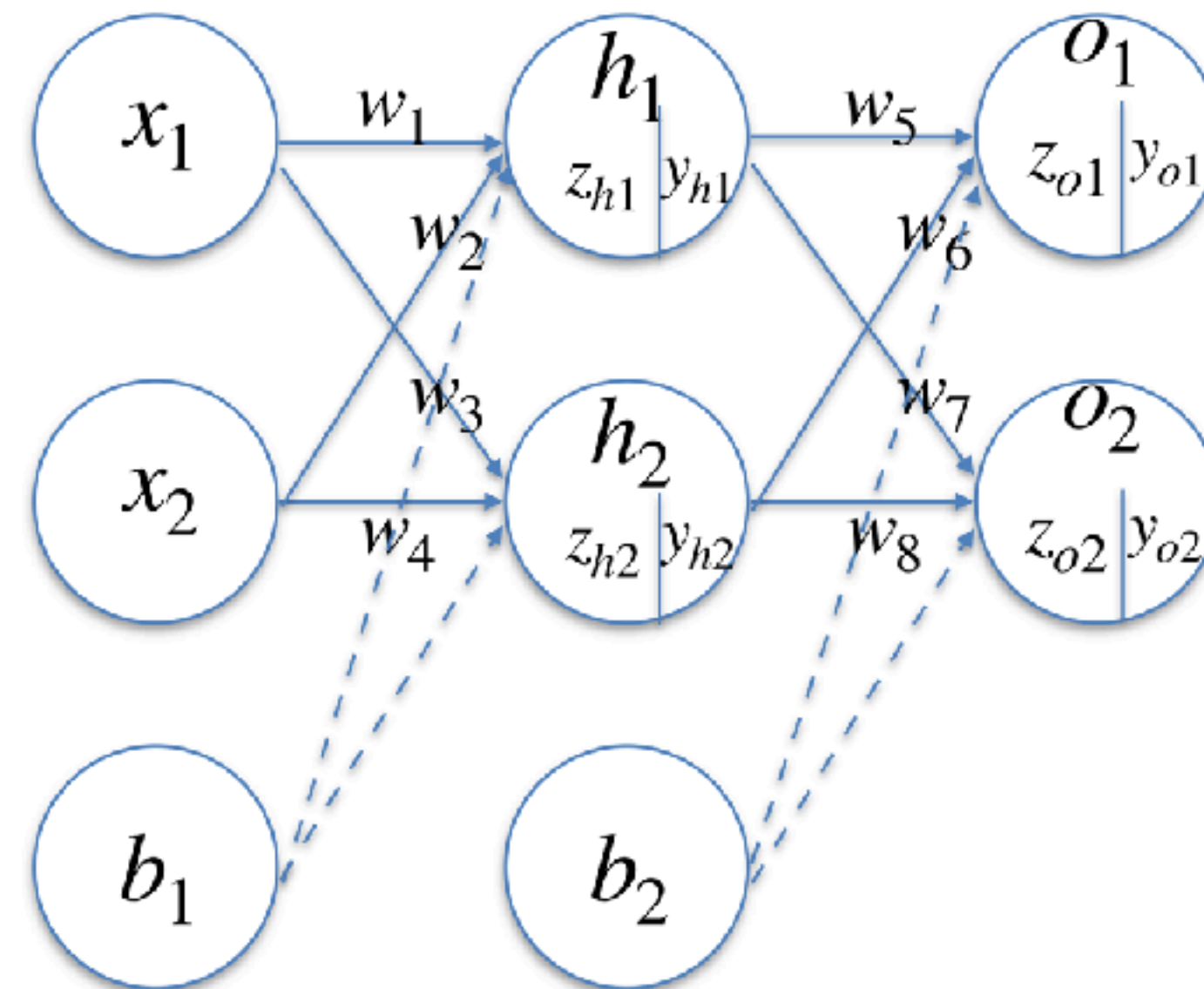
The Backward Pass

$$\frac{\partial E_T}{\partial w_5} = \frac{\partial z_{o1}}{\partial w_5} \frac{\partial y_{o1}}{\partial z_{o1}} \frac{\partial E_T}{\partial y_{o1}}$$

how much does the total net input of o_1 change with respect to w_5 ?

$$z_{o1} = w_5 * y_{h1} + w_6 * y_{h2} + b_2 * 1$$

$$\frac{\partial z_{o1}}{\partial w_5} = 1 * y_{h1} = 0.593269992$$



A Step by Step Backpropagation Example

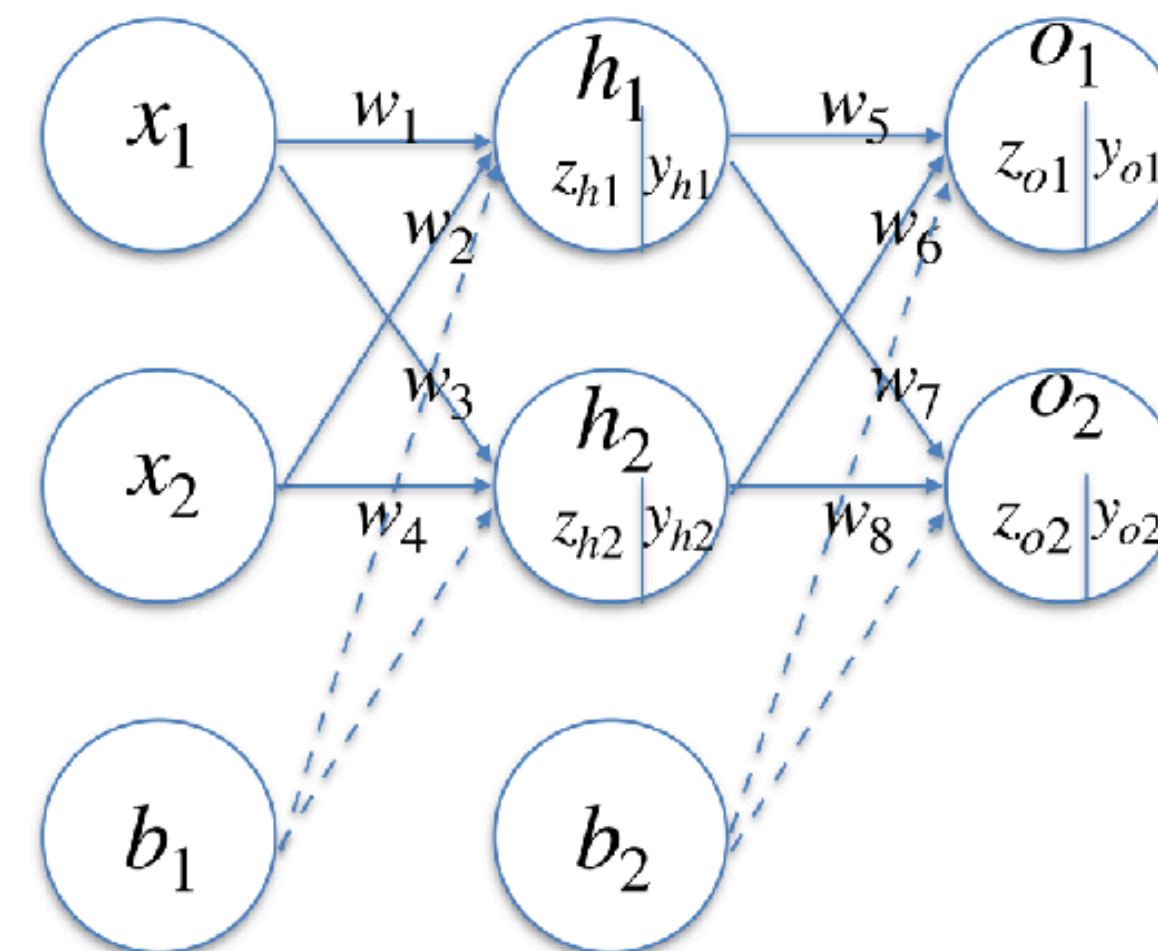
The Backward Pass

Putting it all together: $\frac{\partial E_T}{\partial w_5} = \frac{\partial z_{o1}}{\partial w_5} \frac{\partial y_{o1}}{\partial z_{o1}} \frac{\partial E_T}{\partial y_{o1}}$

$$\frac{\partial E_T}{\partial w_5} = -(t_{o1} - y_{o1}) y_{o1} * (1 - y_{o1}) * y_{h1}$$

$$= 0.74136507 * 0.186815602 * 0.593269992 = 0.082167041$$

$$w_5^+ = w_5 - \eta \frac{\partial E_T}{\partial w_5} = 0.4 - 0.5 * 0.082167041 = 0.35891648$$



A Step by Step Backpropagation Example

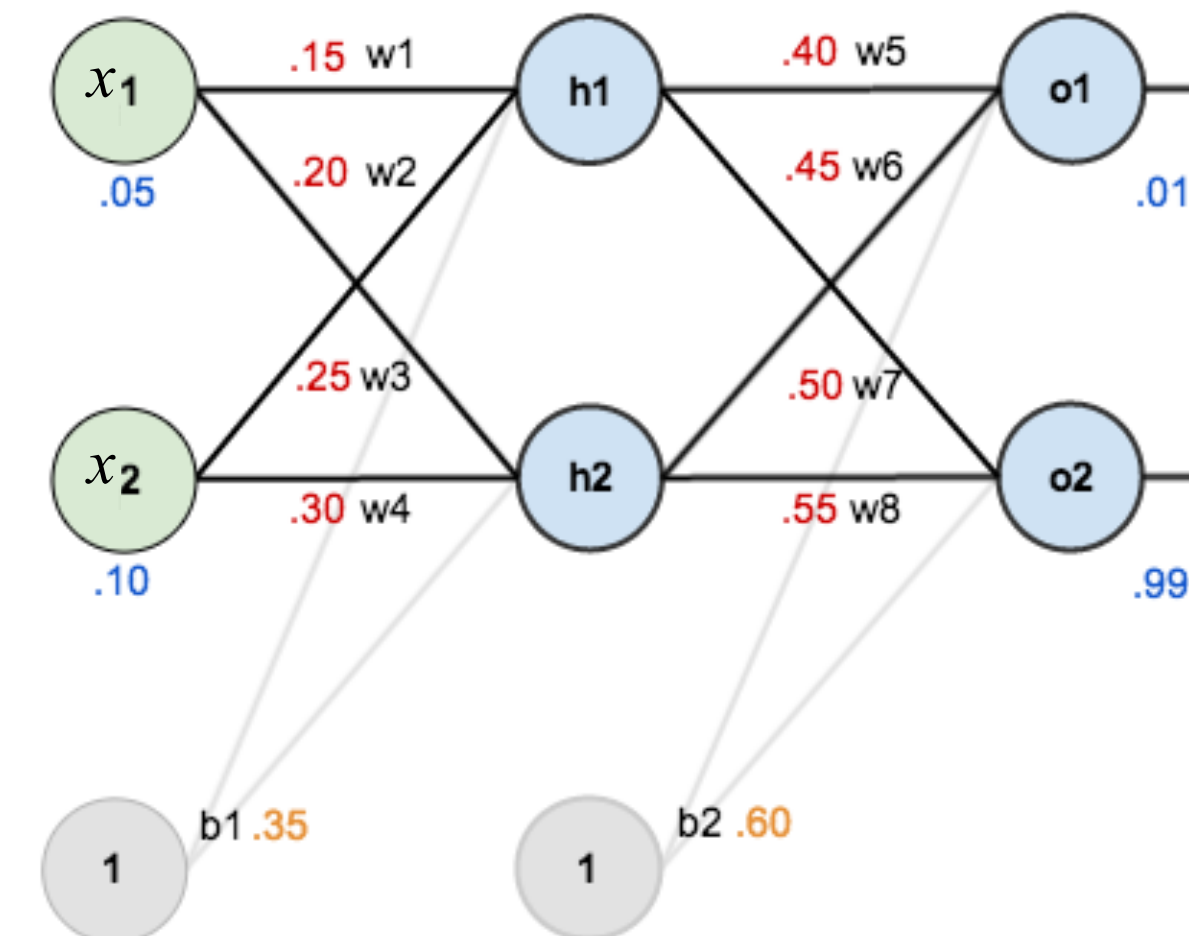
The Backward Pass

We can repeat this process to get the new weights w_6 , w_7 , and w_8

$$w_6^+ = 0.408666186$$

$$w_7^+ = 0.511301270$$

$$w_8^+ = 0.561370121$$



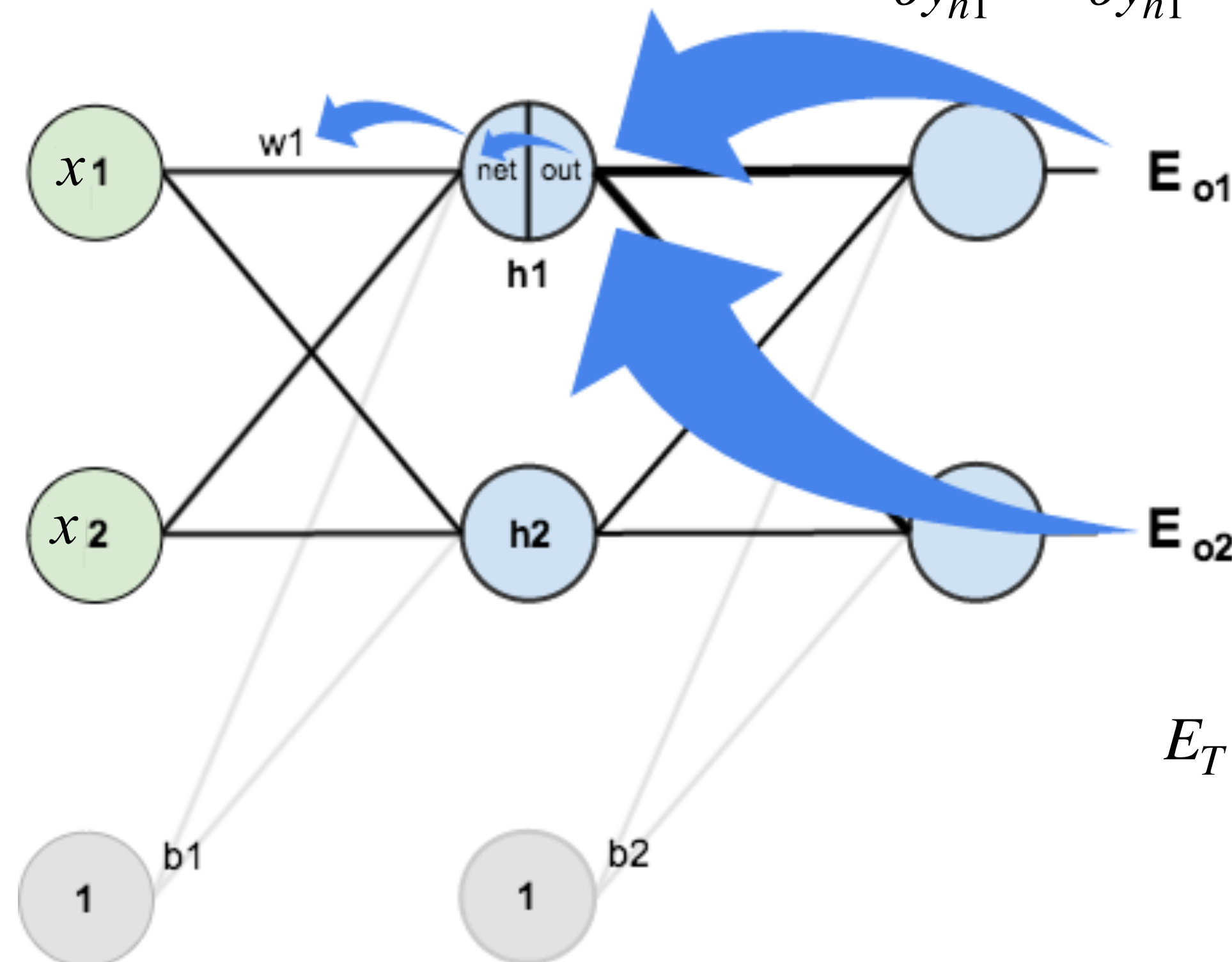
A Step by Step Backpropagation Example

The Backward Pass - Hidden Layer

Continue the backwards pass by calculating new values for w_1 , w_2 , w_3 , and w_4

$$\frac{\partial E_T}{\partial w_1} = \frac{\partial z_{h1}}{\partial w_1} \frac{\partial y_{h1}}{\partial z_{h1}} \frac{\partial E_T}{\partial y_{h1}}$$

$$\frac{\partial E_T}{\partial y_{h1}} = \frac{\partial E_{o1}}{\partial y_{h1}} + \frac{\partial E_{o2}}{\partial y_{h1}}$$



$$E_T = E_{o1} + E_{o2}$$

A Step by Step Backpropagation Example

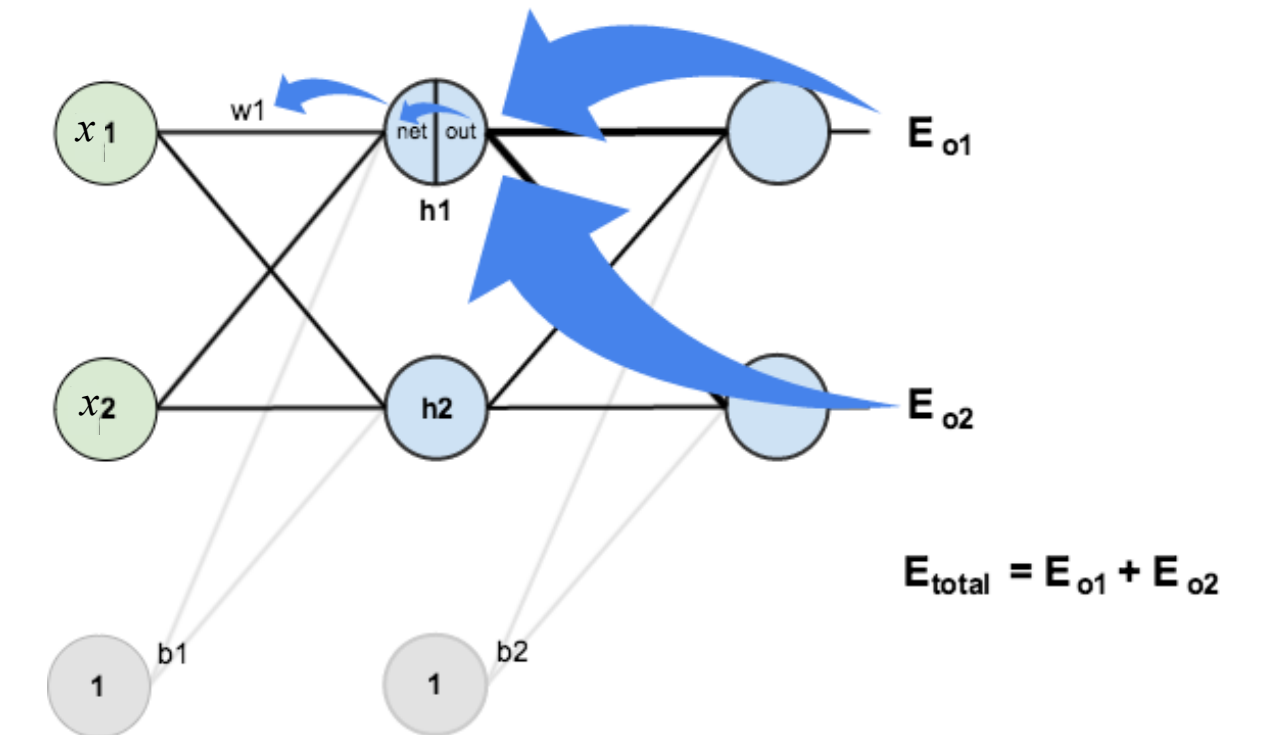
The Backward Pass

$$\frac{\partial E_{o2}}{\partial y_{h1}}$$

Following the same process for $\frac{\partial E_{o2}}{\partial y_{h1}} = -0.019049119$

we get:

Therefore:
$$\frac{\partial E_T}{\partial y_{h1}} = \frac{\partial E_{o1}}{\partial y_{h1}} + \frac{\partial E_{o2}}{\partial y_{h1}} = 0.055399425 + -0.019049119 = 0.036350306$$



A Step by Step Backpropagation Example

The Backward Pass - Hidden Layer

$$\frac{\partial E_T}{\partial w_1} = \frac{\frac{\partial z_{h1}}{\partial w_1} \frac{\partial y_{h1}}{\partial z_{h1}} \frac{\partial E_T}{\partial y_{h1}}}{\frac{\partial z_{h1}}{\partial w_1} \frac{\partial y_{h1}}{\partial z_{h1}} \frac{\partial E_T}{\partial y_{h1}}}$$

The Backward Pass - Hidden Layer

$$\begin{aligned} z_{h1} &= w_1 * x_1 + w_2 * x_2 + b_1 * 1 \\ \frac{\partial z_{h1}}{\partial w_1} &= i_1 = 0.05 \end{aligned}$$

$$\begin{aligned} y_{h1} &= \frac{1}{1 + e^{-z_{h1}}} \\ \frac{\partial y_{h1}}{\partial z_{h1}} &= y_{h1}(1 - y_{h1}) = 0.59326999(1 - 0.59326999) = 0.241300709 \end{aligned}$$

$$= 0.036350306 * 0.241300709 * 0.05 = 0.000438568$$

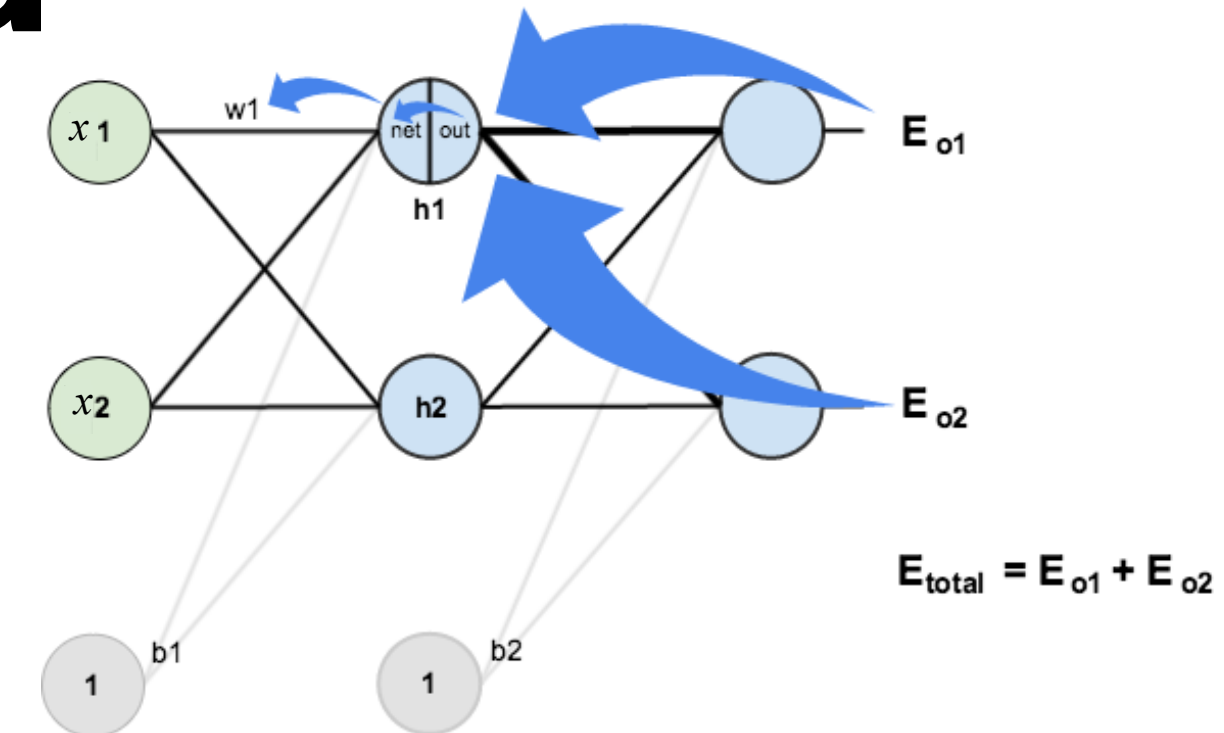
$$w_1^+ = w_1 - \eta * \frac{\partial E_T}{\partial w_1} = 0.15 - 0.5 * 0.000438568 = 0.149780716$$

Similarly,

$$w_2^+ = 0.19956143$$

$$w_3^+ = 0.24975114$$

$$w_4^+ = 0.29950229$$



Take Home - DIY

Derive the formula for

$$\frac{\partial E}{\partial w_2} = ?$$

$$\frac{\partial E}{\partial w_6} = ?$$

$$\frac{\partial E}{\partial w_3} = ?$$

$$\frac{\partial E}{\partial w_7} = ?$$

$$\frac{\partial E}{\partial w_4} = ?$$

$$\frac{\partial E}{\partial w_8} = ?$$

Find the value for

$$\Delta w_6$$

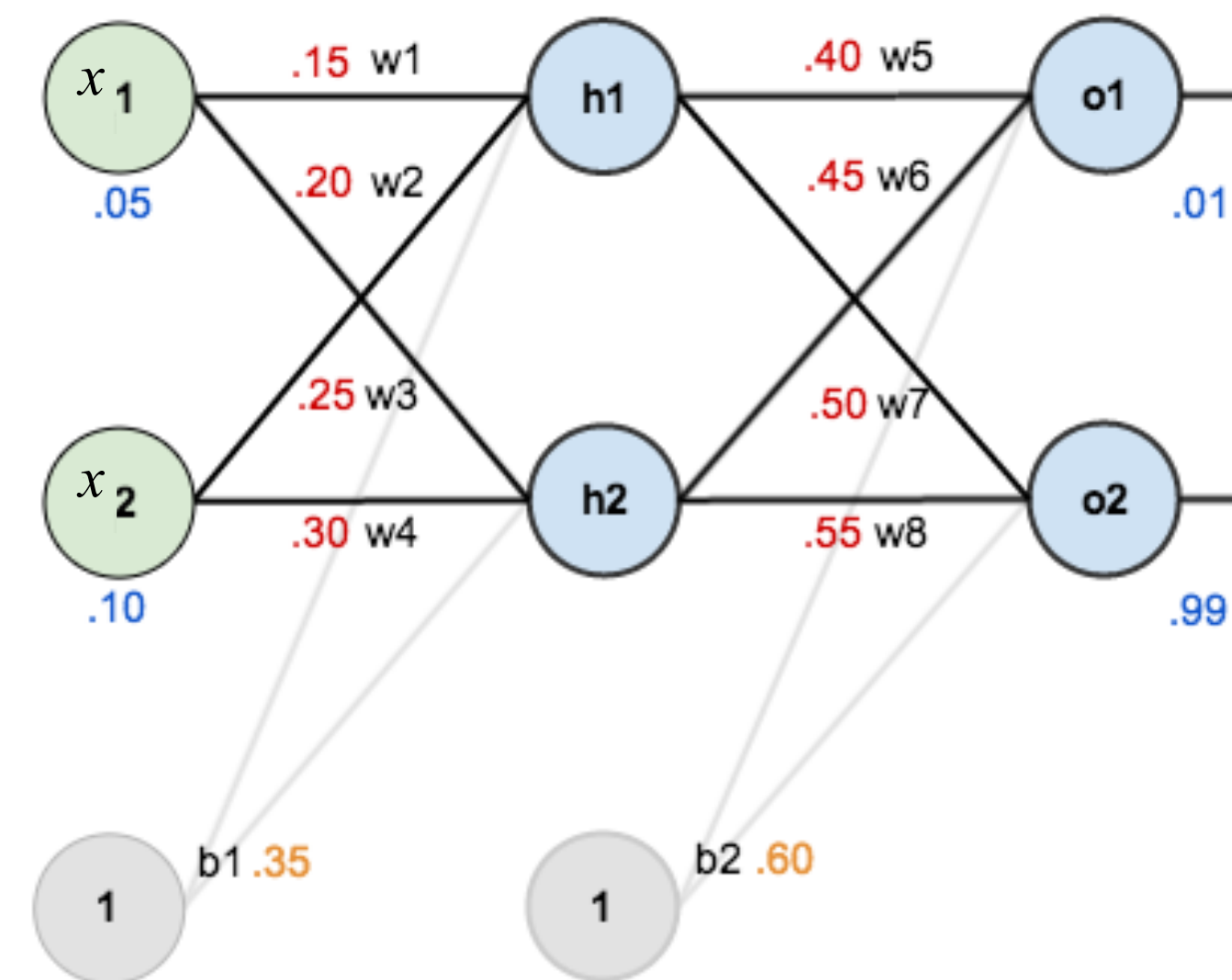
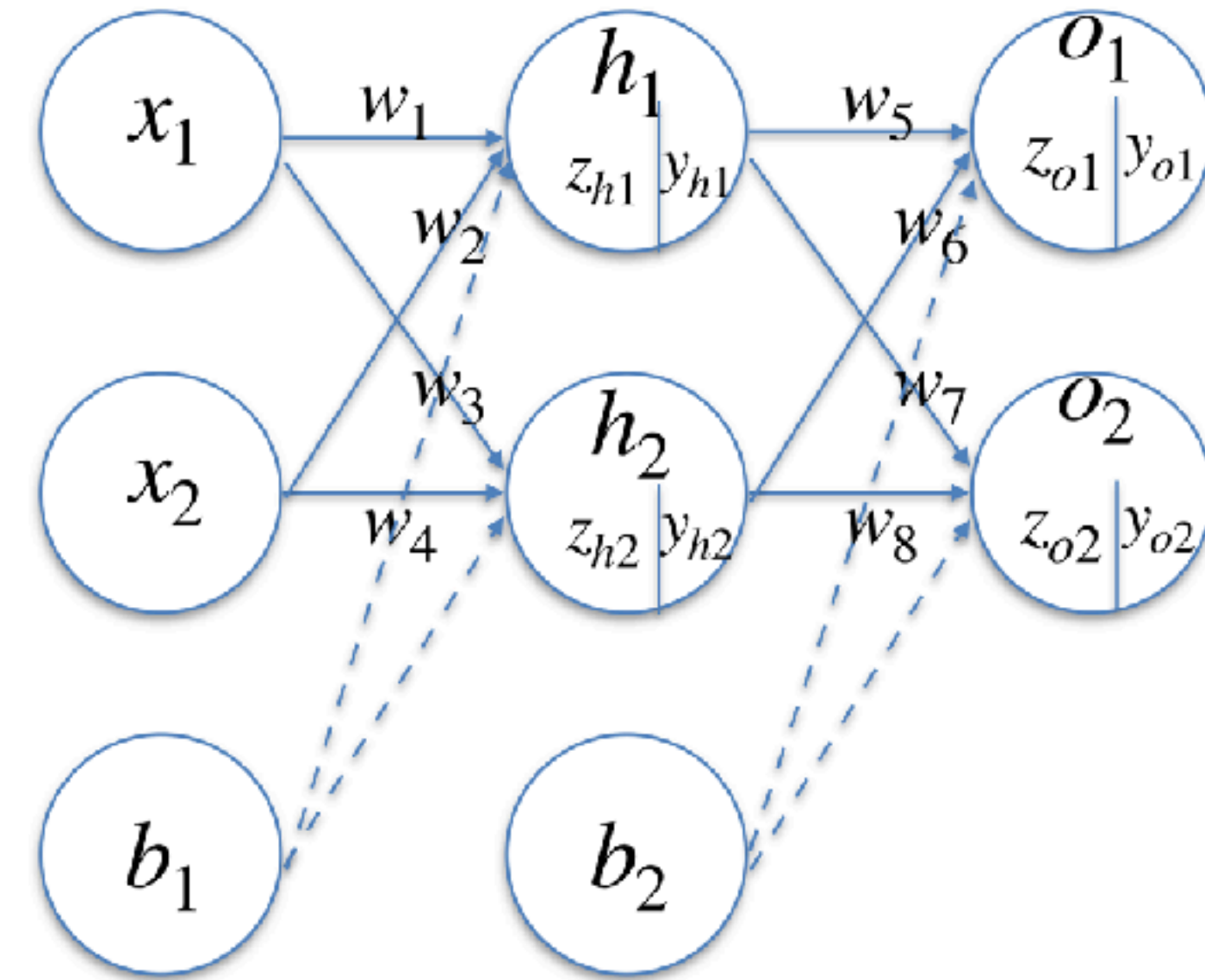
$$\Delta w_2$$

$$\Delta w_7$$

$$\Delta w_3$$

$$\Delta w_8$$

$$\Delta w_4$$



Readings



- Handwritten Notes on LMS
- Linear Algebra Background
 - https://www.deeplearningbook.org/contents/linear_algebra.html
- Probability background
 - <https://www.deeplearningbook.org/contents/prob.html>
- ML Background
 - <https://www.deeplearningbook.org/contents/ml.html>